

Logic Programming

Rodica Potolea (Eng)
Camelia Lemnaru (B)
Richard Ardelean (A)
CS @ UTCN

Lecture #0
Administrative Information
Computer Science

Administrative information

- English track
 - Rodica Potolea
 - Professor, Computer Science Department
 - Room C09 (Teams & Moodle)
 - Rodica.Potolea@cs.utcluj.ro
- Romanian B track
 - Camelia Lemnaru
 - Professor, Computer Science Department
 - Room M03 (Teams & Moodle)
 - Camelia.Lemnaru@cs.utcluj.ro
- Romanian A track
 - Richard Ardelean
 - Lecturer, Computer Science Department
 - Room M03 (Teams & Moodle)
 - Richard.Ardelean@cs.utcluj.ro

Structure of the course

- **Labs (f2f, Baritiu, ex BT building, 5th floor)**
 - same content, every group a different faculty member, or a (graduate) PhD student or master student
 - HANDS ON Problem solving (again: for ANY solution WHY? WHAT IF?)
 - Prolog
- **Seminars (f2f, Baritiu)**
 - Problem solving – analysis and design, evaluation, comparisons
 - Almost always asked: WHY? WHAT IF? With INTERPRETATION (answer **WITHOUT** running the code).
- **Lectures (f2f, Baritiu)**
 - Slides + code + analysis + discussions
 - Open course with Q&A sessions
 - Stop us and ask questions whenever you have
 - If you have a question, most probably other students have the same question! Ask it!
- **Slides + code + analysis + discussions**
 - <https://moodle.cs.utcluj.ro/course/view.php?id=844>
 - <https://github.com/ArdeleanRichard/utcn-pl/>

Lab sessions info

	Enrollment key (moodle)	TA
30231	2026_Group@30231	<i>Mihai Mircea Mera</i>
30232	2026_Group@30232	<i>Richard Ardelean</i>
30233	2026_Group@30233	<i>Vasile Suciu</i>
30234	2026_Group@30234	<i>Alex Lapusan</i>
30235	2026_Group@30235	<i>Remus Petrache</i>
30236_1	2026_Group@30236_1	<i>Alex Lapusan</i>
30236_2	2026_Group@30236_2	<i>Raluca Portase</i>
30237_1	2026_Group@30237_1	<i>Raluca Portase</i>
30237_2	2026_Group@30237_2	<i>Richard Ardelean</i>
30238	2026_Group@30238	<i>Alin Anghelus</i>
30431	2026_Group@30431	<i>Istvan Czarar</i>
30432	2026_Group@30432	<i>Vlad Negru</i>
30433	2026_Group@30433	<i>Remus Petrache</i>
30434_1	2026_Group@30434_1	<i>Vlad Negru</i>
30434_2	2026_Group@30434_2	<i>Mihai Feier</i>
30435	2026_Group@30435	<i>Raluca Portase</i>
Re	2026_Group@RE	-

References

- **No bible this time**
- **Instead, several references**
 - Moodle course page
 - <https://github.com/ArdeleanRichard/utc-n-pl/tree/main/resources/Books>

Go check the CS library (Baritiu 26-28, Room M04)



Evaluation (FG formula)

- **Laboratory works / Hands on evaluation (30% of FG)**
 - Attend laboratory sessions with *your* group
 - 12 lab works, must work during the lab sessions, must upload work on moodle
 - Activity during the semester will be part of your lab grade!
 - Final evaluation, at the end of the semester (written + oral)
 - need a lab grade of 5 or more to be allowed to take the final exam
 - **CANNOT retake** the lab exam
- **Midterm (20% of FG)**
 - In the 5th school week, during the course
 - **CANNOT retake** the midterm
- **FE (50% of FG)**
 - 1h examination (**Moodle**): problem solving only (NO THEORY)
 - This is for FG up to 8
 - Quiz-like questions, a mock quiz will be made available around week 7-8
 - Oral examination just for FG > 8
 - Based on prior enrollment
 - Permission granted based on lab and FE performance (TBD)

Final Grade formula

- **$FG = 30\%LG + 20\%MG + 50\%FEG + \text{Bonus}$**
 - **$FEG = 70\%MEG + 30\%OEG$**
- **Conditions to pass:**
 - **Preconditions:**
 - $LG \geq 5$
 - $MEG \geq 5$
 - $FG \geq 4.5$ (applicable only AFTER preconditions met)
- **BONUS** (awarded based on Seminar & Lectures Activity)
 - Apply a +[0 to 1] point

FG=Final Grade
MG=Midterm Grade
LG=Lab Grade
FEG=Final Exam Grade

MEG – Moodle Exam Grade
OEG – Oral Exam Grade

Logic Programming

Rodica Potolea (Eng)
Camelia Lemnaru (B)
Richard Ardelean (A)
CS @ UTCN

Lecture #1

Introduction to Logic Programming

Agenda

- **What this course is/is NOT about**
- **Logic in Programming**
- **Main LP elements**
- **LP program**
 - Syntax
 - Semantic
 - Execution tree

What this course is about?

- **Course on Logic Programming**
 - Define the LP **syntax** and **semantics**
 - Understand the **execution mechanism**
 - Learn the main **LP paradigms**
 - ***Language as executable specifications***
 - Practice in a **LP language** (Prolog and “dialects”)
- **General CS practice**
 - Basic **problem solving**
 - ***Revise Data Structures and Algorithms*** from the LP perspective

In a nutshell...

- Programming so far:
 - Imperative (procedural languages)
 - assumes the presence of an assignment ($:=$) operator
 - specific control structures used
 - presence of side effects
- Declarative languages (symbolic languages)
 - Functional and *logic* languages
 - Assignment operation NOT present
 - Control removed from programmer (*mostly*; in a pure LP style, all of it)
 - No side effects (*in pure LP*; specific mechanisms implementing side effects exist)

In a nutshell...

- Program = Algorithm + **Data Structure**
 - Wirth (book title)
- Algorithm = ***Logic*** + **Control**
 - Kowalsky's statement

LP philosophy

- Program:
 - In conventional programming languages - describing the algorithm using the language's syntax
 - In LP - describing formal relationships occurring among objects (here objects NOT in the OO meaning)
 - Focus on the descriptive perspective rather than the prescriptive one
 - A LP program = collection of known facts and relationships between components, rather than a sequence of steps to be taken
 - Computation is carried out by the specification (with logic embedded in the declarative semantic + new facts inferred) and just partially by explicit control information supplied by the program
- LP main characteristics:
 - **Logic variables**
 - **Shared structures**
 - **Unification** mechanism (matching)
 - Search for **alternative solutions** (when writing *nondeterministic* solutions)

LP syntax

- We'll use PROLOG to generically denote Logic Languages
 - PROLOG = PROgramming LOGic
 - For a brief history check references and [Prolog Heritage](#)
- NO keywords (just a very few; we'll learn them on the fly)
- NO statements
- The program is just what you write
- **Case sensitive** language
 - Variables – first letter UPPER case (*Variable*, *VARIABLE*, *VaRiAbLe* are all variables; *_variable* is constant)
 - Constants – first letter LOWER case (*constant*, *cONSTANT*, *cOnStAnT* are all constants; *_Constant* is variable)
- Matching basic operation (evaluation rare, just by **explicit request**)
 - Example: **A=a**
 - means A unifies (matches) with a and NOT that A takes the value of a
 - Unification = the attempt to make the two sides the same
 - Successful matching (TRUE) – they are the SAME from now on
 - Unsuccessful matching (FALSE or fail in Prolog jargon) – backtracking mechanism installed (see later)

LP syntax

- Program = collection of predicates
- Predicate = a description (in Prolog syntax) of theorems
 - a collection of clauses
 - name of a predicate = a constant (starts with lower case)
- The general form of a clause is represented by:

$$p(X) :- q_1(Y), q_2(X, Y), \dots, q_n(X, Z) .$$

(to be read as p of X if ... q1 of Y and ...)

$p, q_i, i=1, \dots, n$ represent names of predicates (first letter lower case)

X, Y, Z arguments (variables = start with upper case)

X = parameter

Y, Z = local variables (hidden variables; explained later)

$p(X)$ = head of the clause

$q_i, i=1 \dots n$ = body of the clause (sub-query)

$:-$ = if

,

= and (conjunction)

.

= full stop (end of clause). Dot is mandatory.

Prolog PREDICATES (clauses)

- The general form of a **clause** is represented by:

$$p(X) :- q_1(Y), q_2(X,Y), \dots, q_n(X,Z).$$

head :- body.

Left hand side (lhs) <IF> right hand side (rhs).

- Clause
 - a **theorem** = a representation in Prolog syntax of the relationships (body) among the objects (variables) occurring in the clause; it has a standalone meaning
 - Can be viewed as a procedure (function, method) from imperative (procedural, OO) languages
 - The conclusion of the theorem (head, lhs) is true if all the hypotheses of the theorem (rhs) are true
 - p - head
 - $q_i, i=1\dots n$ - body
 - Meaning: the head is true (p; lhs) **if** the body (rhs) is true ($q_i, i=1\dots n$; rhs)
- Predicate = a set of one or several clauses
- Prolog program = a set of one or several predicates

Prolog PREDICATES (rules, facts, queries)

- The general form of a **clause** is represented by:

$$p(X) :- q_1(Y), q_2(X, Y), \dots, q_n(X, Z).$$

- Particular form of clauses:
 - General form – above (**rule**)
 - Particular forms: **facts** and **queries**

- **Fact**

= a clause without body

= a theorem WITHOUT hypothesis = AXIOM (no need for proof)

$p(a).$ % meaning *p of a is true, without condition.*

- **Query**

= a clause without head

= a theorem WITHOUT conclusion = only hypotheses which need proof

= if true, will be a new axiom

$?- q_2(a, b).$ % meaning *check if q₂ of a and b is true.*

This defines the **declarative semantics** of Prolog = interpretation of the statements

•

•

•

•

•

•

The first LP program

```
man(john) .           %john, lower case start is a man.
.
.
.
parent(john,david) .  % john is david's parent
parent(john,edward) . % john is edward's parent
.                    % could be the other way around
.                    % david is john's parent
.                    % is up to us; depends on how WE define it!

father(X,Y) :-        % X is Y's father IF (by :- sign)
    man(X) ,          % X is man AND (by ,)
    parent(X,Y) .     % X is Y's parent. That's all.

?- father(john, Z).   % whose father is john? Return the result in Z.
```

Execution of the first LP program: Questions to answer

- How is it working?
 - Matching mechanism
 - Match the query against the head of the clause of the predicate with the same name (`father` in our case) = *query/head unification*
- Some questions need to be answered:
 - **Q1: what if query/head unification have several alternatives** (*translates into: predicate has several clauses*)
 - **Q2: what if query/head unification succeeds and we have a body** (*translates into: we have a complete clause we matched with*)
 - **Q3: what if query/head unification fails** (*translates into: we have an unsuccessful matching*)
- The answers define the **procedural semantics** of PROLOG = interpreting the statements as actions to take (call and run) (also named **operational semantics**)

Execution of the first LP program answers to questions

- **Q1:** what if query/head **unification** have several alternatives (translated: predicate with several clauses)
 - **R1:** first clause first = **top-down**
- **Q2:** what if query/head unification succeeds and we have a body (translated: complete clause)
 - **R2:** first subgoal (sub-body) first = **left-right**
- **Q3:** what if query/head unification fails (translated: unsuccessful matching)
 - **R3:** unification fails = **backtracking**
 - Backtracking is a build-in mechanism = it automatically launches in case of failure

LP operational semantics

- Defined by the answers from before:
 - (R1) **Selection** (computation) rule: the top/down approach
 - (R2) **Search** rule: the left/right approach
 - (R3) **Backtracking** by default

Execution of the first LP program

```
[m1] man(john) .  
...
```

```
[p1] parent(john,david) .  
[p2] parent(john,edward) .  
...
```

```
[f1] father(X,Y) :-  
[f11]     man(X) ,  
[f12]     parent(X,Y) .
```

```
[q] ?- father(john,Z) .
```

R1: top-down
R2: left-right
R3: backtrack upon failure

R1: (1) **q/f1:** father(john,Z)/father(X,Y) => X=john, Z=Y

It results the CONCLUSION could be true in case all conjunction in the body is true as well, the body of the clause becomes the NEW goal (or query) to execute.

So, the NEW goal is an instance of f11,f12, that is:

[f11],[f12] = man(X) , parent(X,Y) .

Where X and Y got instantiated already (X and Y are shared variables; shared as they are THE SAME as in the head of the rule)

R2 tells us f11 gets executed before f12; moreover, f12 is executed **iff** f11 is TRUE.

Execution of the first LP program – contd.

R1: top-down

R2: left-right

R3: backtrack upon failure

?- man(X), parent(X, Y) . % **f11, f12**

a2 tells us to *SOLVE f11 and PUT on hold f12*

(what happens: the entire query is put on the exe. stack, f12 on bottom, f11 on top; the exec. pops the top of the stack)

f11: ?- man(X) . is the new query to execute => Q1 => **a1** =>

(2) f11/m1: man(X) /man(john) => X=john succeeds.

Is a fact => unconditionally true => f11 IS TRUE.

(*) In a query/rule match, HOW DO WE CONTINUE? We must check 3 conditions (for a **final** successful unification) in the specific order:

- **C1: current goal succeeds?**
- **C2: is a fact?**
- **C3: is the execution stack empty?**

Continuation depends on how the conditions are (not) met.

(remember: the execution stack contains the goals to execute)

Execution mechanism – conditions to check

- C1: current goal succeeds? *If yes, go C2,
else, backtrack.*
- C2: is it a fact? *If yes, go C3,
else, push the entire body on the
execution stack, and pop stack's top.*
- C3: is the stack empty? *If yes, over (the whole execution ends
successfully),
else pop the top of the execution stack
and continue execution (from C1)*
- Backtrack means WHAT?
 - UNDO the latest unification (that is, REMOVE latest matching) and
 - for the given goal, try a different alternative (next one, if any)
 - If again fail (or no other alternative), the backtracking mechanism propagates in REVERSE order of the execution.

Execution of the first LP program – contd.

?- man (X) , parent (X, Y) . % **f11, f12**

Solved f11; so?

- C1: current goal succeeds?
- C2: is a fact?
- C3: is the stack empty?

YES => unification succeeds =>
successful node in the
execution tree

YES => node is a leaf in the tree

NO => pop the top and it becomes the
right sibling of the latest node

In our case, pop -> **[f12]** ?- parent (X, Y) . is the new query to execute =>
Q1 => A1 =>

(3) f12/p1: parent (X, Y) / parent (john, david) .

BUT, from the previous query we had => X=john, and X is shared (among the body and head), thus here we actually have:

john=john (T) and Y=david (T)

So, successful matching.

Execution of the first LP program – contd.

f12 = parent (X, Y) .

Solved f12; so?

- C1: current goal succeeds? YES => unification succeeds => successful node in the execution tree
- C2: is a fact? YES => node is a leaf in the tree
- C3: is the stack empty? YES => the whole execution ends successfully

So, successful matching. Where is the answer?

Following the deduction tree, we have the entire unification chain (next slides)

Computer Science

Here goes the deduction tree

```
[m1] man(john) .
```

```
...
```

```
[p1] parent(john,david) .
```

```
[p2] parent(john,edward) .
```

```
...
```

```
[f1] father(X,Y) :-
```

```
[f11]     man(X) ,
```

```
[f12]     parent(X,Y) .
```

```
[q] ?-father(john,Z) .
```

Here goes the deduction tree

```
[m1] man(john) .
```

...

```
[p1] parent(john,david) .
```

```
[p2] parent(john,edward) .
```

...

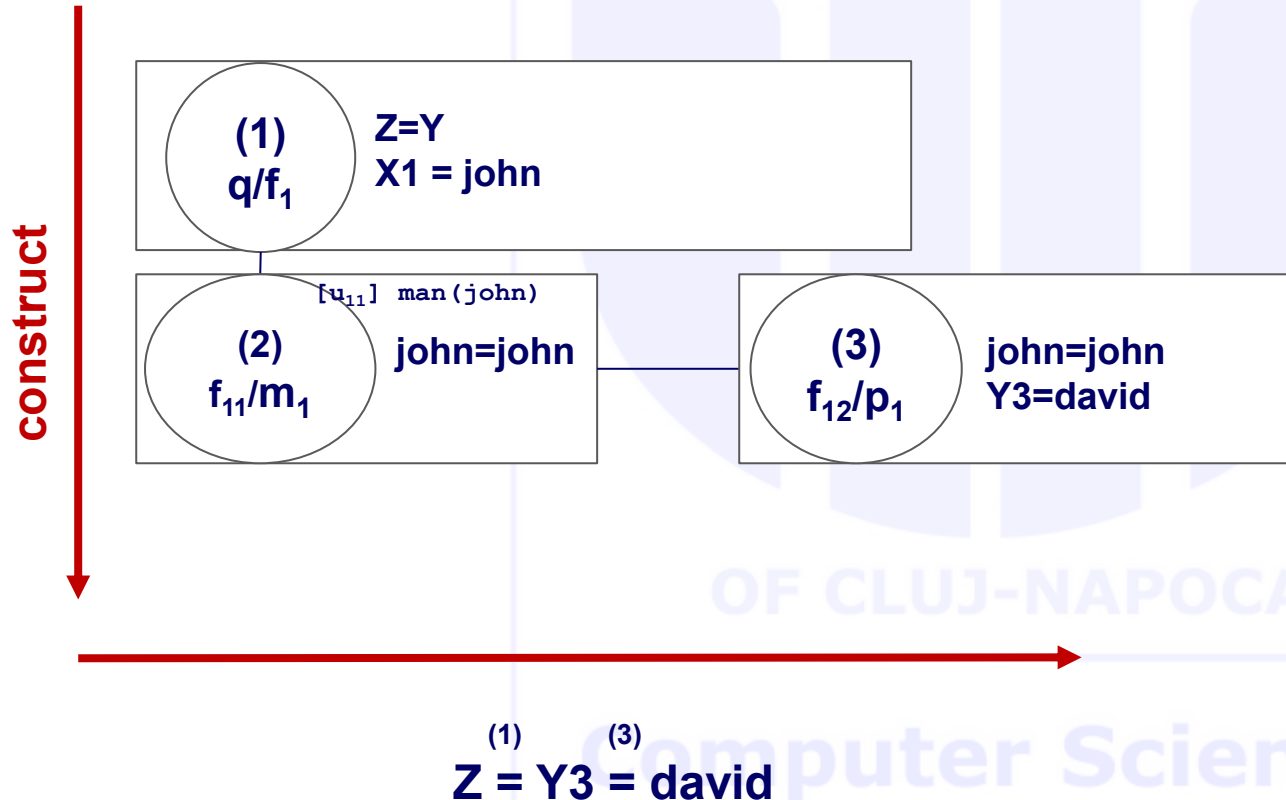
```
[f1] father(X,Y) :-
```

```
[f11]     man(X) ,
```

```
[f12]     parent(X,Y) .
```

```
[q] ?- father(john,Z) .
```

true, Z=david



Deduction Tree

- The final **successful** shape of an execution tree forms the **deduction** tree
 - Execution tree contains all nodes – even erased ones
- It stores the query/head matchings along with the unifications (formal vs actual arguments) made during the execution
- **Nondeterministic** behavior of Prolog = ability to find alternative solutions (if any) of the problem at hand
 - It is performed via launching the **backtracking** mechanism (built-in)
 - It refers to **repeating** the query; repeating the query is NOT asking the SAME query (if so, the SAME answer is provided)
 - repeating the query is: for the SAME query AND in the context of the ALREADY built deduction tree, what OTHER possible answers are there?
 - Syntactically, it is specified by “;” (at command prompt), and it says: in the tree you already have, take the nodes in reverse order and try to find an alternative matching
 - So, the LAST built node attempts to backtrack and only after it has no other choice it asks its predecessor node to backtrack.

Backtracking

- Evolution backwards in a tree already built
- The last built, the first to backtrack
- We had: (1)

(1) q/f1

(2) f12/m1

(3) f12/p1 backtrack on this =>

ERASE THE NODE and try **(3') f12/p2** `parent(X,Y) / parent(john,edward) .`

BUT, from the previous query (2) we had => `X=john`, and `X` is shared (among the body and head), thus here we actually have:

`john=john (T) and Y=edward (T)`

So, successful matching.

Following the new deduction tree we have the whole unification chain:

Here goes the deduction tree

```
[m1] man(john) .
```

...

```
[p1] parent(john,david) .
```

```
[p2] parent(john,edward) .
```

...

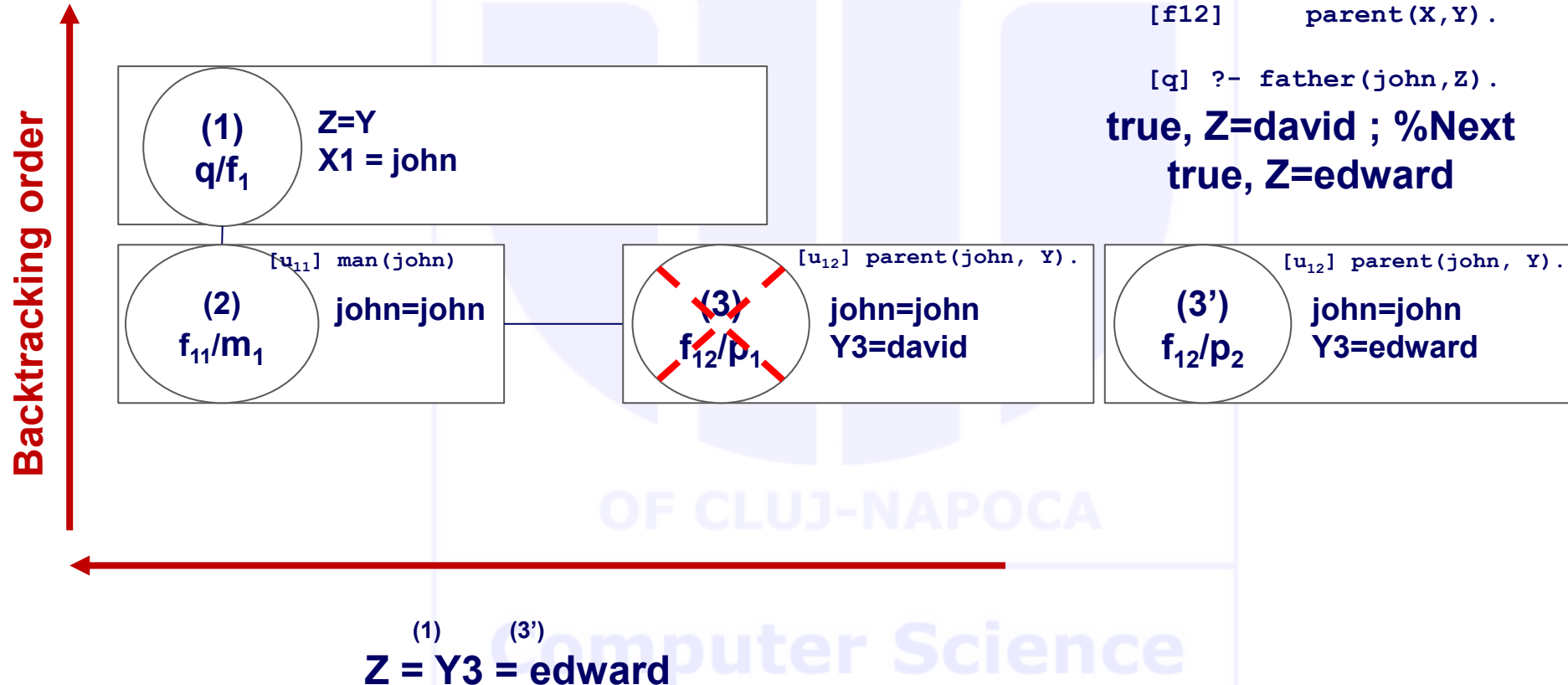
```
[f1] father(X,Y) :-
```

```
[f11]     man(X) ,
```

```
[f12]     parent(X,Y) .
```

```
[q] ?- father(john,Z) .
```

```
true, Z=david ; %Next
true, Z=edward
```



Backtracking until final failure

- If we need ANOTHER answer, the evolution backwards in the new shape of the tree (1, 2 and 3')
- The last built, the first to backtrack
- We had:

(1) q/f1

(2) f12/m1

(3') f12/p2 backtrack on this => ERASE THE NODE and try **(3'') f12/p3**

Assume p3 has as first argument something different from j_{ohn}

(3'') fails to build, (3'') is erased and the backtracking propagates on to 2:

(2') f12/m2 fails (as m2 has something different from j_{ohn} , and 2 constants unify iff they represent the same constant)

(2') fails to build, (2') is erased and the backtracking propagates on to 1:

(1') q/? No other clause exists => FINAL FAILURE

Meaning: NO other solution exists

Action: the whole tree is removed from memory (*go to the prev page to see*)

Execution mechanism – review

- Rule 1: **Search** rule:
the left to right evaluation of goals in conjunction
- Rule 2: **Selection** (computation) rule:
the top-down selection of clauses from the program
- Rule 3: **Backtracking** by default:
(see later lectures mechanisms for avoiding it when not necessary)

OF CLUJ-NAPOCA

Computer Science

First conclusions

- a Prolog **program** is a set of **theorems** (complete clauses) and axioms (facts)
- Executing a Prolog program means **proving a new theorem** (the query/goal) from the existing ones (program)
- The execution relies on **solving** a set of **systems of linear equations**
- The number of systems = number of nodes in the deduction tree
- Each system has a number of equations equal to the number of arguments of the goal executed in the corresponding node

List representation and decomposing lists

Lists: $[1,2,3]$ – comma separated elements

- NO type required (heterogeneous lists by default)
- $[a,b,c]$ how is it different from $[A,B,C]$?
- $[]$ empty list
- $[H|T]$ Head/Tail decomposition pattern with meaning:
 - H = the first element of the list
 - T = the rest of the list (= the list with the first element removed)
- $[H|T] = [a,b,c]$ unification/matching
 - Before, H, T free variables
 - Now, bound to specific constants: $H=a, T=[b,c]$
- $[H|T] = [a]$
 - $H=a, T=[]$
- $[H|T] = []$
 - Fails, as H CANNOT unify the first element of the list (as there is NO element at all)
- $[H1,H2|T] = [H1|[H2|T]]$ just rewriting rule
- In **LP basic operator is matching**
(as opposed to FP where evaluator is the basic operation)

Example 2 – concatenating lists

```
append(L1, L2, R) :-  
    L1 = [],                % L1 empty  
    R = L2.                % R gets L2  
  
append(L1, L2, R) :-  
    L1 = [H1 | T],          % decomposition pattern; dec L1 into head and tail  
    append(T, L2, Rp),      % recurse on tail to get the partial result  
    H = H1,  
    R = [H | Rp].           % recomposition pattern; rec R from head and tail
```

Order of clauses?

impact on correctness? Why?

impact on efficiency? How?

Example 2 contd

```
append(L1, L2, R) :-  
    L1 = [],                % L1 empty  
    R = L2.                % R gets L2  
  
append(L1, L2, R) :-  
    L1 = [H1 | T],          % decomposition pattern; dec L1 into head and tail  
    append(T, L2, Rp),      % recurse on tail to get the partial result  
    H = H1,  
    R = [H | Rp].           % recomposition pattern; rec R from head and tail
```

Here we have explicit unifications.

By replacing them with **default** unifications, we get:

```
append([], L, L).  
append([H | T], L, [H | R]) :-  
    append(T, L, R).
```

Example 2 execution

```
[a1] append([],L,L).  
[a2] append([H|T],L,[H|R]):-  
[a21]         append(T,L,R).  
[q]  ?- append([1,2],[3,4],Result).
```

Here goes equations and tree.

(1) **q/a1** $[1,2]=[]$ fails, default backtracking

Example 2 execution

```
[a1] append([],L,L) .  
[a2] append([H|T],L,[H|R]) :-  
[a21]         append(T,L,R) .  
[q]  ?- append([1,2],[3,4],Result) .
```

Here goes equations and tree.

(1) **q/a1** $[1,2]=[]$ fails, default backtracking

(1') **q/a2** $[H1|T1]=[1,2] \Rightarrow H1=1, T1=[2]$

$L1=[3,4]$

$Result=[H1|R1] \Rightarrow Result=[1|R1]$

a21: $append(T1,L1,R1)$

Example 2 execution

```
[a1] append([],L,L) .
[a2] append([H|T],L,[H|R]) :-
[a21]         append(T,L,R) .
[q]  ?- append([1,2],[3,4],Result) .
```

Here goes equations and tree.

```
(1)  q/a1  [1,2]=[ ] fails, default backtracking
(1') q/a2  [H1|T1]=[1,2] => H1=1, T1=[2]
      L1=[3,4]
      Result=[H1|R1] => Result=[1|R1]
a21: append(T1,L1,R1) (why?) and translates into append([2],[3,4],R1)
(2)  a21/a1 fails
(2') a21/a2 [H2|T2]=[2] => H2=2, T2=[ ]
      L2=[3,4]
      R1=[H2|R2] => R1=[2|R2]
a21: append(T2,L2,R2)
```

Example 2 execution

```
[a1] append([],L,L) .
[a2] append([H|T],L,[H|R]) :-
[a21]         append(T,L,R) .
[q]  ?- append([1,2],[3,4],Result) .
```

Here goes equations and tree.

```
(1)  q/a1  [1,2]=[]  fails, default backtracking
(1') q/a2  [H1|T1]=[1,2] => H1=1, T1=[2]
      L1=[3,4]
      Result=[H1|R1] => Result=[1|R1]
a21: append(T1,L1,R1) (why?) and translates into append([2],[3,4],R1)
(2)  a21/a1 fails
(2') a21/a2 [H2|T2]=[2]   => H2=2, T2=[]
      L2=[3,4]
      R1=[H2|R2]   => R1=[2|R2]
a21: append(T2,L2,R2) translates into append([], [3,4],R2)
(3)  a21/a1 []=[] L3=[3,4] R2=L3
      Result=[1|R1]=[1|[2|R2]]=[1,2|R2]=[1,2|L3]=[1,2|[3,4]]=[1,2,3,4]
```

TECHNICAL UNIVERSITY

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #2

Execution trees

OF CLUJ-NAPOCA

Computer Science

Agenda

- **LP paradigms – review**
- **Operational Semantic - review**
- **Execution tree**
 - Representation
 - Mechanism

Prolog PREDICATES

(rules, facts, queries)

- **Clause** general form:

$p(X) :- q_1(Y), q_2(X, Y), \dots, q_n(X, Z) .$
= a **THEOREM** in form "conclusion if hypotheses"
meaning $q_1 \wedge q_2 \wedge \dots \wedge q_n \rightarrow p$

- **Fact**

$p(a) .$

= a clause without body

= a theorem WITHOUT any hypothesis = **AXIOM** (no need for proof)

- **Query**

$?- q_2(a, b) .$

= a clause without head

= a theorem WITHOUT conclusion

They define the **declarative semantics** of Prolog = interpretation of the statements

Execution of LP programs

Unification mechanism – the core of LP

- **Q1:** For the query/head unification take
 - A1: first clause first = top-down
- **Q2:** In the query/head successful unification the body becomes the new goal. In a conjunction of goals
 - A2: first subgoal (sub-body) first = left-right
- **Q3:** In the query/head failed unification
 - A3: unification fails = backtracking

Typical predicate (has exceptions):

```
/*general<OR>*/    p:- ... .  
                  p:- ... .  
/*specific<OR>*/   p:- q /*specific<AND>*/, ..., t /*general<AND>*/.
```

Operational semantics in action

When building the tree, we start from the initial query in the matching process.
For each (sub)goal (one at a time):

- C1: current goal succeeds? If *yes*, current node built successfully & go C2, *e/se*, backtrack.
- C2: does it match a fact? If *yes*, current node is a leaf & go C3, *e/se*, push the entire body on the execution stack, and pop stack's top.
- C3: is the stack empty? If *yes*, over (the whole execution ends successfully, the execution tree becomes at this very moment the deduction tree), *e/se* pop the top of the execution stack & go C1

First conclusions

- a Prolog **program** is a set of **theorems** (complete clauses) and **axioms** (facts)
- Executing a Prolog program means **proving a new theorem** (the query/goal) from the existing ones (program)
- The execution relies on **solving** a set of **systems of linear equations**
- The number of systems = number of nodes in the deduction tree
- Each system has a number of equations equal to the number of arguments of the goal executed in the corresponding node

Main elements of Prolog

Built-in predicate call	Outcome
var(X)	if X unbound then T else F
nonvar(X)	if X bound then T else F
atom(X)	if X constant then T else F
integer(X)	if X integer then T else F
atomic(X)	if X bound and not compound then T else F
call(X)	executes X , where X is the name of a predicate used in metaprogramming, where X gets instantiated at runtime

Main elements of Prolog – contd.

- = infix operator (*equality*)
 - Prolog attempts to match (unify) the *lhs* with *rhs*
 - if successful, they are bound together from this point on
 - an un-instantiated var will become equal to ANYTHING as the unification succeeds
- == infix operator (*identity*)
 - if $X == Y$ then $X = Y$
 - if $X = Y$ then $X == Y$ is NOT MANDATORY!!!
 - an uninstantiated var will become identical to another uninstantiated variable ONLY if they are already sharing same location, otherwise FAILS
 - is unifiable (does **not** actually do the unification):
 - ?- not(not($X = Y$)).

Examples

?- X==Y .

false (fails; even if both X and Y are free variables)

?- X==X .

true, X=_some_number

?- X=Y, X==Y.

true, X=_some_number; Y =_some_number (SAME)

?- X=[a,b], Y=[a,b], X==Y. [in STANDARD Prolog]

false (although same list [a,b] is JUST same content, they are NOT shared in memory, NOT same location)

?- X=[a,b], Y=[a,b], X==Y. [in SWI Prolog]

yes (same list [a,b] → same content, constants whether atomic or compound are considered identical)

?- X=[a,b], Y=X, X==Y.

true

Examples

is infix operator; *rhs* gets evaluated and the result instantiates the *lhs* (if successful) or else fails. ONLY *lhs* may get instantiated.

X **is** some_expression

?- X **is** 2+3.
true, X=5.

?- X=2+3.
true, X=2+3.

?- 2+3 **is** X.

Arguments are not sufficiently instantiated

?- X **is** (2+2)/(4*1).
true, X=1.

?- X=2+3, Y **is** X.
true, X=2+3, Y=5.

Examples

is infix operator; *rhs* gets evaluated and the result instantiates the *lhs* (if successful) or else fails. ONLY *lhs* may get instantiated

X **is** some_expression

?- X **is** Y+1.

Arguments are not sufficiently instantiated

?- Y=2, X **is** Y+1.

true, X=3, Y=2.

?- X=Y+1, Y=2, Z **is** X.

true, X=2+1, Y=2, Z=3.

Matching rules

Goal (q)	Rule head (r)	Outcome
?- p(a).	p(b).	fails
?- p(a).	p(a).	succeeds
?- p(X).	p(a).	succeeds - X gets instantiated to a
?- p(a).	p(X).	succeeds - X gets instantiated to a
?- p(Q).	p(R).	succeeds - Q and R become the same

Execution tree representation

Tree built top-down and left-right (DFS)

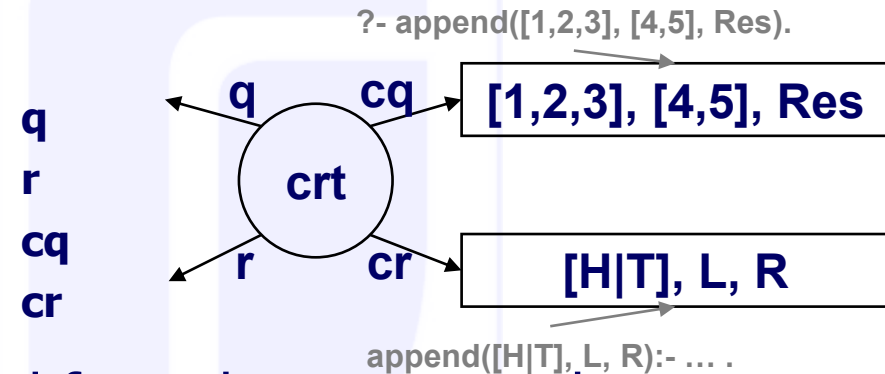
- Is a multiway tree (represented as binary)
- Each node in the exe tree contains **pointers** to several structures
 - Data: Program and memory locations of the variables (**1**)
 - Linking: connections in the tree (to navigate during execution, including backtracking) (**2**)
- Several **actions** to build the tree (**3**)

Computer Science

Execution tree: Data pointers (1)

A set of **4 pointers** to the:

- **query (goal)**
- **head of the rule**
- **context of the query (goal)**
- **context of the rule**

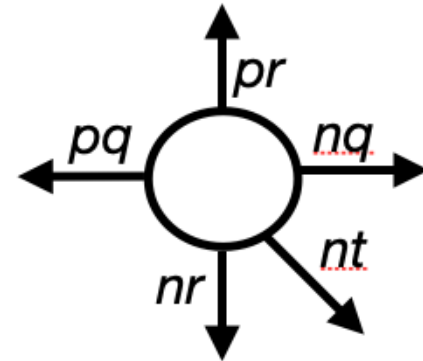


- The **query context (cq)** inherited from the parent node
 - *rule context of the parent node becomes query context of the child node* (it may be enhanced by the query context of a brother to the left due to hidden variables, or enhanced with new variables here)
 - variables of the query context are called **actual parameters**
- The **rule context (cr)** is allocated at the level of the current node
 - *ALL **formal variables** are indexed with the number of the node* (to keep track of the variable occurrence)

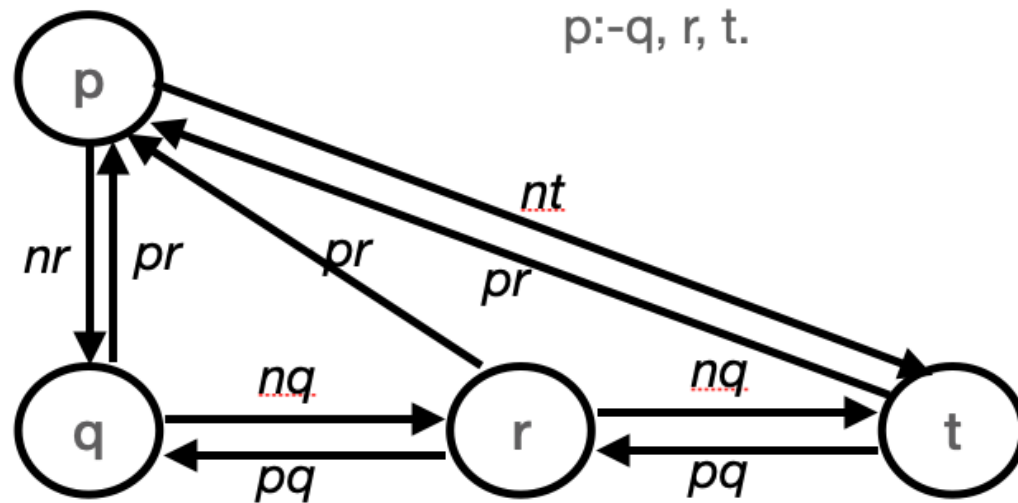
Execution tree: Linking pointers (2)

A set of **5 pointers** to other nodes in the tree (they provide the ability to make the sound actions according to the operational semantics):

- **parent rule** **pr**
 - (parent node; for root, null)
- **previous query** **pq**
 - (sibling node to the left; for first child, null)
- **next rule** **nr**
 - (first child, if any; for leaves, null)
- **next query** **nq**
 - (sibling node to the right; for last child, null)
- **next try** **nt**
 - (the next alternative to try in case of failure/backtracking; points to the rightmost child, to provide fast reachability to the last node in the tree)

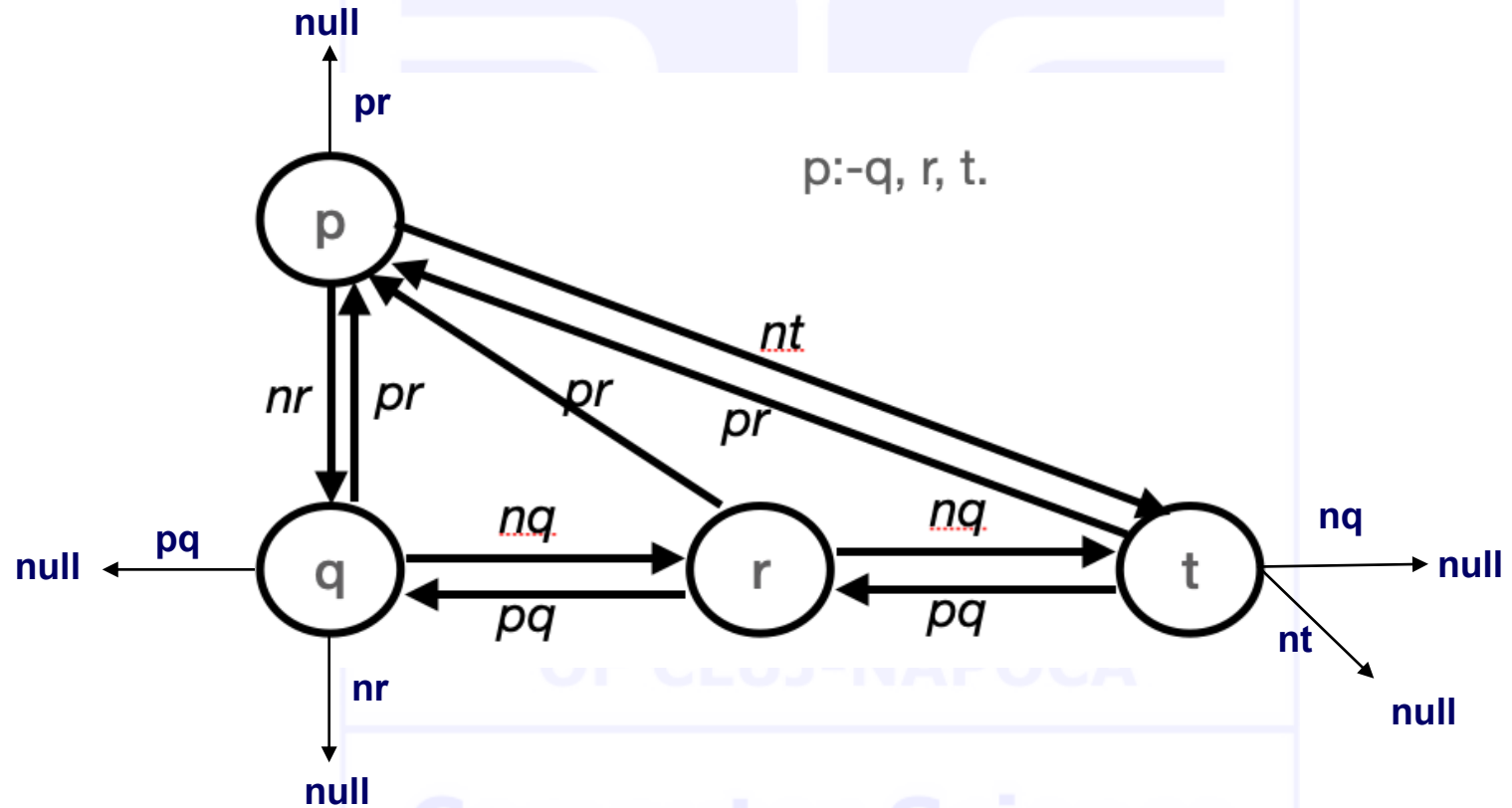


Execution tree: Linking pointers (2)

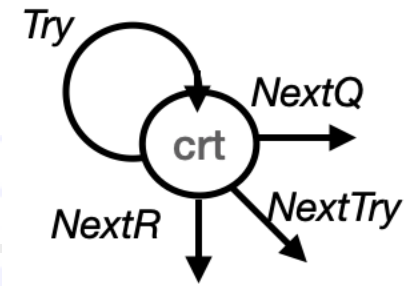


Computer Science

Execution tree: Linking pointers (2)



Execution tree: Actions (3)



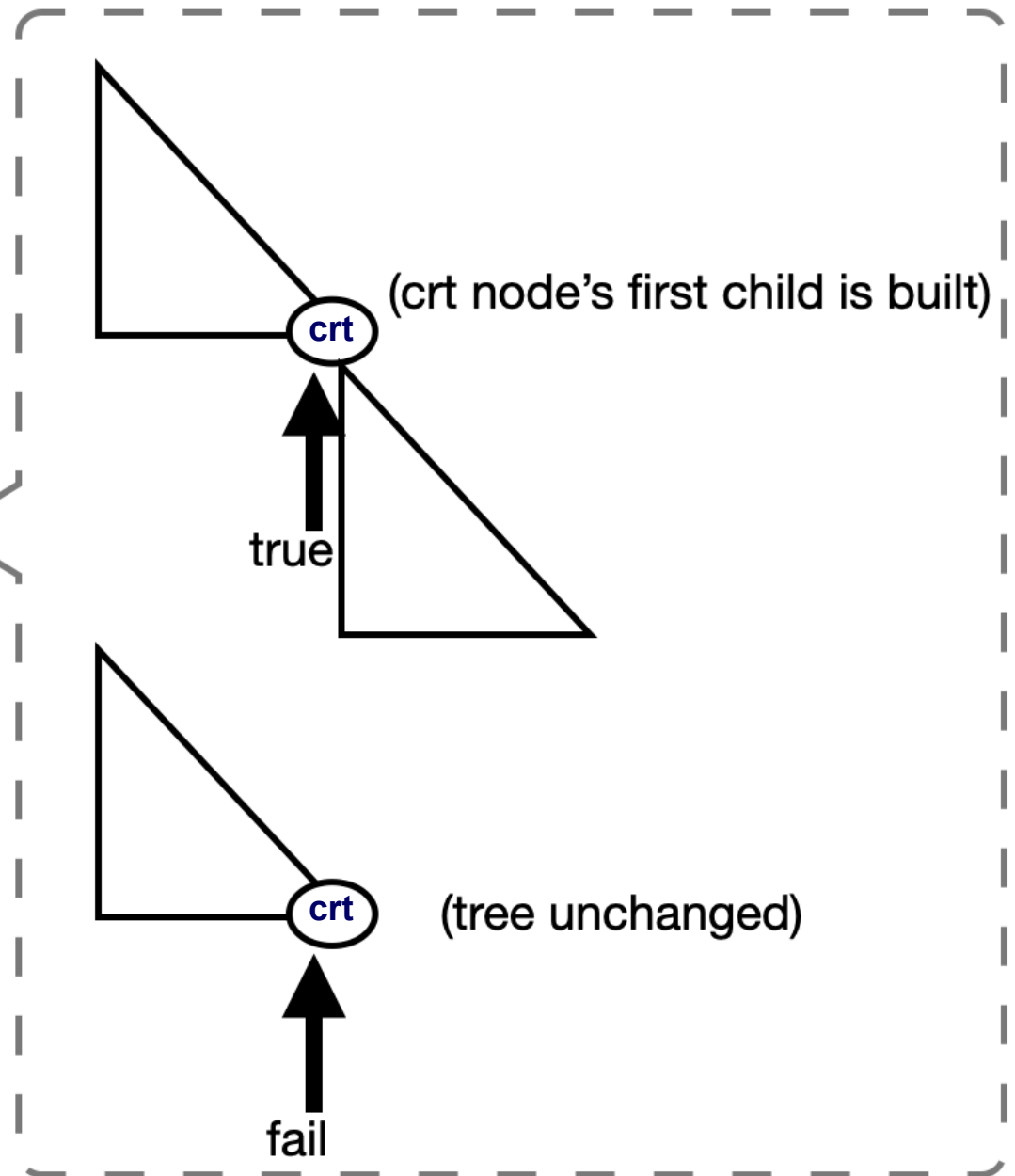
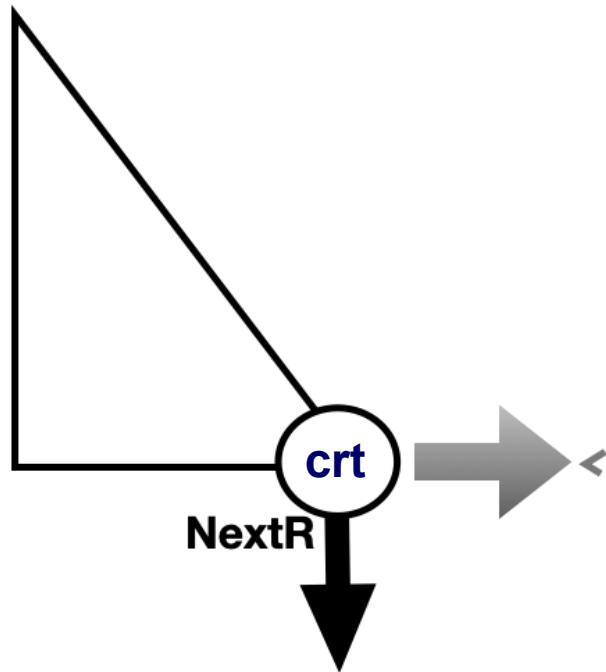
- **Try** - tries the definition of the predicate with the same name as the query (represents the matching between a query and the head of the rule)
 - the action BUILDS the **current node**
- **NextR** - enters the body of the clause at the first subgoal in the body, passing the conditional (**$:-$**)
 - the action BUILDS the **first child** of the current node
- **NextQ** - continues the body of the clause at the next subgoal in the body, passing a conjunction (**$,$**)
 - the action BUILDS the **first sibling to the right** of the current node
- **NextTry** – determines the first node to backtrack and launches the backtracking mechanism
 - the action takes place in a tree already built, by trying an alternative solution for the node identified as responsible to backtrack (examples during seminars)

Execution tree: Pointers and Actions – NextR

NextR

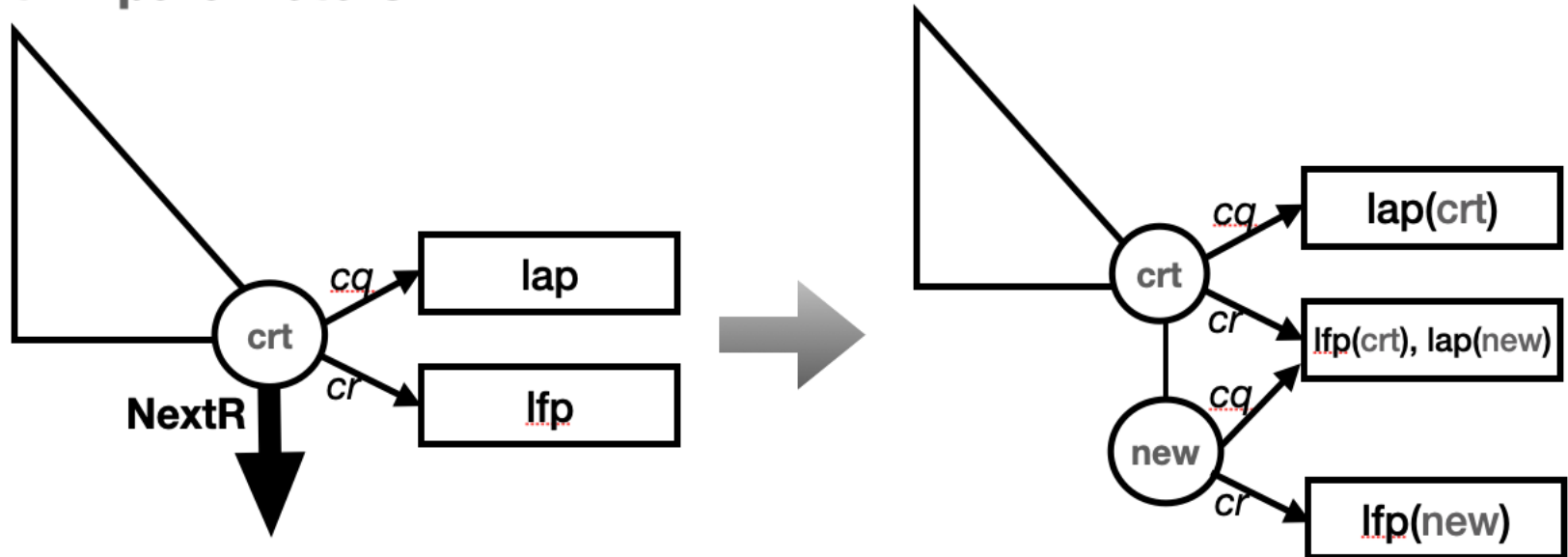
- At least one *new* node is generated, the left child of the current node
- If it (*new* node; first child of current) matches a fact, it is a leaf
- If matches a complete rule, the entire tree rooted by it (*new* node; first child of current) is generated
- the list of **actual parameters** of the new node is **inherited** from the list of formal parameters of the current node
- The list of **formal parameters** of the new node is **allocated now** (thus, the corresponding variables are indexed with the number of the node)

NextR



NextR – how arguments evolve

NextR - parameters



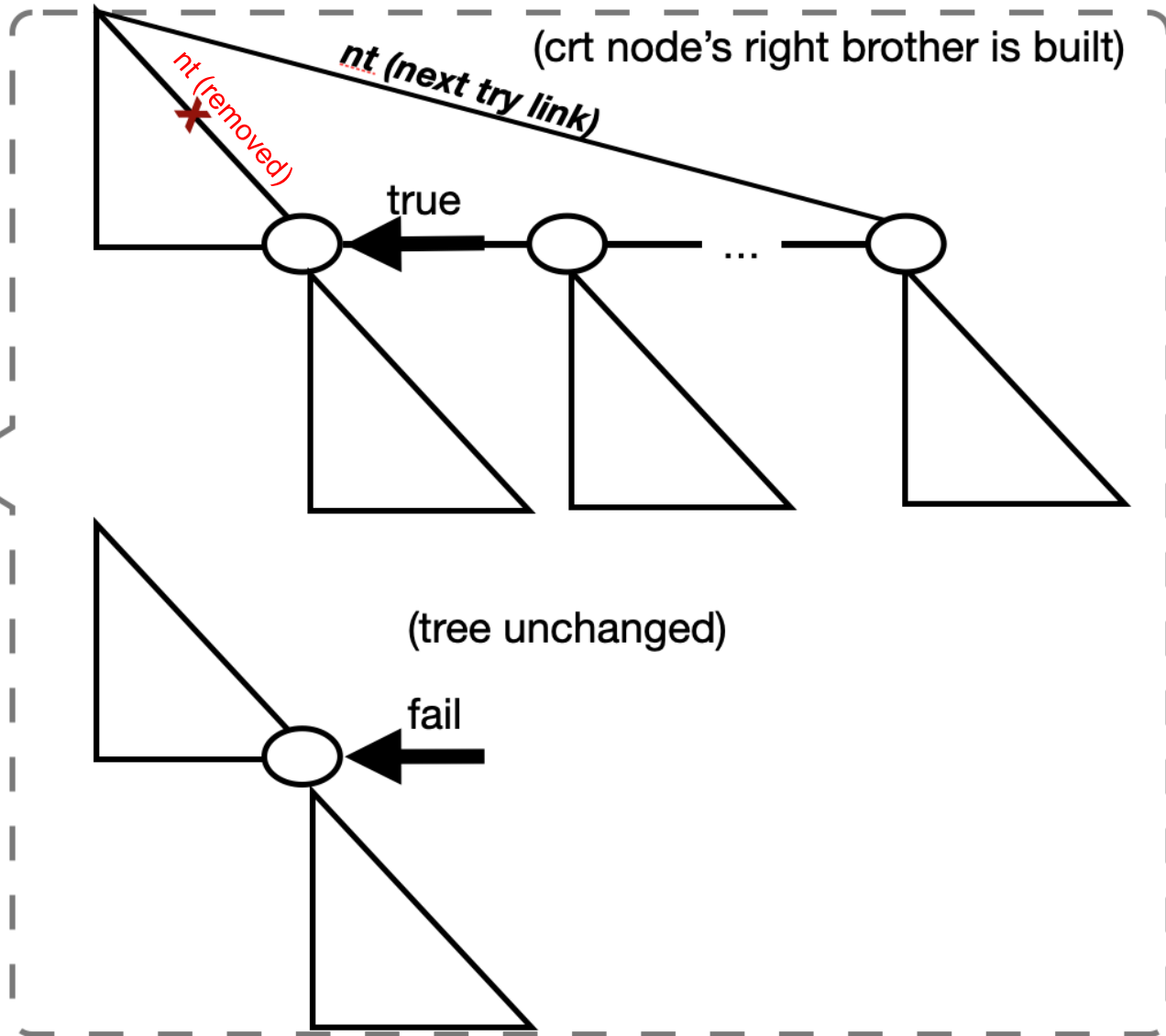
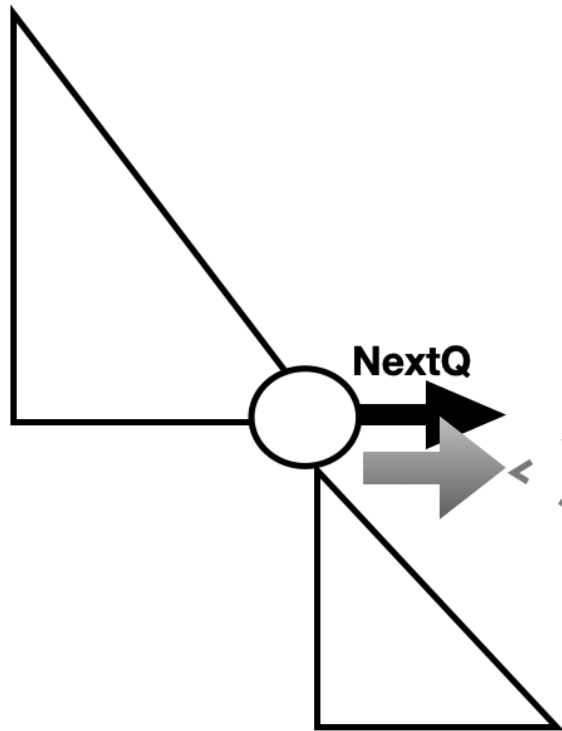
- the list of **actual parameters** of the new node is **inherited** from the list of **formal parameters** of the current node
- the list of **formal parameters** of the new node is **allocated** now (thus, the corresponding variables are indexed with the number of the node)

Execution tree: Pointers and Actions – NextQ

NextQ

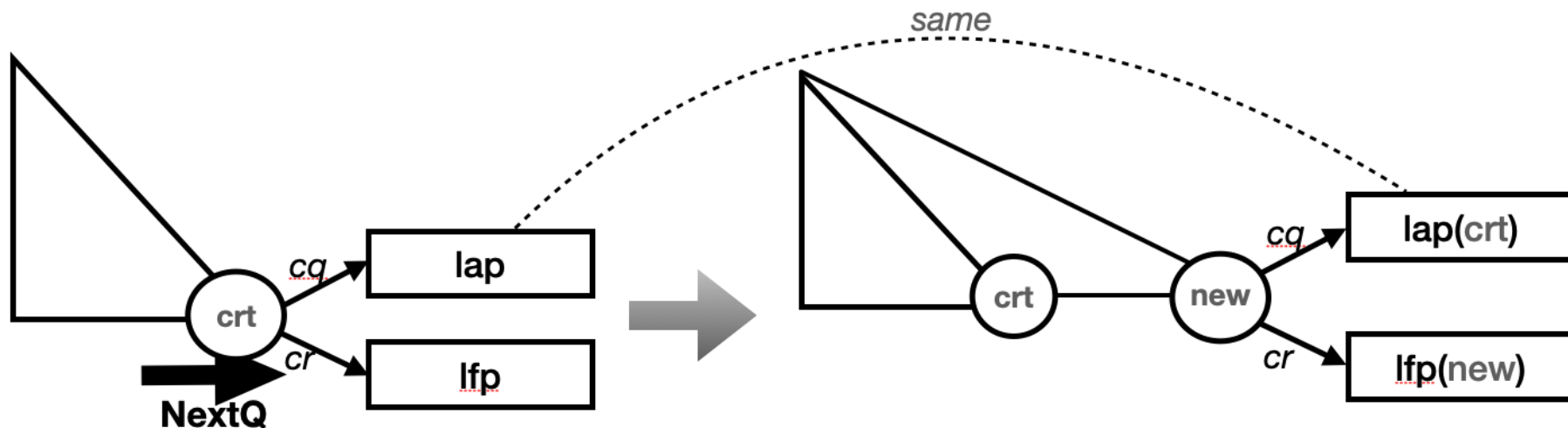
- At least one *new* node is generated, the right brother of the current node
- If it (*new* node; right brother of current)
 - matches a fact, it is a leaf;
 - else the entire tree rooted by is generated
- If it is NOT the rightmost sibling (NOT the last in conjunction, NOT before), all its right siblings (with subtrees) are generated when completed
- the list of **actual parameters** of the new node is **inherited** from the list of actual parameters of the current node
- The list of **formal parameters** of the new node is **allocated now** (thus, the corresponding variables are indexed with the number of the node)

NextQ



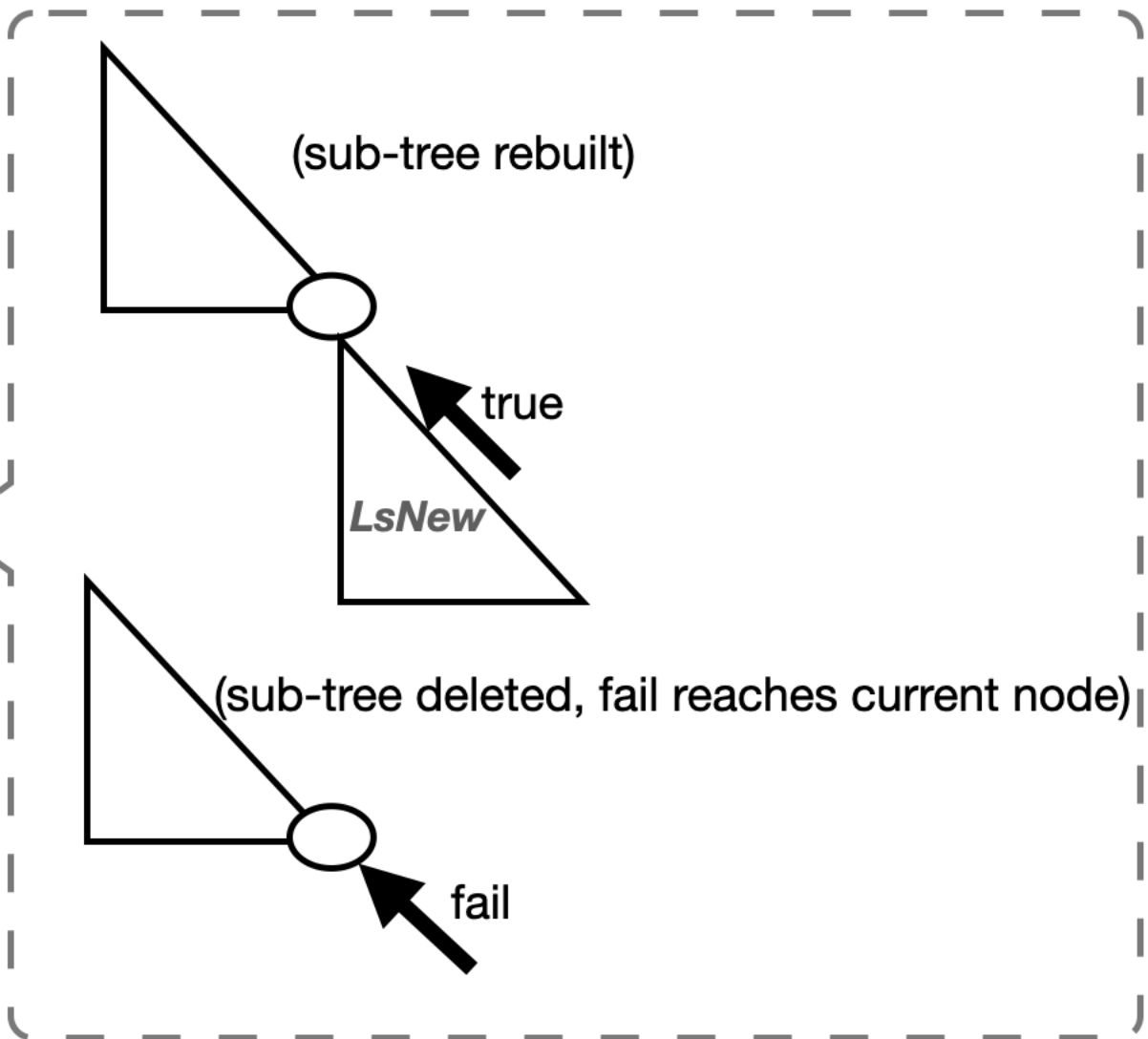
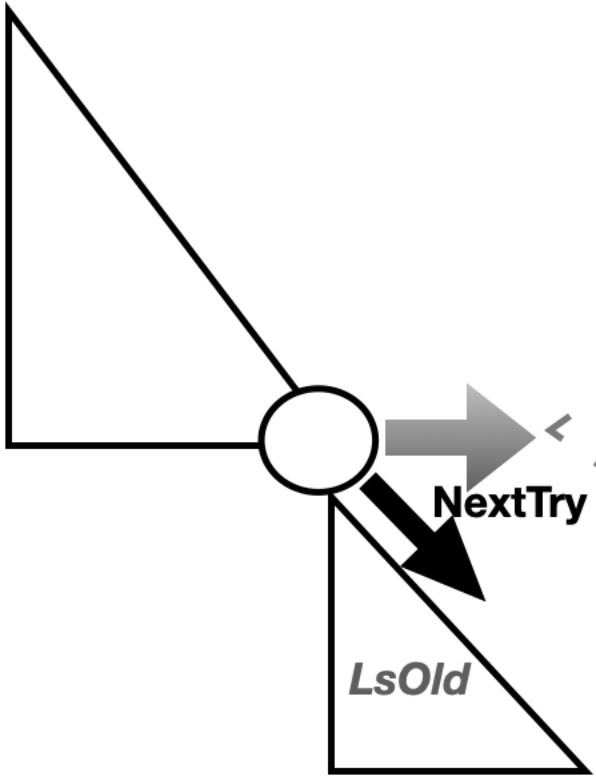
NextQ – how arguments evolve

NextQ - parameters



- the list of **actual parameters** of the new node is **inherited** from the list of **actual parameters** of the current node
- the list of **formal parameters** of the new node is **allocated** now (thus, the corresponding variables are indexed with the number of the node)

NextTry



Example 3rd

Check if an element is present in a list

How many arguments/why?

Predicate's signature is `member/2 (+Element, +List)`

```
member (X, L) :-  
    L = [H | T],  
    X = H.
```

```
member (X, L) :-  
    L = [H | T],  
    X \= H,  
    member (X, T).
```

```
member (X, L) :-  
    L = [],  
    fail.
```

Example 3rd contd.

```
member (X, L) :-
    L = [H | T],           % list not empty
    X = H.                 % searched element matches the head of the list

member (X, L) :-
    L = [H | T],           % list not empty
    X \= H,                 % searched element doesn't match the head of the list
    member (X, T).         % searched element found in the rest of the list

member (X, L) :-
    L = [],                % if reached the empty list
    fail.                  % searched element not found in the list
```

With default unifications, the predicate becomes:

```
member (X, [X | T]).       % default decomposition and unification
member (X, [H | T]) :-    % default decomposition
    X \= H,                % Is it necessary? Why/why not?
    member (X, T).

member (X, []) :-
    fail.                  % Is it necessary? Why/why not?
```

Example 3rd contd.

```
member (X, [X|T]) .      % default decomposition and unification
member (X, [H|T]) :-    % default decomposition
    X\=H,                % Is it necessary?
    member (X, T) .      % not really due to the search rule (A1)
member (X, []) :-        % Is it necessary?
    fail .               % not really due to the default failure by absence.
```

Thus, the predicate becomes:

```
member (X, [X|T]) .
member (X, [H|T])
    member (X, T) .
```

Qs:

- What happens if we reverse the order of clauses? Would it still be a correct predicate? Why/why not?
- How is better? Why?

The member/2 predicate

(1)
q/m₁

[m₁] member(X, [X|_]).

[m₂] member(X, [_|T]) :-

[m₂₁] member(X, T).

[q] ?- member(2, [1,2,3]).

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

$[q]$ `?- member(2, [1,2,3]) .`

(1)
q/m₁

$2 = X_1$
 $[1,2,3] = [X_1|_]$

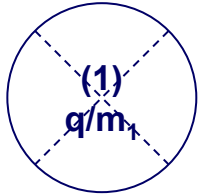
R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]     member(X, T).
```

```
[q] ?- member(2, [1,2,3]).
```



$2 = X_1$
 $[1,2,3] = [X_1|_]$ $\rightarrow$ **false**

C1 X

Computer Science

R1: clauses $\rightarrow$ top-down
 R2: subgoals $\rightarrow$ left-right
 R3: failure $\rightarrow$ backtrack

C1: q/head succeeds? $\rightarrow$ C2 or backtrack
 C2: fact? $\rightarrow$ C3 or body push stack
 C3: Stack empty? Stop or pop

The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

$[q]$ `?- member(2, [1,2,3]) .`

(1')
q/m₂

$2 = X_1$
 $[1,2,3] = [_|T_1]$

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

$[q]$ `?- member(2, [1,2,3]) .`

(1')
q/m₂

$2 = X_1$
 $[1,2,3] = [_|T_1]$ $\rightarrow$ $X_1 = 2$
 $T_1 = [2,3]$



C1 ✓
C2 ✗

R1: clauses $\rightarrow$ top-down
R2: subgoals $\rightarrow$ left-right
R3: failure $\rightarrow$ backtrack

C1: q/head succeeds? $\rightarrow$ C2 or backtrack
C2: fact? $\rightarrow$ C3 or body push stack
C3: Stack empty? Stop or pop

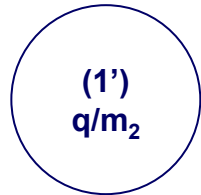
The member/2 predicate

$[m_1] \text{ member}(X, [X|_]).$

$[m_2] \text{ member}(X, [_|T]) :-$

$[m_{21}] \text{ member}(X, T).$

$[q] \text{ ?- member}(2, [1,2,3]).$



$2 = X_1$
 $[1,2,3] = [_|T_1] \rightarrow X_1 = 2$
 $T_1 = [2,3]$

m_{21}

$m_{21}: \text{member}(2, [2,3]).$

R1: clauses $\rightarrow$ top-down
 R2: subgoals $\rightarrow$ left-right
 R3: failure $\rightarrow$ backtrack

C1: q/head succeeds? $\rightarrow$ C2 or backtrack
 C2: fact? $\rightarrow$ C3 or body push stack
 C3: Stack empty? Stop or pop

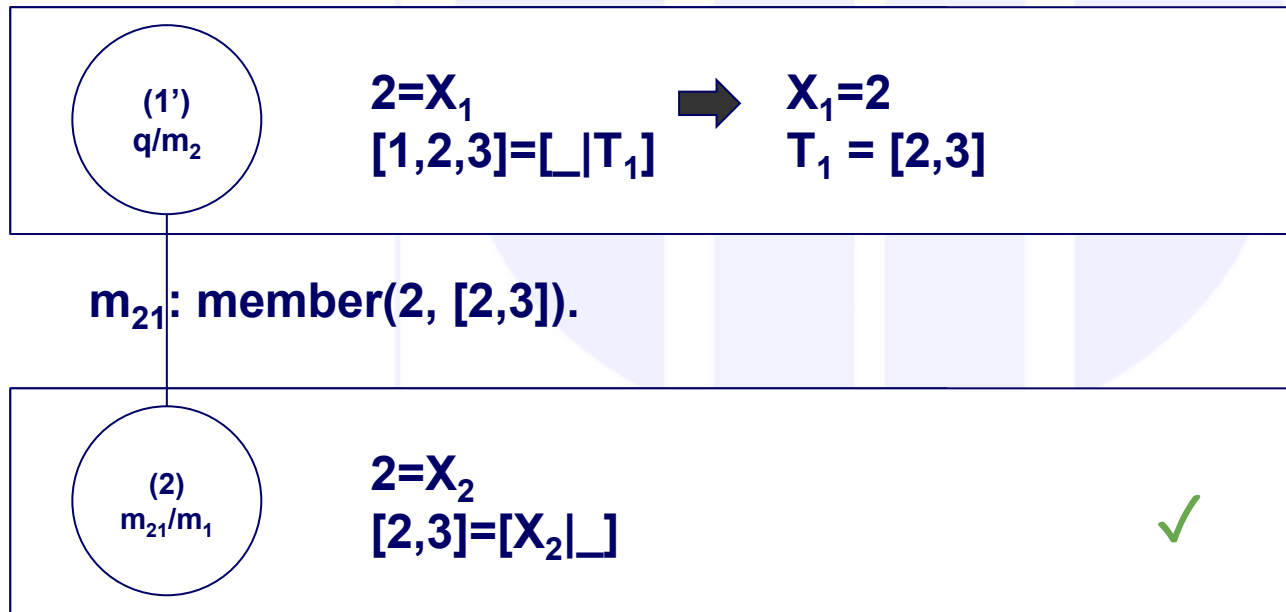
The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

$[q]$ `?- member(2, [1,2,3]) .`



R1: clauses $\rightarrow$ top-down
R2: subgoals $\rightarrow$ left-right
R3: failure $\rightarrow$ backtrack

C1: q/head succeeds? $\rightarrow$ C2 or backtrack
C2: fact? $\rightarrow$ C3 or body push stack
C3: Stack empty? Stop or pop

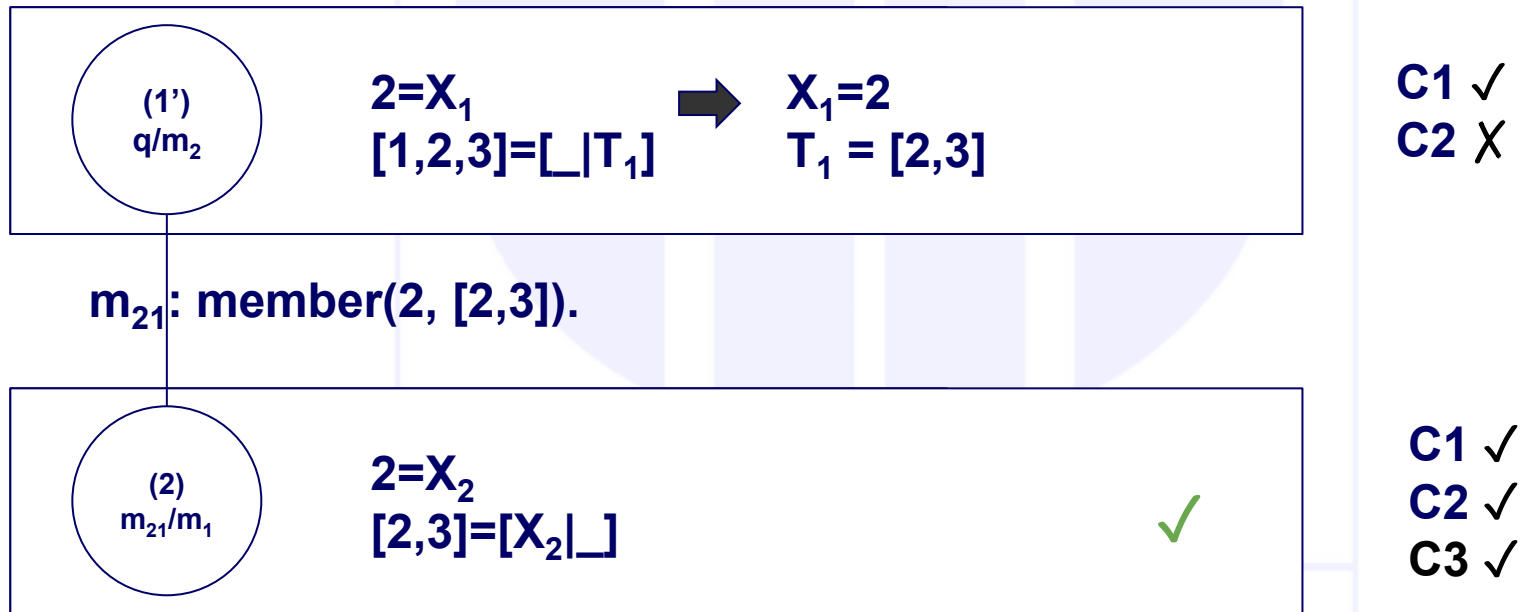
The member/2 predicate

$[m_1]$ `member(X, [X|_]) .`

$[m_2]$ `member(X, [_|T]) :-`

$[m_{21}]$ `member(X, T) .`

$[q]$ `?- member(2, [1,2,3]) .`



We have overlapped with a fact, the node is a leaf,
and we do not have any other clause in the stack.

We have obtained the execution tree

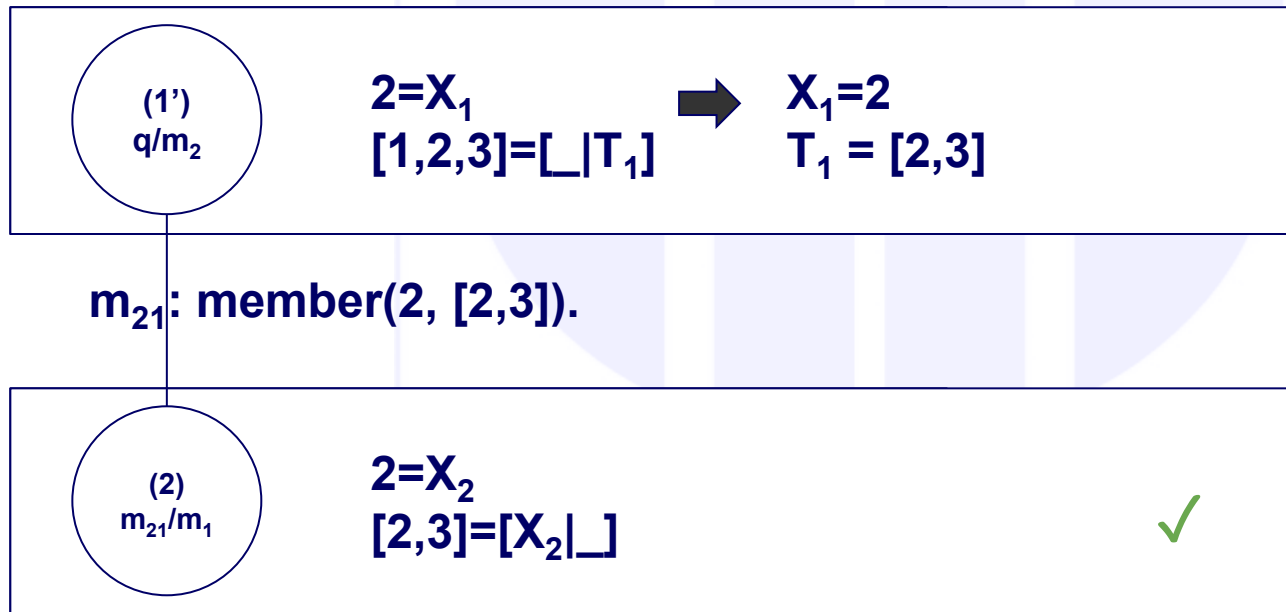
R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? → C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]) :-
[m21]      member(X, T).
```

```
[q] ?- member(2, [1,2,3]).
```

The member/2 predicate



We have overlapped with a fact, the node is a leaf
and we do not have any other clause in the stack.
We have obtained the execution tree

Do remember:

- (1) failed and only then through (1') did we get an answer.

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #3
List operations + The Cut
Computer Science

Agenda

- **Introducing recursion types**
- **Example – length of a list**
 - Types of parameters
- **Example – reversing a list**
 - Introducing pattern position
 - Meaning of arguments
- **Backtracking prevention – the CUT (!)**
 - Negation (as failure)

Recursion types

- What does recursion require?
 - Call to itself
 - Stopping condition
 - Step (*to get to the stopping condition*)
 - Process (*to get the result?*)

- **Backward**

```
bw([], ?) .
```

```
bw([H|T], R) :-
```

```
    bw(T, Rpart),  
    R created from Rpart
```

- **Differences?**

- When and where the result is constructed. **Backward** recursion builds the result as recursion returns, while **forward** builds a partial result on each recursive call.

- **Forward**

```
fw([], ?) .
```

```
fw([H|T], R) :-
```

```
    NewR created from R  
    fw(T, NewR) .
```

Actually, not quite like this, we shall see why in the following slides.

List length - backward recursion

- Determine the length of a list, number of arguments?

```
% len_bw/2 (+List, -Length)
```

```
len_bw([],0).
```

% length of empty list is 0

```
len_bw([_|T],N):-
```

```
    len_bw(T,N1),
```

% length of tail calculated as N1

```
    N is N1+1.
```

% add 1 to the length of tail

- Order of clauses? This way? Swapped? Which is correct? Which is better? Why? Is it important? Why?
 - (discuss indexation on the first argument. Oral explanation.)
- Meaning of N1?
 - partial result (N1) not quite relevant (is length of the tail)
- What if we need to keep the number of “consumed” elements?
- This solution uses **backward** recursion, meaning that the **result is built as recursion RETURNS**

List length - forward recursion

- What if we build it while advancing?

```
% len_fw/2 (+List, -Length)
```

```
len_fw(PR, [_|T]) :-
```

```
    NPR is PR+1,
```

% add 1 to the partial result (PR) so far
% i.e. length of the covered part

```
    len_fw(NPR, T) .
```

% go count the rest of the elements

```
len_fw(R, []) .
```

% R has the final list length

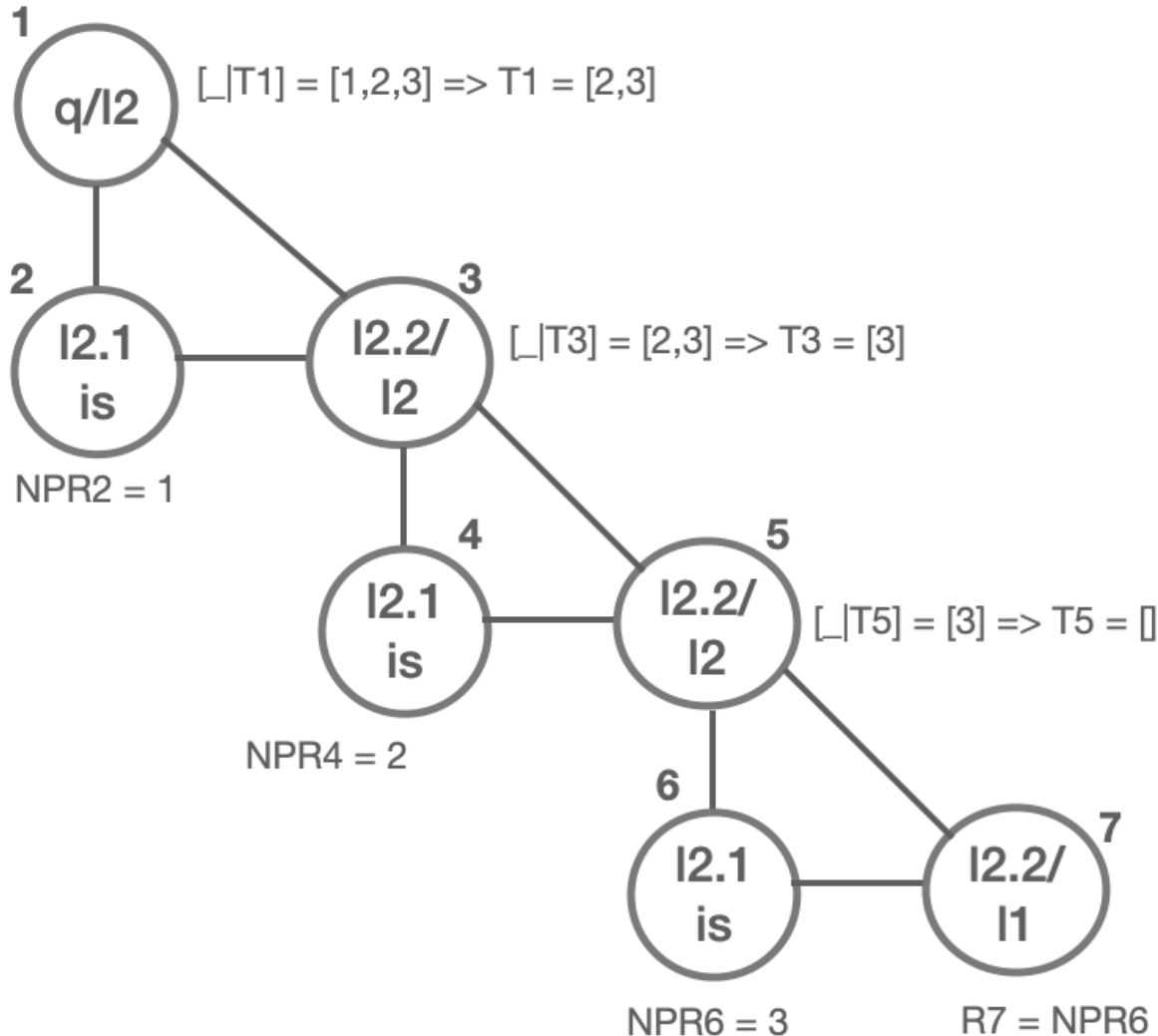
- What's the difference compared to the previous solution?
- PR represents here what we *have covered*; before – what we've *yet to cover*.

Computer Science

len_fw/2

Deduction tree

```
[11] len_fw([],R).
[12] len_fw([_|T],PR):-
[121]   NPR is PR+1,
[122]   len_fw(T, NPR).
[q] ?- len_fw([1,2,3],?).
```



List length - forward recursion

```
% len_fw/2 (+List, -Length)
```

- Not enough – as result, although calculated, is NEVER reported

```
len_fw([],N) .
```

```
len_fw([_|T],N) :-
```

```
    N is N1+1,
```

```
    len_fw(T,N1) .
```

- Upgrade it with 3rd argument = “steal” the result from the rightmost leaf

```
len_fw([],PR,FR) :- FR=PR.
```

```
len_fw([_|T],PR,FR) :-
```

```
    NPR is PR+1,
```

```
    len_fw(T,NPR,FR) .
```

- Observe that in the second clause does not modify FR at all

List length – forward recursion

- With default unification:

```
len_fw([],R,R) .  
len_fw([_|T],PR,FR) :-  
    NPR is PR+1,  
    len_fw(T,NPR,FR) .
```

- How should we call it? Needs specific INITIALIZATION:

```
?- len_fw(InList,0,Result) .
```

- To avoid mandatory initial call, use a wrapper:

```
len_fw(InList,Result) :-  
    len_fw(InList,0,Result) .
```

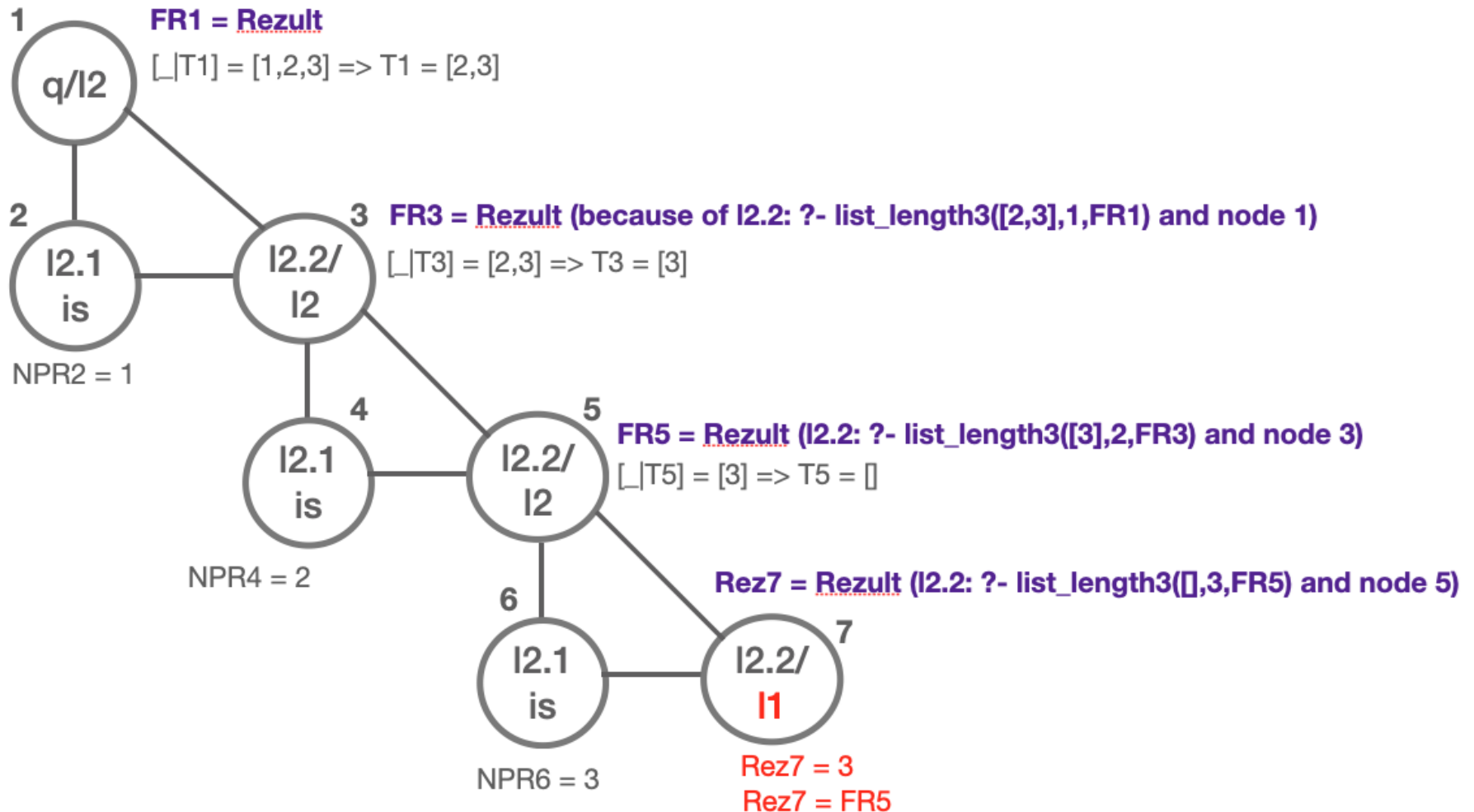
- Can have same name? Why/why not?
- Thus, the query becomes:

```
?- len_fw(InList,Result) .
```


Length2/3 deduction tree

```
[11] len_fw([],Res,Res).
[12] len_fw([_|T],PR,FR):-
[121]     NPR is PR+1,
[122]     len_fw(T,NPR,FR).
```

```
[q] ?- len_fw(InList,0,Result).
[q] ?- len_fw(InList,Result). %wrapper
```



Forward vs Backward recursion

Forward recursion

+

- Potentially useful partial results
- Efficiency if parallelized
- LCO (Last Call Optimization)

-

- Need for extra variable (final result)
- Need for specialized call (wrapper for accumulator initialization)

Backward recursion

- No need for extra variable
- No need for special call (*wrapper*)

- Rarely useful intermediate results
- Not efficient if parallelized (execution of concurrent process(es) - delayed until recursion returns)

List reverse - backward recursion

- How many arguments?

- 2 = list + reversed list

```
% rev_bw/2 (+List, -RevList)
```

```
rev_bw([], []). % empty input returns empty output
```

```
rev_bw([H|T], R) :-
```

```
    rev_bw(T, PR), % reverses the tail of list
```

```
    append(PR, [H], R). % concatenates the partial results
```

- Needs specialized call? Why/why not?
- Efficiency? How is it calculated?
- Can we avoid 2 traversals through the list? How?

Computer Science

List reverse - forward recursion

- How many arguments?

```
% rev_fw/3(+List, +PartRevList, -RevList)
```

```
rev_fw([], PR, PR) .
```

% when input gets empty the partial
% result becomes the final one

```
rev_fw([H|T], PR, R) :-  
    NPR=[H|PR],
```

% concatenates the partial results
% PR, NPR acts as a stack

```
rev_fw(T, NPR, R) .
```

% reverses the tail of list

- Wrapper (MUST have)

```
rev_fw(In, Out) :-  
    rev_fw(In, [], Out) .
```

- Efficiency? Is it better?

List reverse - forward recursion

```
reverse([], PR, PR) .  
reverse([H|T], PR, R) :-  
    reverse(T, [H|PR], R) .
```

- Same as before, but explicit → implicit unification
- Let's change the order of arguments 2 and 3 (makes no difference in the functionality)

```
reverse([], PR, PR) .  
reverse([H|T], R, PR) :-  
    reverse(T, R, [H|PR]) .
```

- Rename variables:

```
reverse([], L, L) .  
reverse([H|T], L, R) :-  
    reverse(T, L, [H|R]) .
```

Comparative analysis

```
reverse([], L, L) .
reverse([H|T], L, R) :-
    reverse(T, L, [H|R]) .

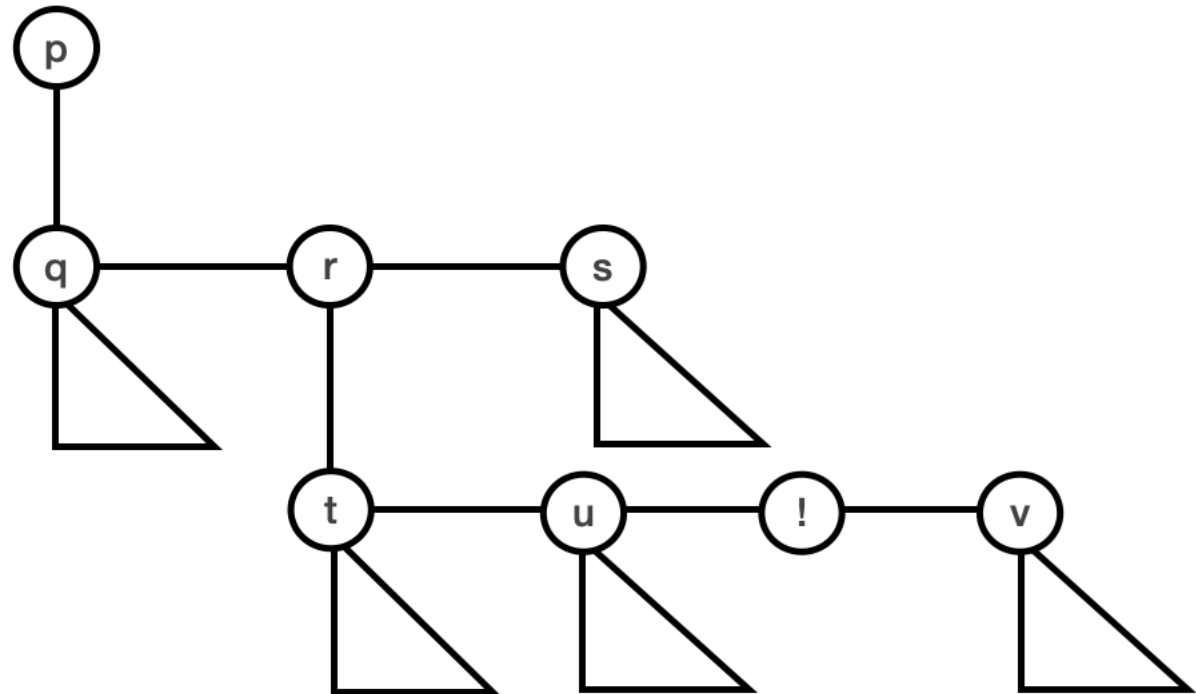
append([], L, L) .
append([H|T], L, [H|R]) :-
    append(T, L, R) .
```

- What is the difference?
- Meaning of pattern's position
- Length of arguments analysis

Backtracking avoidance (the CUT !)

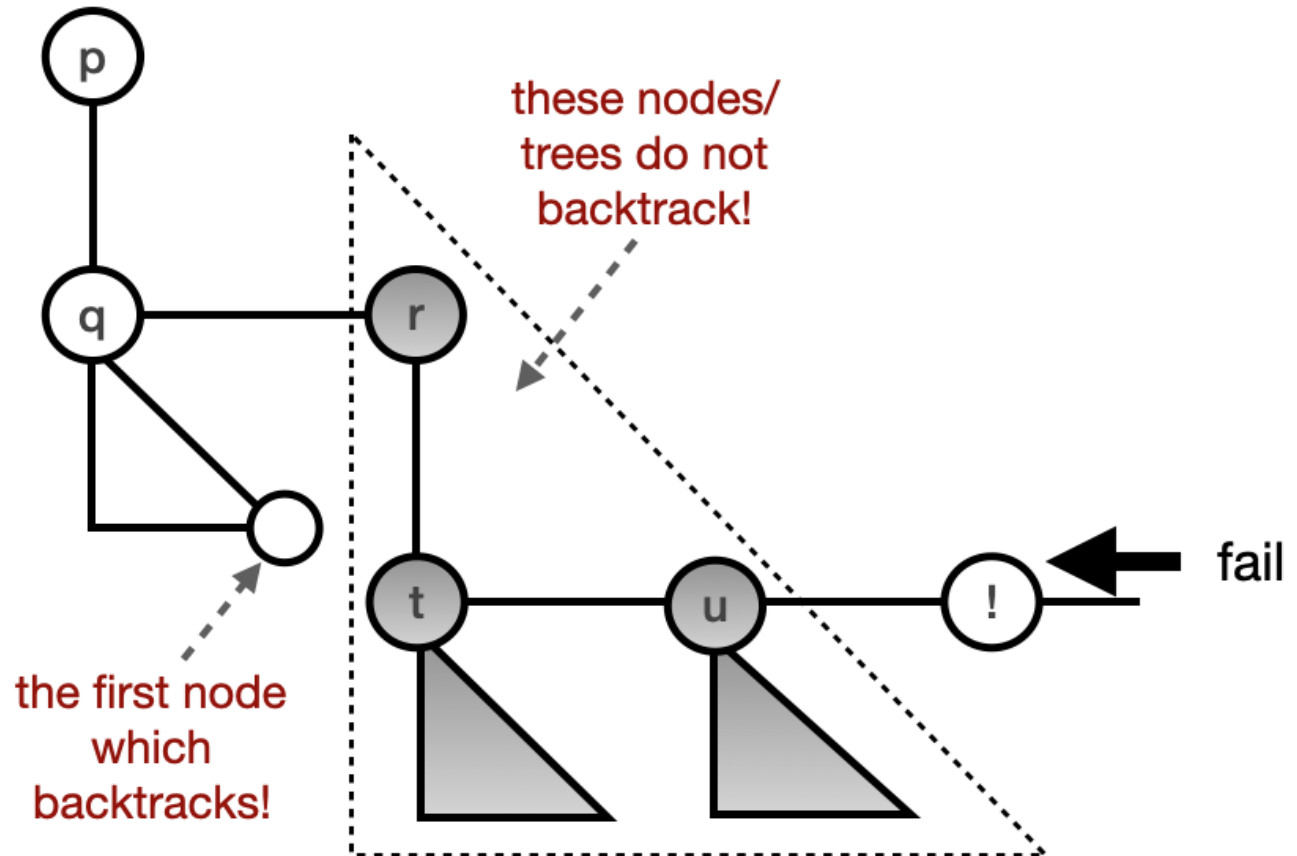
- Built-in predicate, the “!” operator
- Always succeeds
- Never backtracks

$p:-q, r, s.$
 $r:-t, u, !, v.$



The CUT (!)

$p:-q,r,s.$
 $r:-t,u,! ,v.$



The CUT (!) – effects

- **!** Prevents the backtracking of the following categories:
 - All left siblings
 - All subtrees of left sibling nodes
 - Parent node
- NO other node is affected:
 - Siblings on the right/their subtrees have the ability to backtrack
 - Left siblings DO backtrack BEFORE **!** Is executed
 - Their subtrees also
- As everything, **!** takes effect ONLY AFTER its execution (before execution it does NOT actually exist)

Negation (as failure)

- Unfair behavior
- As it assumes the **inability to prove True means False**

`not (P) :-`

`P, !, fail.`

`not (P) .`

- $P - \text{true} \Rightarrow \text{not}(P) - \text{false}; \text{OK.}$
- $P - \text{false} \Rightarrow \text{not}(P) - \text{true}; \text{OK.}$
- $P - ?$ Don't know how it is. Why? Incomplete information, not known at the time.
- How is `not(P)`? Let's run it:
 - P 's execution fails (P is NOT yet True),
 - we enter the second clause and succeeds.

Negation (as failure)

```
not (P) :-  
    P, !, fail.  
not (P) .
```

- Negation - assumes negated is True just because we don't have information about P.
- Negation as failure (to check validity)

TECHNICAL UNIVERSITY

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #4

List operations

OF CLUJ-NAPOCA

Computer Science

Agenda

- **Forward vs backward recursion**
 - sum
 - generics
 - Comparative analysis, pros and cons
- **append3**
 - Forms
 - Efficiency analysis and justification
 - Nondeterministic call
- **Delete from a list**
 - One, just one, all, repeating the query

Forward vs backward recursion

- Forward vs backward - alternative approaches to handle data
- Data is split into partitions
- **Forward** = processing takes place when the current item is processed when is first encountered, and the rest of the data (the other components than the item) are processed AFTER the current item is processed.
- **Backward** = starts by processing all next after current item (the other components than the item) via recursive call(s) while the current item is processed just AFTER we return from recursion(s).
 - *NOTE:* it is returned from recursive call (and **NOT from backtracking!!!**).
- In case of lists [H|T]:
 - *forward* handles H first and next T (via recursive call),
 - *backward* starts by solving the problem on T (via recursive call), and just when returning from the call H is processed.

Forward sum

```
% sum_fw/3 (+List, +Accumulator, -Result).
```

```
sum_fw([], PartialSum, PartialSum).
```

```
sum_fw([H|T], PartialSum, Sum) :-  
    NewPartialSum is PartialSum + H,  
    sum_fw(T, NewPartialSum, Sum).
```

% final result, copies the value of the partial
% result with default unification

% do process the current item
% go ahead with the remaining struct

- List decomposed into [H|T]
 - Starts by processing (addition here) the current item (H here)
 - Continue with processing the rest of the structure (one recursive call here, as partition is H and T)
 - This implies the items are added in the sum forward (starting from the first one).
- ->->->...-> processing is performed in the order of items in the structure
- The accumulated result represents the sum of values from the beginning to the current item

Backward sum

```
% sum_bw/2 (+List, -Result).
```

```
sum_bw([], 0).
```

```
sum_bw([H|T], Sum) :-
```

```
    sum_bw(T, TailSum),
```

```
    Sum is TailSum + H.
```

% result gets initialized. Empty input, null

% call processing first on rest of the partition

% do process the current item

- List decomposed into [H|T]
 - Starts with processing T, same predicate, recursive call
 - When returned, use the obtained result ($TailSum$) to evaluate the overall result on the whole list (Sum)
 - This implies the items are added in the sum backwards (starting from the last one).
- ---... --> first go this way (all way to the end) doing nothing (just call)
- <- the processing occurs after returning from the call, one at a time
 - in the opposite order, *last* item *first* processed, before last item second processed,
 - third, second and first items being last processed in this specific order (first is last)
- The accumulated result is the sum of values from the current item to the end

Forward vs backward sum

- Forward solution needs specific initial call
 - write a special predicate (a wrapper) to make it in a sound way (avoid the need of user knowing how to initialize the accumulator parameter):

```
% sum_fw/2 (+List, -Result)
sum_fw(List, Sum) :-
    sum_fw(List, 0, Sum).
```

```
% same partial result as in case of backward
% nothing yet processed, null result.
% same as in stop condition for bwd.
```

- If the input list is [1,2,3,4,5,6,7] what is going to be the partial result (PartialSum) and (TailSum) respectively on forward and backward solutions?
- Try to **estimate before running** them.
- Follow the textbook to update the predicate with the necessary lines to print the values.

```
% sum_bw/2 (+List, -Result).
sum_bw([], 0).
sum_bw([H|T], Sum) :-
    sum_bw(T, TailSum),
    Sum is TailSum + H.
```

```
% sum_fw/3 (+List, +Accumulator, -Result).
sum_fw([], PartialSum, PartialSum).
sum_fw([H|T], PartialSum, Sum) :-
    NewPartialSum is PartialSum + H,
    sum_fw(T, NewPartialSum, Sum).
```

Forward generic

```
% forward_recursion/3 (+Input, +Accumulator, -Output)

% final result (arg 3) copies partial result(arg 2) value with default unification
forward_recursion([], Result, Result) .

% partition data, split into H and T
forward_recursion([H|T], PartialResult, Result) :-
    % start by processing the current item and thus
    % updating previous PartialResult to NewPartialResult via processing do/3
    do(PartialResult, H, NewPartialResult) ,

    % process the rest of the structure with recursive call.
    forward_recursion(T, NewPartialResult, Result) .

% forward_recursion_call/2 (+Input, -Output)
% make the initialization with a separate predicate (wrapper) to avoid mandatory user initialization
forward_recursion_call(Input, Output) :-
    forward_recursion(Input, InitialValueOfResult, Output)
```

Backward generic

```
% backward_recursion/2 (+Input, -Output)

% empty input, make initialization backwards
backward_recursion([], InitialValueOfResult).

backward_recursion([H|T], NewPartialResult):-
    % starts with processing the rest of the structure;
    % all partition but the current item.
    backward_recursion(T, PartialResult),

    %process the current item
    do(PartialResult, H, NewPartialResult).
```

- No need for a specific initial call, hence, no wrapper predicate.

Forward vs backward recursion

```
fw_rec([],R,R).  
fw_rec([H|T],PR,R):-  
    do(PR,H,NPR),  
    fw_rec(T,NPR,R).  
  
fw_rec(Input,Output):-  
    fw_rec(Input,InitialValueOfResult,Output)  
  
bw_rec([],InitialValueOfResult).  
bw_rec([H|T],NPR):-  
    bw_rec(T,PR),  
    do(PR,H,NPR).
```

Forward vs backward pros and cons

	Forward	Backward
+	• Process "as we go" => structure is processed from front to end => intermediate results could be useful (the result of the structure "so far"). • In concurrent processing that result is made available and another process using it can start immediately • Last Call Optimization = reusing the same stack area without the need of restoring it back	• Needs no initialization => needs no specific call => needs no wrapper => needs no additional argument
-	• Needs specific initial call => always make a wrapper to initialize the accumulator • Needs additional argument (partial result)	• Intermediate results are seldom useful • The result of the structure is known just at the end of processing, so, concurrency is postponed on sync

Concatenate 3 lists

- Use what you ***have*** vs use what you ***know***
- We have the concatenation of 2 lists
- Use it twice.

```
append([], List, List).  
append([Head|Tail], List, [Head|Rest]) :-  
    append(Tail, List, Rest).
```

- To put together L1, L2 and L3 do the following
 - $(L1+L2) + L3$ OR $L1 + (L2+L3)$

Concatenate 3 lists: Use what you have 1

```
append([], L, L) .
append([H|T], L, [H|R]) :-
    append(T, L, R) .
```

- $(L1+L2) + L3$; say $|L1|=n1$, $|L2|=n2$, $|L3|=n3$

```
append3_1(L1, L2, L3, Result) :
    append(L1, L2, Intermediate) ,           % link L2 at the end of L1 = Intermediate
    append(Intermediate, L3, Result) .       % link L3 at the end of the intermediate result
```

- Efficiency:
 - $O(n1)$ for the 1st call (links $L2$ at the end of $L1$)
 - $O(n1+n2)$ for the 2nd call, length $Intermediate$ of is length of $L1$ and $L2$ (links $L3$ at the end of $Intermediate$)
 - Overall: $t(n)=2n1+n2$

Computer Science

Concatenate 3 lists: Use what you have 2

```
append([], L, L) .
append([H|T], L, [H|R]) :-
    append(T, L, R) .
```

- $L1 + (L2 + L3)$; say $|L1|=n1$, $|L2|=n2$, $|L3|=n3$

```
append3_2(L1, L2, L3, Result) :-
    append(L2, L3, Intermediate),           % link L3 at the end of L2 = Intermediate
    append(L1, Intermediate, Result) .      % link intermediate at the end of the first list L1
```

- Efficiency:
 - $O(n2)$ for the 1st call, decomposes $L2$ (links $L3$ at the end of $L2$)
 - $O(n1)$ for the 2nd call, decomposes $L1$ (links $Intermediate$ at the end of $L1$)
 - Overall: $t(n)=n1+n2$
- Observations:
 - **Second version better regardless of the input!**
 - Order of calls matters (again!)
 - How can we use this in a standalone predicate?

Concatenate 3 lists: Use what you know

- What do we know?
 - Concatenation via decomposition of the first argument! Use it!

```
append3_3([H|T], L2, L3, [H|R]) :-  
    % as long as the first argument is nonempty, decompose it  
    append3_3(T, L2, L3, R) .  
append3_3([], [H|T], L, [H|R]) :-  
    % once first argument becomes empty, you are back on 2 list concatenation  
    append3_3([], T, L, R) .  
append3_3([], [], L, L) .
```

- Observations:
 - Clauses 2 and 3 are disjoint from clause 1 (indexation on the first argument would treat them separately, i.e. cl1 one entrance, cl 2 and 3 another one). Therefore, it does NOT matter where clause 1 is placed
 - On the other hand, clause 3 should come AFTER clause 2 (as indexation on the second argument is NOT available)

Concatenate 3 lists: Use what you know – contd.

```
append3_3([H|T], L2, L3, [H|R]) :-
    append3_3(T, L2, L3, R).           % decomposes first list
append3_3([], [H|T], L, [H|R]) :-
    append3_3([], T, L, R).           % decomposes second list
append3_3([], [], L, L).
```

• Observations:

- How is it done? (resembles behavior of version1)
 - Decomposes $L1$ to traverse it and adds each item in result (behaves as version 1, as in "link $L2$ at the end of $L1$ ")
 - Decomposes $L2$ to traverse it and adds each item in result (links $L3$ at the end of $L1$ concatenated to $L2$).
- Efficiency (resembles in performance to version _2): $t(n)=n1+n2$
 - $O(n1)$ first clause
 - $O(n2)$ second clause

```
append3_2(L1, L2, L3, Result) :-
    append(L2, L3, Intermediate),      % link L3 at the end of L2 = Intermediate
    append(L1, Intermediate, Result).  % link intermediate at the end of the first list L1
```

Concatenate 3 lists: Use what you know – contd.

- Behavior: resembles version_1 $(L1+L2) + L3$
- Efficiency: resembles version_2 $t(n)=n1+n2$
- How is possible? Behavior V1 and performance V2?
- Explain!
- Explain which of the 3 versions is best?
- Which to use and why?
- Which should never be used? Why?

Computer Science

Concatenate 3 lists: Nondeterministic call

- Given `append3` predicate and the call: `?-append3(X,Y,Z,[1,2])`.
- What is the meaning of the call?
 - Non-deterministic call
 - Which lists `X`, `Y` and `Z` concatenated form list `[1,2]`.

- What is/are the result/s?

X	Y	Z
[]	[]	[1,2]
[]	[1]	[2]
[]	[1,2]	[]
[1]	[]	[2]
[1]	[2]	[]
[1,2]	[]	[]

- Other results? Why not?
- Which order/why?
 - Depends on the implementation.
 - Identify (BEFORE running) the order of results in each implementation
 - For `append3` with `append`, the `append` order of clauses is MANDATORY (fact first), otherwise it enters infinite loop with 3 free variables on the first call.

Delete from a list

- Given a list, remove one item from it.
- How many arguments/why?

```
% delete/3 (+Element, +InputList, -OutputList)
```

```
delete(X, [X|T], T) .      % if item to remove = head of input, don't put it in output
delete(X, [H|T], [H|R]) :- % otherwise, keep it on output and
    delete(X, T, R) .      % remove from tail
```

- What are the results on this query when the item occurs several times?

```
?- delete(4, [1, 4, 2, 4, 3, 4], R) .
R=[1, 2, 4, 3, 4] ;      % Next
R=[1, 4, 2, 3, 4] ;      % Next
R=[1, 4, 2, 4, 3] ;      % Next
false
```

- What happens if the item is not present in the list?

```
?- delete(5, [1, 4, 2, 4, 3, 4], R) .
false
```

Delete from a list

```
delete(X, [X|T], T) .
delete(X, [H|T], [H|R]) :-
    delete(X, T, R) .
```

So, the meaning of the predicate is:

- **delete exactly one occurrence** of an item from the list.

- What are the results on this query? Why?

```
?- delete(1, [1,2,1,3,1], R) .
R = [ 2,1,3,1] ; % Next
R = [1,2, 3,1] ; % Next
R = [1,2,1,3 ] ; % Next
false.
```

- What about this query? Why?

```
?- delete(4, [], R) .
false.
```

```
?- delete(4, [1,2,1,3,1], R) .
false.
```

Delete from a list

```
delete(X, [X|T], T) .      % if item to remove = head of input, don't put it in output
delete(X, [H|T], [H|R]) :- % otherwise, keep it on output and
    delete(X, T, R) .      % remove from tail
delete(_, [], []).
% when the empty list is reached,
% whatever element is assumed to
% be deleted, done, result empty
```

- What are the results on call when the item occurs several times?
- What happens in case the item is not present in the list?

Computer Science

Delete from a list

```
delete(X, [X|T], T) .  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .  
delete(_, [], []) .
```

- What are the results on call?

```
?- delete(1, [1,2,1,3,1], R) .  
R = [ 2,1,3,1] ;  
R = [1,2, 3,1] ;  
R = [1,2,1,3  ] ;  
R = [1,2,1,3,1] ;  
false.
```

% Next
% Next
% Next, what's next? Why?

- What about the call? Why?

```
?- delete(4, [1,2,1,3,1], R) .  
R = [1,2,1,3,1] ;  
false.
```

% Next

So, the meaning of the predicate is:

- **delete one occurrence** of an item from the list;
- if **absent**, do **NOTHING** (leave the list unchanged).

Delete from a list

- What are the differences when the 2 implementations (without/with 3rd clause) are compared? Explain!
- What happens if the clause when the item is found cuts the backtrack?
- Implementation without 3rd clause:

```
delete(X, [X|T], T) :- ! .  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:
 - $p:-q,r,!s$.
 - The cut implies a “default” negation of q,r (as $\text{not}(q,r)$) in all clauses after it.
- In the predicate `delete`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete from a list

```
delete(X, [X|T], T) :- !.
delete(X, [H|T], [H|R]) :-
    delete(X, T, R).
```

% means: delete(X, [H|T], T) :- **X=H**, !,
% therefore, an implicit **X=H** is here

```
?- delete(1, [1,2,1,3,1], R).
R=[2, 1, 3, 1] ; % Why? Next?
% There is no next answer.
?- delete(4, [1,2,1,3,1], R2).
false.
```

- In the implementation with 3rd clause:

```
delete(X, [X|T], T) :- !.
delete(X, [H|T], [H|R]) :-
    delete(X, T, R).
delete(_, [], []).
```

```
?- delete(1, [1,2,1,3,1], R).
R=[2, 1, 3, 1] ; % Why? Next?
% There is no next answer.
?- delete(4, [1,2,1,3,1], R2).
R=[1,2,1,3,1] ; % Why? Next?
% There is no next answer.
```

Delete all occurrences of an item from a list

Start from the first version of the predicate

```
delete(X, [X|T], T) .  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .
```

```
delete(X, [X|T], R) :- % if item to remove = head of input, don't put it in output  
    delete(X, T, R) . % but also remove other occurrences from tail  
delete(X, [H|T], [H|R]) :- % otherwise, keep it on output and  
    delete(X, T, R) . % remove from tail
```

- Could it be just this?
- Justify!

Delete all occurrences of an item from a list

```
delete_all(X, [X|T], R) :-           % if the item=head, don't put it on output
    delete_all(X, T, R) .           % but also remove from tail
delete_all(X, [H|T], [H|R]) :-      % otherwise, keep it on output and
    delete_all(X, T, R) .           % remove from tail
delete_all(_, [], []).               % when [ ] is reached, done, result empty
```

```
?- delete_all(1, [1, 2, 1, 3, 1], R) .
R = [2, 3]
```

- What happens if we repeat the query? Why?
- How many answers? Which order? Explain!
- How can we obtain just the answer from above?

% unification used to delete
% last 1, is unmade

R = [2, 3, 1]

R = [2, 1, 3]

R = [2, 1, 3, 1]

R = [1, 2, 3]

R = [1, 2, 3, 1]

R = [1, 2, 1, 3]

R = [1, 2, 1, 3, 1]

Delete all occurrences of an item from a list

```
delete_all(X, [X|T], R) :-  
    !,  
    delete_all(X, T, R).  
delete_all(X, [H|T], [H|R]) :-  
    delete_all(X, T, R).  
delete_all(_, [], []).
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:
 - $p:-q,r,!s$.
 - The cut implies a “default” negation of q,r (as $\text{not}(q,r)$) in all clauses after it.
- In the predicate `delete_all`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete all occurrences of an item from a list

```
delete_all(X, [X|T], R) :- !,  
    delete_all(X, T, R).
```

What does ! negate?

It's the default unification of X! The clause above is actually:

```
delete_all(X, [H|T], R) :-  
    X=H, !,  
    delete_all(X, T, R).
```

Therefore, the cut negates **X=H**, thus, in the next clauses, there is a default **X\=H**.

Delete all occurrences of an item from a list

```
delete_all(X, [X|T], R) :-  
    !,  
    delete_all(X, T, R).  
delete_all(X, [H|T], [H|R]) :-  
    delete_all(X, T, R).  
delete_all(_, [], []).
```

- What is the result of this query? Why?

```
?- delete_all(1, [1, 2, 1, 3, 1], R).
```

R=[2, 3]

% There is no next answer.

- What happens if we repeat the query? Why? How many answers? Which order? Explain!

Conclusion

- Order of clauses matters
- Order of calls matters
- Always estimate performance
- Meaning of
 - Repeating the query
 - The cut
- Questions?

TECHNICAL UNIVERSITY

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #5

Trees and operations

Computer Science

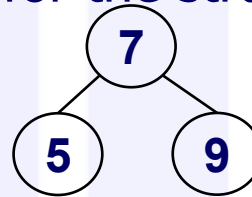
Agenda

- **Trees**
 - representation
 - basic operations
- **Tree traversal**
 - pre, in, post orders
- **Tree transformation into an ordered list**
 - Forward, backward, can we do better?
- **Trees (BST and regular trees)**
 - insert, search, delete

Trees

- Structure not defined; it is (used) as we define them
- Traditional definition is either prefix or infix notation
 - with a letter as name for the structure: *n* or *t*.

- Therefore, the tree



- Is represented in one of the forms:

prefix notation:

```
tree(t(7,t(5,nil,nil),t(9,nil,nil))).
```

infix notation:

```
tree(n(n(nil,5,nil),7,n(nil,9,nil))).
```

- Existing trees are represented as facts (predicate tree) in the program

Trees – operations

- Same as in other languages
- **Basic operations:**
 - Traversal
 - Search
 - Insert
 - Delete

Trees – traversal

- 3 types: pre/in/post order

```
inorder(nil) .
```

```
inorder(n(Left,Key,Right)) :-  
    inorder(Left) ,  
    do_something(Key) ,  
    inorder(Right) .
```

- The predicate has no outcome/output. Why?
 - No arguments!
- The order of clauses doesn't matter (due to indexation on the first argument)
- The predicate `do_something` does the actual job
- It runs $O(n)$ if `do_something` has constant time $O(1)$.
- Otherwise, the time is calculated with the Master Theorem.
- pre/post orders are similar, `do_something` goes first/last.

Tree transformation into an ordered list

- Given a Binary Search Tree (BST; what is it? Why do we represent trees as search trees? Benefits? Limitations? Means of overcoming limitation?)
- Generate the ordered list of the keys in the tree
- Approaches: divide et impera
 - divide – already done (the tree structure is a partition already)
 - impera – combine the results obtained on the subsets of the partition
 - Depending on how efficient we combine them we get the efficiency of the solution
- Approaches considered:
 - Forward and backward recursion
 - Pre/in/post order (which one is available in the context)

Computer Science

Tree 2 list forward

- By the approach, besides the output argument, we need a supplementary argument, some partial result to accumulate the results "so far"
- The arguments needed are:
 - input tree, accumulator (list type), output list
- Order of arguments:
 - input has to come first to benefit from the first argument indexation
- The other arguments are in any order
- The meaning of the accumulator = the elements from the tree we already covered placed in an ordered list

```
% inorder1/3 (Tree, Partial_result_list, Final_result_list)
```

Tree 2 list forward – code

```
inorder1(nil,L,L).  
inorder1(n(Left,Key,Right), Lin, Lout):-  
    inorder1(Left, Lin, LLout),  
    append(LLout,[Key],LRin),  
    inorder1(Right,LRin,Lout).
```

Clauses:

- When we reached the empty tree, the accumulator instantiates the result (default unification) as always in forward recursion.
- For the nonempty tree, the forward approach solves the parts of the partition in the standard order (first to last flow)
 - Solve the problem on the left subtree; the accumulator (content of the tree already covered, `Lin`) is passed as partial result. The result computed by this call will add all the keys in the left subtree to that accumulator and passes as accumulator (`LLout`) to the next processing task; `inorder1(Left,Lin,LLout)`,
 - Solve the problem on the next part in the partition = `Key`; `append(LLout,[Key],LRin)`,
 - Solve the problem on the right subtree; the accumulator (content of the tree already covered, `LRin`) is passed as partial result. The result computed by this call will add all the keys in the right subtree to that accumulator and passed as final result = on the whole tree); `inorder1(Right,LRin,Lout)`.

Tree 2 list forward – code analysis

```
inorder1(nil,L,L).
```

```
inorder1(n(Left,Key,Right), Lin, Lout):-  
    inorder1(Left, Lin, LLout),  
    append(LLout,[Key],LRin),  
    inorder1(Right,LRin,Lout).
```

- Needs specific initial call (wrapper) with additional argument (accumulator):

```
inorder1(Tree,Result):-  
    inorder1(Tree,[],Result).
```

- Evaluation:

- 2 recursive calls ($a=2$)
- 2 proportional parts (assuming tree is balanced) ($b=2$)
- Outside recursive call linear time (append) ($c=1$)
- Hence, **$O(n \lg n)$** – supralinear time, but a small quantity over n
- Please come back to this code AFTER we study difference lists

Tree 2 list backward – code

- Needs no additional argument
- The arguments needed are:
 - input tree, output list
- Order of arguments:
 - input has to come first to benefit from the first argument indexation

```
% inorder2/2 (+Tree, -ResultList)
```

```
inorder2(nil, []).
```

```
inorder2(n(Left,Key,Right),Lout):-
```

```
    inorder2(Left,LLout),
```

```
    append(LLout, [Key], Lintout),
```

```
    inorder2(Right,LRout),
```

```
    append(Lintout, LRout, Lout).
```

Tree 2 list backward – code

```
inorder2(nil, []).  
inorder2(n(Left, Key, Right), Lout) :-  
    inorder2(Left, LLout),  
    append(LLout, [Key], Lintout),  
    inorder2(Right, LRout),  
    append(Lintout, LRout, Lout).
```

Clauses:

- When we reached the empty tree, the result is empty.
- For the nonempty tree, the backward approach solves:
 - left subtree; the keys from left subtree generate the list from left out (LLout),
 - on the next part in the partition = Key to be added at the end of the LLout to create Lintout; `append(LLout, [Key], Lintout)`,
 - right subtree; the keys from right subtree generate the list from right out (LRout),
 - finally, the results of parts are concatenated: `append(Lintout, LRout, Lout)`.

Tree 2 list backward– code analysis

```
inorder2(nil, []).  
inorder2(n(Left, Key, Right), Lout) :-  
    inorder2(Left, LLout),  
    append(LLout, [Key], Lintout),  
    inorder2(Right, LRout),  
    append(Lintout, LRout, Lout).
```

- Evaluation
 - 2 recursive calls ($a=2$)
 - 2 proportional parts (assuming tree is balanced) ($b=2$)
 - Outside recursive call linear time (append) ($c=1$)
 - Hence, **$O(n \lg n)$** – supralinear time, but a small quantity over n
- Compared to the previous strategy?
 - Time is larger (2 append calls instead of 1)

Tree 2 list backward – better?

- What can be done to improve?
- 2 append calls, both traversing (almost) through the same list
- The first append goes over the whole first argument (large list) in order to add just one element (Key) at the end.
- Can we make it better? What if we rephrase:
 - Add at the end of the list with add at the beginning of a list. How?
 - Instead of adding `Key` at the end of `LLout`
 - Add it at the beginning of `LRout`
 - But `LRout` is NOT available at this moment
 - Change order of processing

Computer Science

Tree 2 list backward – code

```
inorder2(nil, []).  
inorder2(n(Left, Key, Right), Lout) :-  
    inorder2(Left, LLout),  
    append(LLout, [Key], Lintout),  
    inorder2(Right, LRout),  
    append(Lintout, LRout, Lout).
```

```
inorder3(nil, []).  
inorder3(n(Left, Key, Right), Lout) :-  
    inorder3(Left, LLout),  
    inorder3(Right, LRout),  
    append(LLout, [Key | LRout], Lout).
```

Clauses:

- When we reached the empty tree, the result is empty.
- For the nonempty tree, the backward approach solves:
 - left subtree; the keys from left subtree generate the list from left out (LLout),
 - right subtree; the keys from right subtree generate the list from right out (LRout),
 - finally, the results of parts are concatenated with the unit element in between (before the result on the right): `append(LLout, LRout, Lout)`.

Tree 2 list backward– code analysis

```
inorder3(nil, []).  
inorder3(n(Left, Key, Right), Lout) :-  
    inorder3(Left, LLout),  
    inorder3(Right, LRout),  
    append(LLout, [Key| LRout], Lout).
```

- Evaluation
 - 2 recursive calls ($a=2$)
 - 2 proportional parts (assuming tree is balanced) ($b=2$)
 - Outside recursive call linear time (append) ($c=1$)
 - Hence, **$O(n \lg n)$**
- Better
 - than the previous backward as just one append (multiplicative constant)
 - than forward – needs no additional parameter
- Can we do even better? Answer – we can! Check after difference lists

Tree 2 list conclusions (so far)

- Neither (fwd or bwd) is optimal (both are supralinear)
- Both suffer from the additional time with `append/3`
- How can we handle that? TBD soon

Insert in BST

```
% insert_bst/3(+Tree, +Key, -OutTree).
```

Q: is the order of arguments relevant ? Why/why not?

```
insert_bst(nil, K, n(nil,K,nil)):-!. %insert_bst(T, K, R):-T=nil,!, R = t(nil,K,nil).
insert_bst(n(Left,K,Right), K, n(Left,K,Right)):-!,
    write("in tree").
insert_bst(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-
    K<Key,!,
    insert_bst(Left,K,NewLeft).
insert_bst(n(Left,Key,Right), K, n(Left,Key,NewRight)):-
    insert_bst(Right,K,NewRight).
```

Qs:

1. Where is inserted (position)? ALWAYS LEAF! NO EXCEPTION!
2. What does ! in clause 1 negate? In clause 2?

Time estimate: a=1, b=2 (if balanced), c=0 =>O(lgn)

Q: Estimate best/worst case (situation) and time

Insert in regular (not search) tree

```
% insert_tree/3(+Tree, +Key, -OutTree).  
  
insert_tree(nil, K, n(nil,K,nil)):-!. % insert_tree(Tin, K, Tout):-Tin=nil, !, Tout=t(nil,Key,nil).  
insert_tree(n(Left,K,Right), K, n(Left,K,Right)):- % insert_tree(Tin,K,Tout):-  
    !, write("in tree"). % Tin=t(Left,K,Right),!,write("in tree"),Tout= t(Left,K,Right).  
insert_tree(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-  
    insert_tree(Left, K, NewLeft).  
insert_tree(n(Left,Key,Right), K, n(Left,Key,NewRight)):-  
    insert_tree(Right, K, NewRight).
```

- **Qs:**
 - **1.** Where is inserted (position)?

Search in BST (search Node by Key)

Tree stored as a fact `tree/1` in knowledge base

infix notation: `tree(n(n(nil, 5, nil), 7, n(nil, 9, nil)))`.

```
% search_bst/2 (+Key, +Tree)
```

```
search_bst(Key, n(_, Key, _)).
```

```
search_bst(Key, n(L, K, _)):-
```

```
    Key < K, !,
```

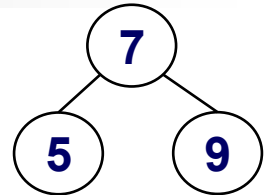
```
    search_tree(Key, L).
```

```
search_bst(Key, n(_, _, R)):-
```

```
    search_bst(Key, R).
```

```
?- tree(T), search_bst(5, T).
```

```
true, T=...
```



Computer Science

Search in BST

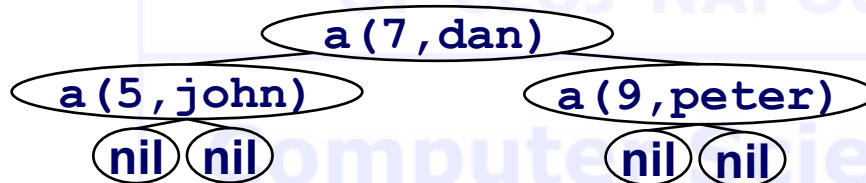
Assume the node contains a structure of type `a(Key,Value)`

The tree is BST based on key

Ex:

```
tree(
  n(
    n(nil, a(5,john), nil),
    a(7,dan),
    n(nil, a(9,peter), nil)
  )
).
```

Represents a tree with 7 and 2 leaves (5 and 9).



If `tree/1` is stored in the knowledge base, processing can be done with:

```
?- tree(T), process(T, ...).
```

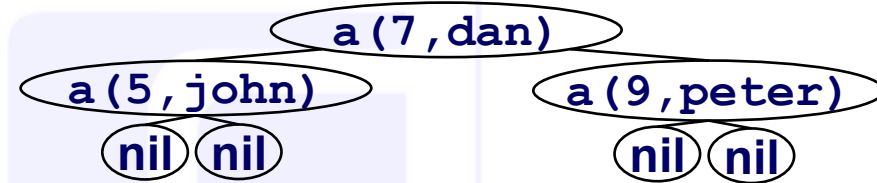
Search in BST (search Node by Key)

```
%search_bst/2 (+Node, +Tree)

search_bst(N, n(_,Node,_)):-
    eq(N, Node),!,
    N=Node.
search_bst(N, n(Left,Node,_)):-
    ord(N, Node),!,
    search_bst(N, Left).
search_bst(N, n(_,_,Right)):-
    search_bst(N, Right).

eq(a(K,_),a(Key,_)):-
    nonvar(K),
    K=Key,! .

ord(a(K,_),a(Key,_)):-
    nonvar(K),
    K<Key,! .
```



Search in BST (search Node by Key)

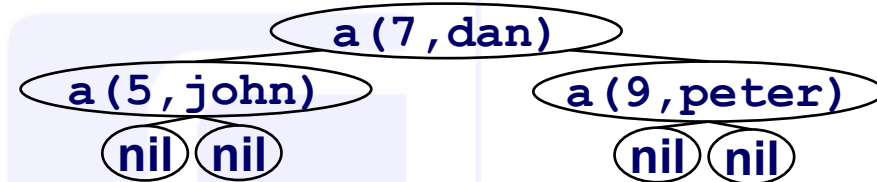
```
eq(a(K, _), a(Key, _)) :-  
    nonvar(K),  
    K=Key, !.
```

- Predicate to check if the *key* we are *searching* for has the **same** value as the *key* in the *current node* in the tree.
- Qs:
 - What is the meaning of `nonvar(K)` and why is needed?
 - "!" is not mandatory! Why?

```
ord(a(K, _), a(Key, _)) :-  
    nonvar(K),  
    K<Key, !.
```

- Predicate to check if the key *K* we are *searching* for has a **smaller** value than the key *Key* in the *current node* in the tree.
- Qs:
 - What is the meaning of `nonvar(K)` and why is needed?
 - "!" is not mandatory! Why?

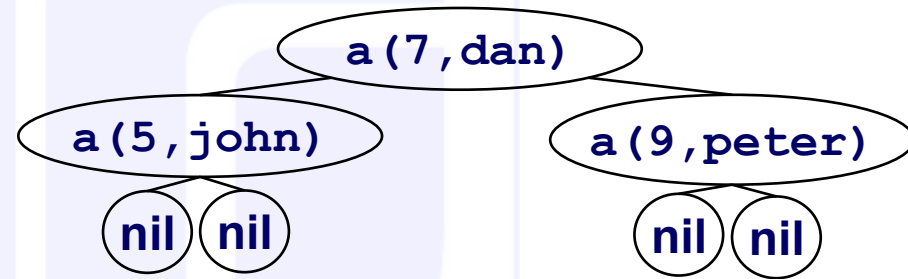
```
?- tree(T), search_bst(a(9, X), T).  
true, T=..., X=peter.
```



Search in regular tree (search Node by Value)

```
% search_tree/2 (+Key, +Tree)

search_tree(N, n(_,N,_)).
search_tree(N, n(Left,_,_)):-
    search_tree(N, Left).
search_tree(N, n(_,_,Right)):-
    search_tree(N, Right).
```



```
?- tree(T), search_tree(a(R,john),T).
true, T=..., R=5.
```

% T would be the tree read, R the key assoc. to the value (name) provided

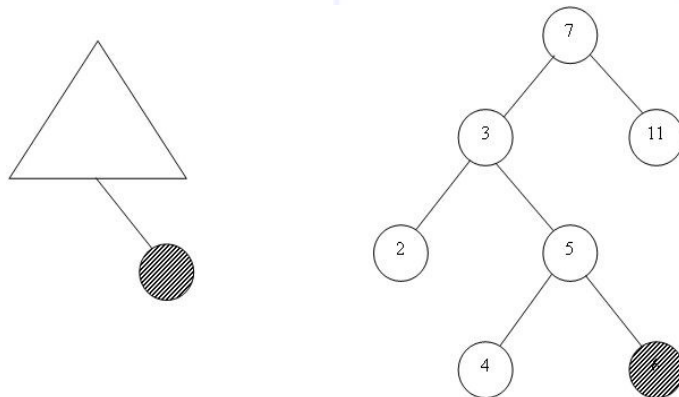
```
?- tree(T), search_tree(a(9,X),T).
```

% can we ask this way? What's the answer? What's the meaning?
true, T=..., X=peter

Delete from BST

- Search for the node with that key + remove the node
- Delete a key = remove the node containing the key
- Cases to consider:
 - Leaf – just remove the node
 - One child node – shortcut the node (link the child to its grandparent)
 - Two children ($\Leftrightarrow$ delete the root after the search stage) different story

Case 1



Case 2

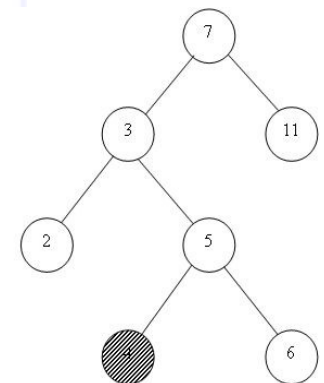
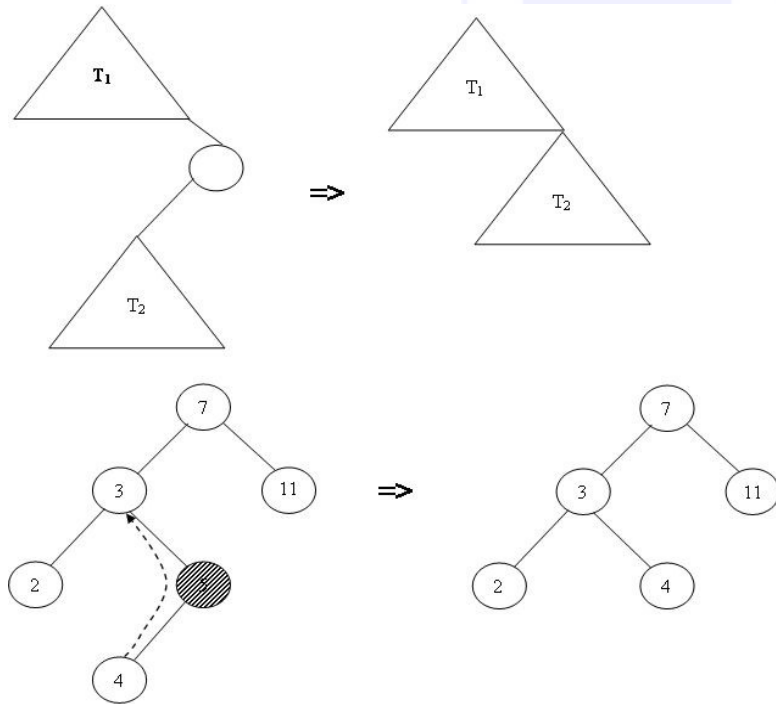


Figure. Removal of leaves

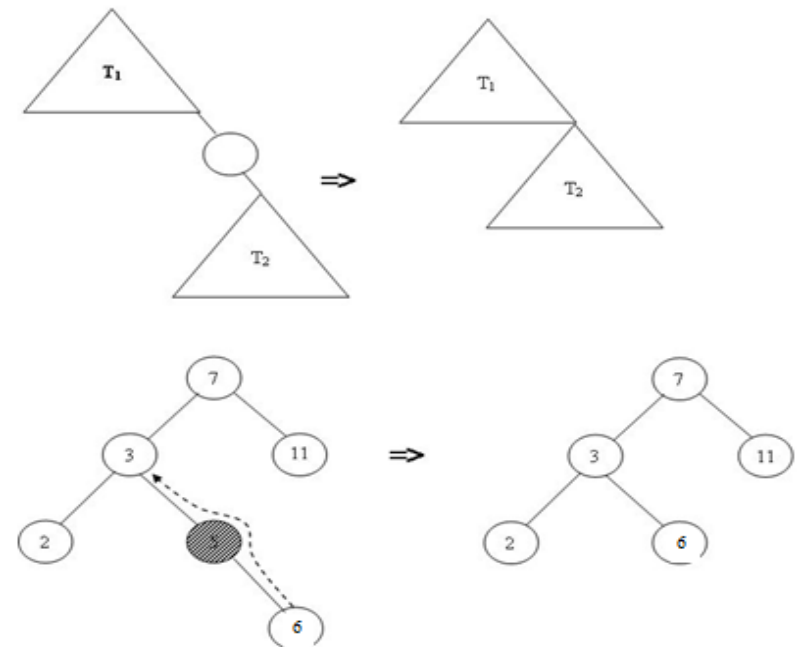
Delete from BST

Removal of a right child node with just one subtree

Case 3

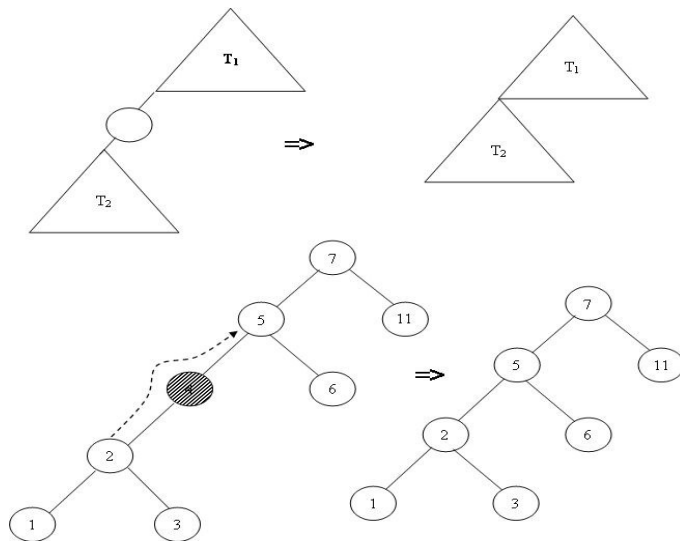


Case 4



Delete from BST

Case 5



Case 6

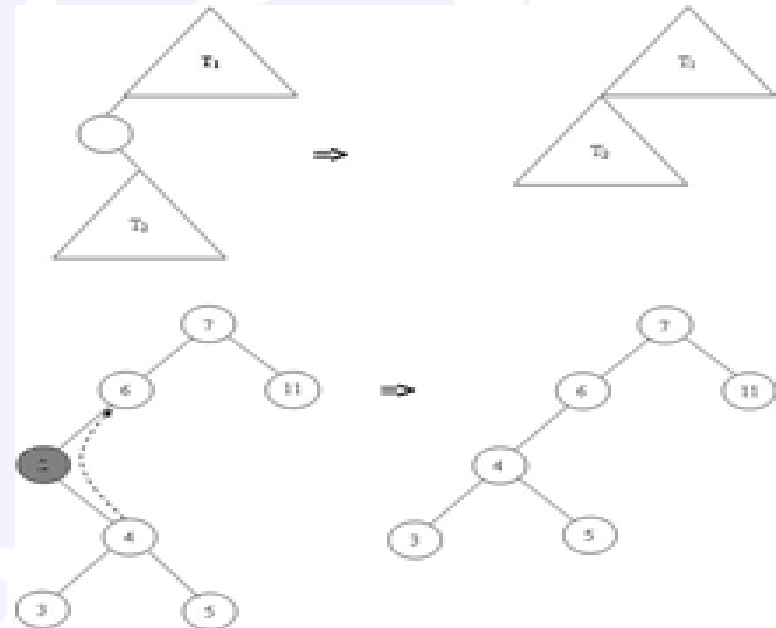


Figure. Removal of a left child node with just one subtree

- How about root? Postpone for the moment.
- Let's reach this point with code.

Delete from BST - code

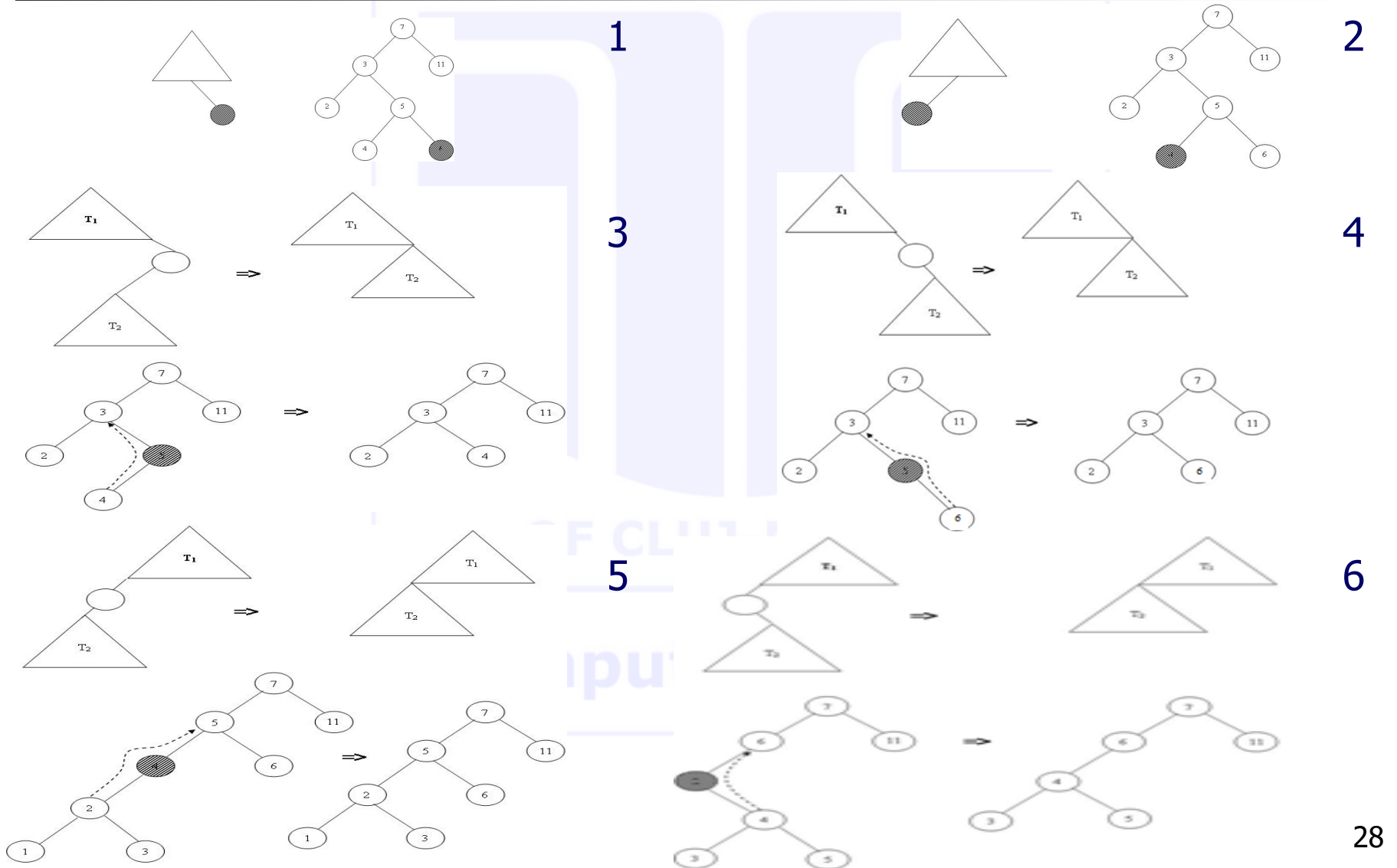
```
% delete_bst/3 (+Tree, +Key, +OutTree)

% search key; find the node to contain the key
delete_bst(n(Left,Key,Right),K,n(NewLeft,Key,Right)):-
    K<Key,!,
    delete_bst(Left,K,NewLeft).
delete_bst(n(Left,Key,Right),K,n(Left,Key,NewRight)):-
    K>Key,!,
    delete_bst(Right,Key,NewRight).

% found key; let's solve the easy cases discussed before
delete_bst(nil, K, nil):-!,
    write(K), write('not in tree').
delete_bst(n(nil, K, nil), K, nil):-!.
delete_bst(n(nil, K, Right), K, Right):-!.
delete_bst(n(Left, K, nil), K, Left):-!.

% "difficult" case postponed
```

Delete cases



Delete from BST - code

```
% delete_bst/3 (+Tree, +Key, +OutTree)
[d1] delete_bst(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-
    K<Key,!,
    delete_bst(Left, K, NewLeft).
[d2] delete_bst(n(Left,Key,Right), K, n(Left,Key,NewRight)):-
    K>Key,!,
    delete_bst(Right, K, NewRight).

[d3] delete_bst(nil, K, nil):-!,
    write(K),write('not found').
[d4] delete_bst(n(nil,K,nil), K, nil):-!.
[d5] delete_bst(n(nil,K,Right), K, Right):-!.
[d6] delete_bst(n(Left,K,nil), K, Left):-!.
```

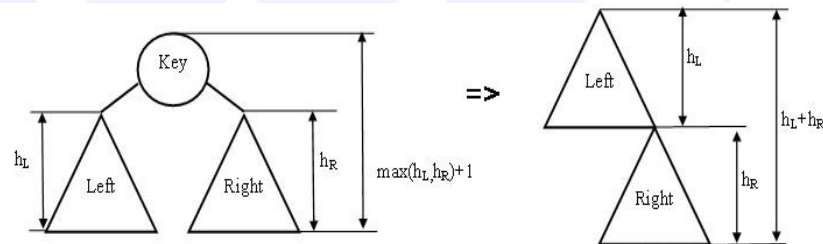
Qs:

- What cases did we cover so far?
- Which clause covers which case? Explain!
- Do we need all clauses so far? Why/why not? Justify

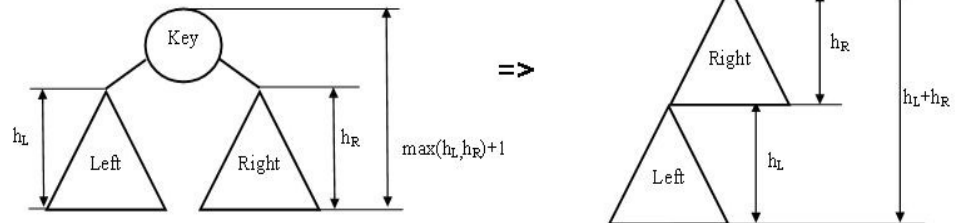
Delete from BST – code

- We reached the node containing the key to delete
- Has 2 children = is the root of a subtree =>
 - problem becomes to remove the root of the tree
- How to...? We have alternatives:
 - Either link the subtrees:
 - Easy to apply (time estimate)
 - Disadvantage?

Sol1



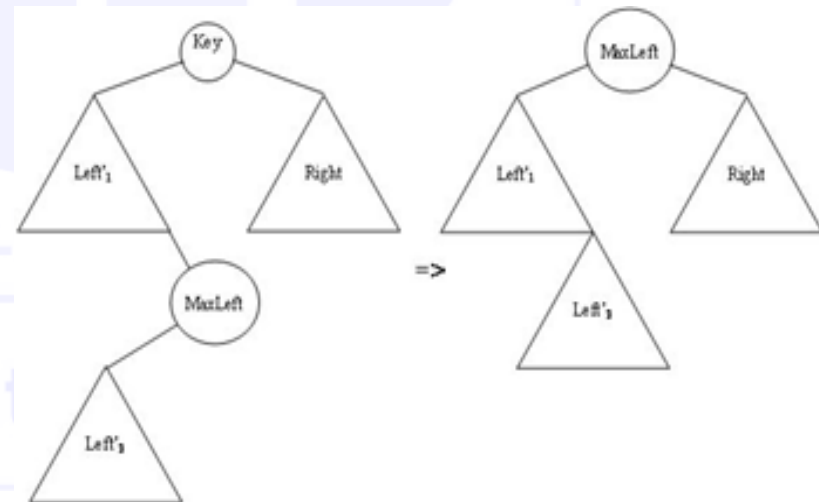
Sol2



Delete from BST – code

- Or replace problem to solve with a problem you know how to solve!
- Replace **INSTEAD** the node you want to remove (the root) with another one. How:
 - Find an easy to remove node
 - Place it instead of the root (is possible?)
 - To be possible, that node should be a node in inorder:
 - Either before the key in the root
 - Or after the key in the root
 - So, replace with
 - Predecessor
 - Successor
 - !
 - Are those nodes with the desired property?
 - Justify

Sol3



Delete from BST – code

```
delete_bst(n(Left,Key,Right), Key, n(NewLeft,MaxLeft,Right)):-!,
    deleteMaxFromTree(Left,MaxLeft,NewLeft).
```

```
% deleteMaxFromTree/3(In_tree,Max_key,Out_Tree).
```

- The predicate takes a tree as input `In_tree`, finds the max key `Max_key` (the rightmost node) and removes it from the tree, returning the tree `Out_tree` without that key.

```
deleteMaxFromTree(n(Left,Key,Right), MaxKey, n(Left,Key,NewRight)):-
    deleteMaxFromTree(Right,MaxKey,NewRight).
deleteMaxFromTree(n(Left,MaxKey,nil), MaxKey, Left):-!.
```

- Clause 1 says: as long as the right subtree is not empty, go to the right
- Clause 2 says: if right is `nil`, you found the max key `MaxKey` and `Left` is out tree.
- Do we ever reach second clause? Infinite loop. Clauses are not in the correct order! Let's change it!

Delete from BST – code

```
deleteMaxFromTree (n (Left, MaxKey, nil), MaxKey, Left) :- !.  
deleteMaxFromTree (n (Left, Key, Right), MaxKey, n (Left, Key, NewRight)) :-  
    deleteMaxFromTree (Right, MaxKey, NewRight) .
```

- It is inefficient. We always try the non-useful clause first
- Does it really need to have the fact first?
- No, it can go second (JUSTIFY!).

```
deleteMaxFromTree (n (Left, Key, Right), MaxKey, n (Left, Key, NewRight)) :-  
    deleteMaxFromTree (Right, MaxKey, NewRight) .  
deleteMaxFromTree (n (Left, MaxKey, nil), MaxKey, Left) :- !.
```

Computer Science

Delete from BST – complete code

% search

```
delete_bst(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-  
    K<Key, !,  
    delete_bst(Left, K, NewLeft).  
delete_bst(n(Left,Key,Right), K, n(Left,Key,NewRight)):-  
    K>Key, !,  
    delete_bst(Right, K, NewRight).
```

% cases of no children or 1 child

```
delete_bst(nil, K, nil):- !, write(K), write('not found').  
delete_bst(n(nil,K,nil), K, nil):- !.  
delete_bst(n(nil,K,Right), K, Right):- !.  
delete_bst(n(Left,K,nil), K, Left):- !.
```

% case of 2 children (starts search from predecessor on Left)

```
delete_bst(n(Left,Key,Right), Key, n(NewLeft,MaxLeft,Right)):-!,  
    deleteMaxFromTree(Left,MaxLeft,NewLeft).
```

% search for max (always go on the Right, stop when nil is reached)

```
deleteMaxFromTree(n(Left,Key,Right),MaxKey,n(Left,Key,NewRight)):-  
    deleteMaxFromTree(Right,MaxKey,NewRight).  
deleteMaxFromTree(n(Left,MaxKey,nil),MaxKey,Left):-!.
```

Delete from BST – code analysis

- Search phase: $O(h)$
- Delete (no matter the solution/case chosen): $O(h)$
- Overall: $2h$
- $O(h)$, constant 2. Is it so?
- It is just $O(h)$, constant 1. JUSTIFY!

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #6

Incomplete structures

Agenda

- **Incomplete structures**
 - Meaning & utility
 - Lists
 - Trees

Incomplete structures

Lists

- Incomplete structures:
 - Structures ending in variables
 - = instead of the specific empty structure it ends in a variable.
- This **IS** a list ending in a variable: $L_1 = [1, 2, 3 | \mathbf{A}]$
 - Tail is a VARIABLE
 - Can you specify its length?
 - Tail can be anything (including a list, such as $[4, 5]$). BUT in this case, it unifies with the variable, and the structure would be a homogeneous one: $L_1 = [1, 2, 3, \mathbf{4}, \mathbf{5}]$
- This **IS NOT** a list ending in a variable $L_2 = [1, 2, 3, \mathbf{B}]$
 - Only the last element is a variable, which can be anything (including a list, such as $[4, 5]$). BUT in this case, it unifies the variable, the structure would be a heterogeneous one:
 $L_2 = [1, 2, 3, \mathbf{[4, 5]}]$

Incomplete Lists - search

```
member(X, [X|_] ).  
member(X, [_|T]) :-  
    member(X, T) .
```

```
[q1] ?- L=[1,2,3|_], member(2,L) .  
true, L=[1,2,3|_]
```

```
[q2] ?- L=[1,2,3|_], member(4,L) .  
true, L=[1,2,3,4|_]
```

- Wrong behavior. Why? How can we fix the issue?

Incomplete Lists - search

```
member_IL(X, [X|_]) :- !.           % item found, stop.
member_IL(X, [_|T]) :-             % item not found,
    member_IL(X, T).               % go search in tail
member_IL(_, L) :-                 % reached final free variable
    var(L), !,                     % stop with failure.
    fail.
```

```
[q1] ?- L=[1,2,3|_], member_IL(2,L).
true, L=[1,2,3|_]
```

```
[q2] ?- L=[1,2,3|_], member_IL(4,L).
true, L=[1,2,3,4|_]
```

- Again, same wrong behavior. How come we didn't fix the issue?
- It NEVER reached clause #3? Why?

Incomplete Lists - search

```
member_IL(_, L) :-  
    var(L), !, % this order: check, cut, fail  
    fail.  
member_IL(X, [X|_]) :- !. % cut mandatory  
member_IL(X, [_|T]) :-  
    member_IL(X, T).
```

```
[q1] ?- L=[1,2,3|_], member_IL(2,L).  
true, L=[1,2,3|_]. % exe stops on clause 2
```

```
[q2] ?- L=[1,2,3|_], member_IL(4,L).  
false. % exe stops on clause 1
```

- **Stop conditions** for **incomplete** structures are (i) with **cut** (!) and (ii) **ALWAYS come first**. Otherwise, it behaves as if they are missing!
 - Starts with failure condition(s). It **MUST** be explicit (NEVER default).
 - Continues with the successful stop condition(s).

Incomplete Lists – insert

```
insert_IL(X,L):-  
    var(L),                % did it reach the final free variable?  
    !,                     % stop backtracking  
    L=[X|_].               % update the list with the new added item  
insert_IL(X,[X|_]):-!.     % item found, and don't allow backtracking  
insert_IL(X,[_|T]):-  
    insert_IL(X,T).
```

```
[q1] ?- L=[1,2,3|_], insert_IL(4,L).  
true, L=[1,2,3,4|_].
```

```
[q2] ?- L=[1,2,3|_], insert_IL(3,L).  
true, L=[1,2,3|_].
```

- q1: pure insert behavior (stop on clause 1)
- q2: member behavior (stop on clause 2)

Incomplete Lists – insert

- We don't really need clause 1. Clause 2 covers it (default).

```
insert_IL(X,L):-  
    var(L),!,  
    L=[X|_].  
insert_IL(X,[X|_]):- !.  
insert_IL(X,[_|T]):-  
    insert_IL(X,T).
```

```
[q1] ?- L=[1,2,3|_], insert_IL(4,L).  
true, L=[1,2,3,4|_].
```

Has pure *insert* behavior. Clause 1 behaves as:

```
insert_IL(X,L):- var(L), !, L=[X|_].
```

```
[q2] ?- L=[1,2,3|_], insert_IL(3,L).  
true, L=[1,2,3|_].
```

Has *member* behavior. Clause 1 behaves as:

```
insert_IL(X,[H|_]):- X=H, !.    % member(X,[X|_]):- !.
```

Incomplete structures - review

- Incomplete structures (lists, trees)
 - Replace the empty structure at the end **with a variable**
- Used as I/O structure (saves time and space)
- Change the behavior of regular predicates
- *Failure* conditions NEED to be:
 - ALL Explicit (no default failure in the absence of the clause)
 - Come FIRST (otherwise, is as if it does not exist)
- *Success* conditions should be:
 - All explicit (as before)
 - Start with the condition for the variable at the end
 - which comes right after the failure condition(s)

Search in BST

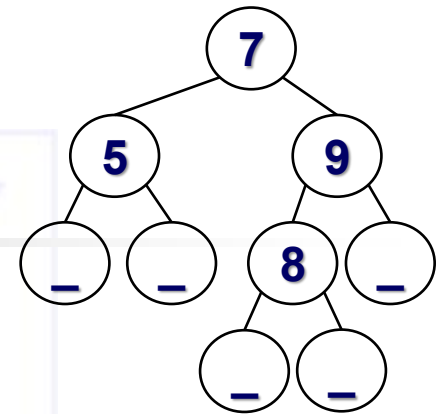
- This is an Incomplete (ending in variable) BST

```
tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
```

```
search(Key, t(Key,_,_)):-!,
    write("found").
```

```
search(Key, t(K,L,_)):-
    Key<K,!,
    search(Key, L).
```

```
search(Key, t(_,_,R)):-
    search(Key, R).
```



- The search predicate works well for a **regular** BST
- What behavior would it have in case of an incomplete tree?

Search in BST

- It has **dual** significance in the search process:
 - if **K present**, reports so
 - else **inserts** it as leaf

```
tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
```

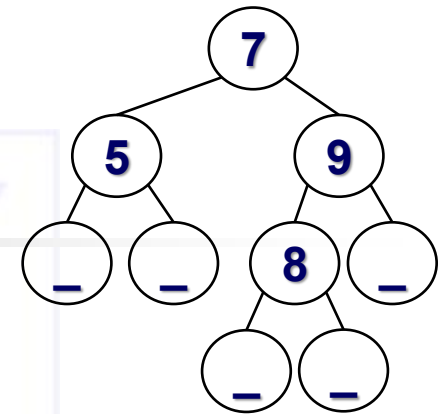
```
search_insert(Key,t(Key,_,_)):-!,
    write("found OR inserted").
```

```
search_insert(Key, t(K,L,_)):-
    Key<K,!,
    search_insert(Key, L).
```

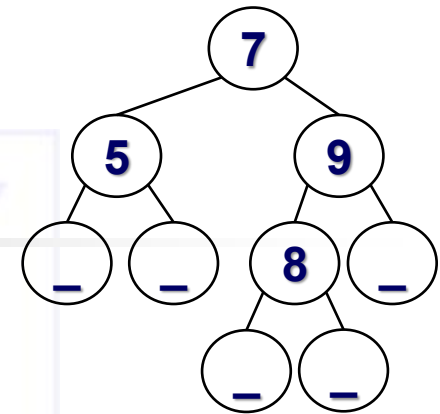
```
search_insert(K, t(_,_,R)):-
    search_insert(Key, R).
```

```
[q] ?- tree(T), search_insert(9,T).
      % behaves as search (= search_insert)
```

```
[q] ?- tree(T), search_insert(6,T).
      % behaves as insert (= search_insert)
```



Search in BST



```
[q] ?-tree(T),search_insert(9,T).
      % behaves as search (= search_insert)
```

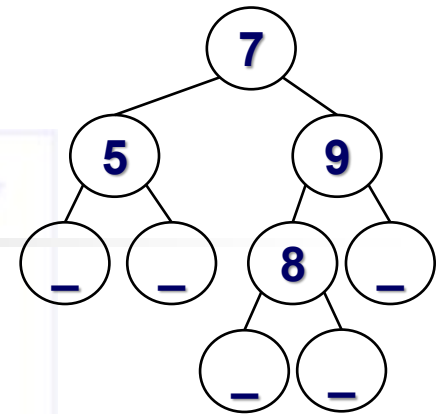
```
search_insert(Key,t(Key,_,_)):-!,
      write("found OR inserted").
```

The meaning of the clause is:

```
search_insert(Key,T):-
      nonvar(T), T=t(K,_,_), Key=K,!,
      write("found OR inserted").
```

```
search_insert(Key,t(Key,_,_)):-!,
      write("found").
search_insert(Key,t(K,L,_)):-
      Key<K,!,
      search_insert(Key,L).
search_insert(K,t(_,_,R)):-
      search_insert(Key,R).
```

Search in BST



```
[q] ?-tree(T),search_insert(6,T).
      % behaves as insert (= search_insert)
```

```
search_insert(Key,t(Key,_,_)):-!,
      write("found OR inserted").
```

The meaning of the clause is:

```
search_insert(Key,T):-
      var(T), !, T=t(Key,_,_),
      write("found OR inserted").
```

```
search_insert(Key,t(Key,_,_)):-!,
      write("found").
search_insert(Key,t(K,L,_)):-
      Key<K,!,
      search_insert(Key,L).
search_insert(K,t(_,_,R)):-
      search_insert(Key,R).
```

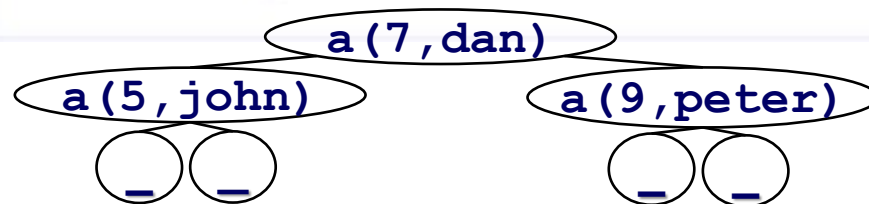

PURE Search in Incomplete BST (search by Key)

```
search_IT(Key, T) :-  
    var(T), !,  
    fail.  
search_IT(Key, t(Key, _, _)) :-  
    !,  
    write("found").  
search_IT(Key, t(K, L, _)) :-  
    Key < K, !,  
    search_IT(Key, L).  
search_IT(Key, t(_, _, R)) :-  
    search_IT(Key, R).
```

Ex:

```
tree(
  n(
    n(_, a(5, john), _),
    n(_, a(7, dan), _)
  ),
  n(_, a(9, peter), _)
).
```

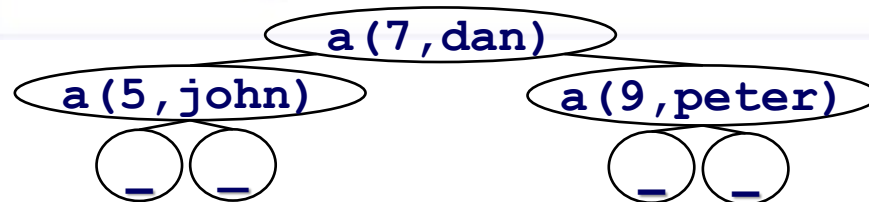
Represents the tree



Search in BST (search based on Key)

```
search_insert(Key, Value, n(_,a(Key,Value),_)):-!.
search_insert(Key, Value, n(L,a(K,_),_)):-
    Key<K, !,
    search_insert(Key, Value, L).
search_insert(Key, Value, n(_,_,R)):-
    search_insert(Key, Value, R).
```

```
[q] ?- tree(T), search_insert(9,V,T).
true, V=peter
```



Search in BST (search based on Key)

```
search_insert(Key, Value, n(_,a(Key,Value),_)):-!.
```

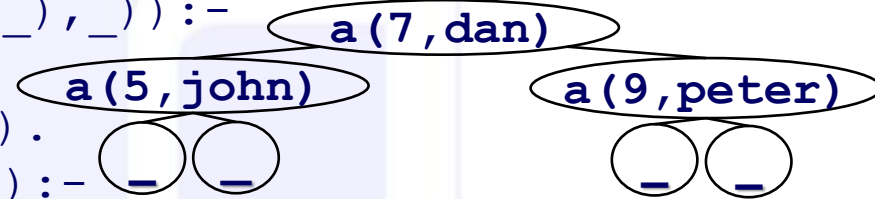
```
search_insert(Key, Value, n(L,a(K,_),_)):-
```

```
    Key<K, !,
```

```
    search_insert(Key, Value, L).
```

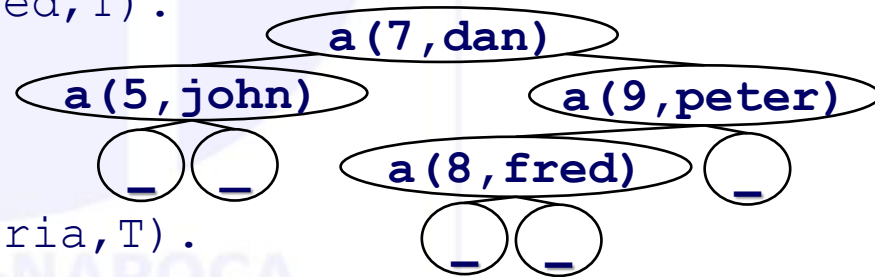
```
search_insert(Key, Value, n(_,_,R)):-
```

```
    search_insert(Key, Value, R).
```



```
[q] ?- tree(T), search_insert(8,fred,T).
```

```
true, ... % T increases
```

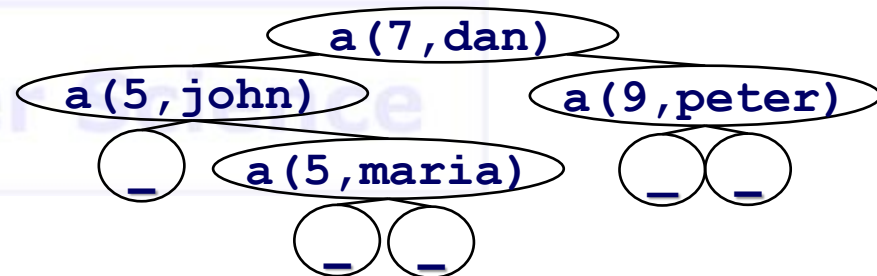


```
[q] ?- tree(T), search_insert(5,maria,T).
```

```
true, ... % T increases
```

• Does it fail?

- If Yes, How?
- If Not, Why? How is the tree affected?



Search/insert in Incomplete BST

- The regular search in a BST in an Incomplete BST has dual behavior:
 - Returns found - in the case it is in tree
 - Inserts it at the bottom level - in the case it is not found
- If you want to fail when the key is not found?
 - Add an explicit **first** clause as:

```
search(_, T) :-  
    var(T), !,  
    fail.
```

- What other situations are there?
 - Nodes contain <Key,Value>
 - search to fail if Key exists even with a different Value !

```
search(Key, Value, n(_, a(K, I), _)) :-  
    Key=K, !,  
    Value/=I, !,  
    fail.
```

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #7

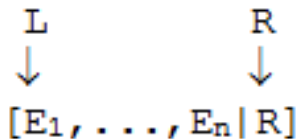
Difference Lists

Agenda

- **Difference lists**
 - Quicksort
 - queues
 - Flattening lists

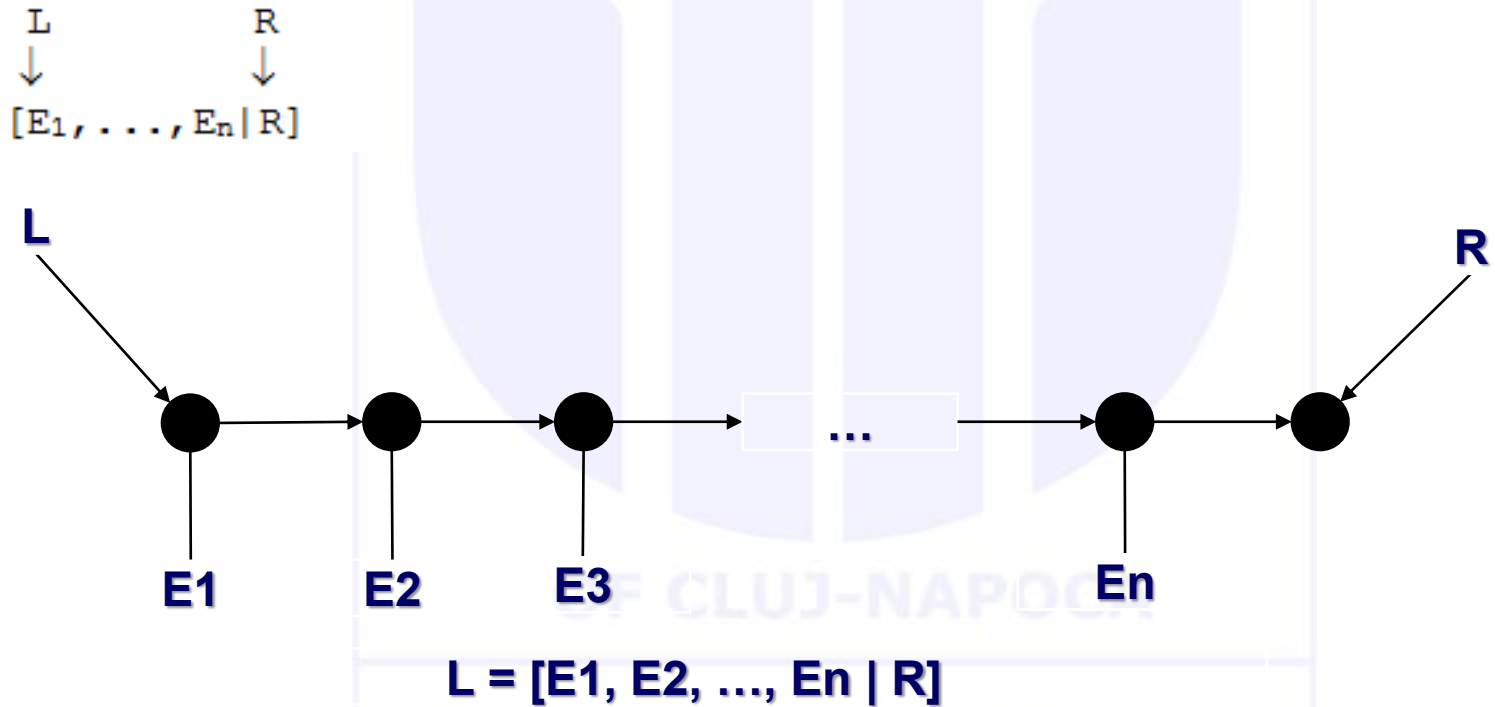
Difference Lists

- Another type of “irregular” structures
- Allow access to both the start and the END of lists 😊
- Named as lists with **Start** and **End** element
 - (that is, besides the variable =“pointer” at the beginning of the structure, we have a variable to “point” at the end of the list).
 - Similar to imperative languages where we have lists with pointers in the beginning and end of the list
- Allow for increased performance
- Difference list:

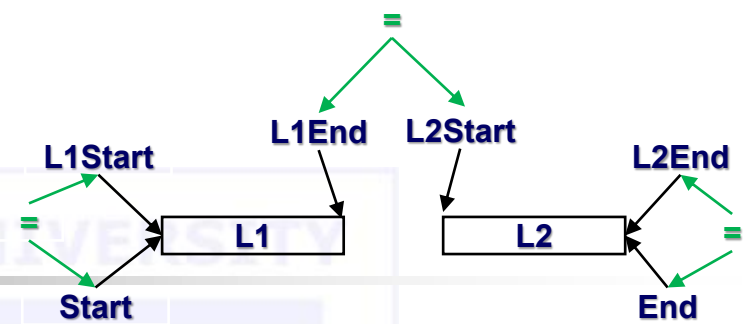


- D, denoted as L-R (sometimes, in code, it is preferred to be represented as 2 arguments, L and R) is:
 - $D = L - R = [E_1, \dots, E_n]$
 - L “points” at the start, R at the “end”

Difference Lists



Append



```
% append/3 (+List1, +List2, -OutList)
append([], L2, L2).
append([H|T], L2, [H|R]):-
    append(T, L2, R).
```

For Difference Lists (DL), each list has 2 arguments.

```
% append_DL    % Number of arguments?
% /6 (+L1Start, +L1End, +L2Start, +L2End, -OutStart, -OutEnd)
append_DL(LS1, LE1, LS2, LE2, RS, RE):-
    RS = LS1,
    LE1 = LS2,
    RE = LE2.
```

Append

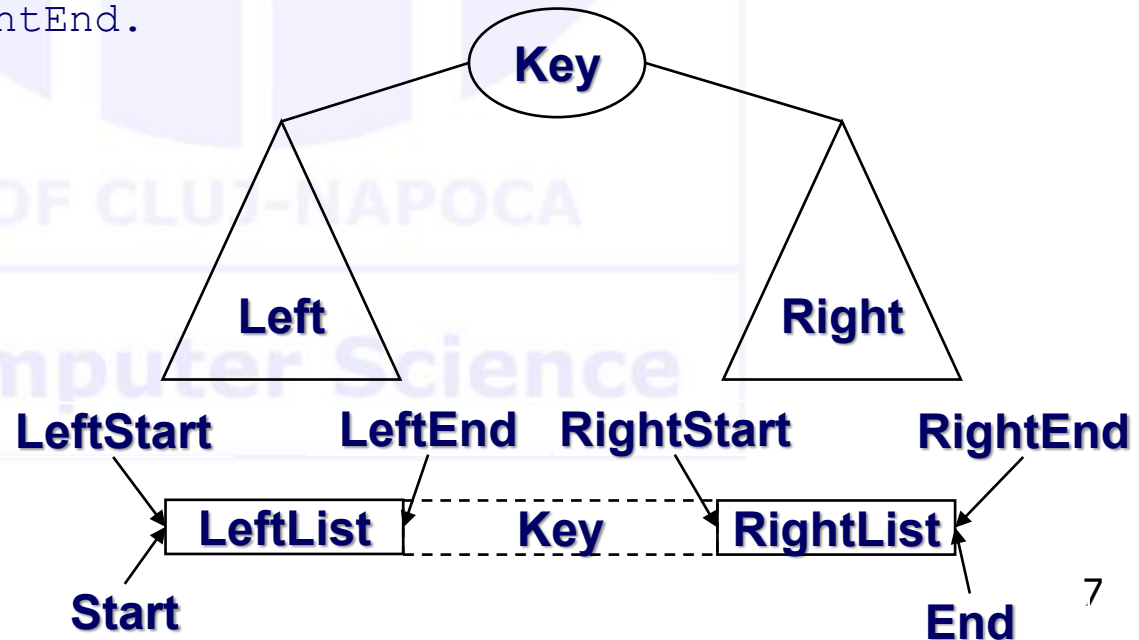
```
% append_DL/6 (+L1Start, +L1End, +L2Start, +L2End, -OutStart, -OutEnd)
append_DL(LS1, LE1, LS2, LE2, RS, RE):-
    RS = LS1,
    LE1 = LS2,
    RE = LE2.

?- append_DL([1,2|E1], E1, [3,4|E2], E2, S, E).
    % S = [1,2|E1],
    % E1 = [3,4|E2],           % → S=[1,2|[3,4|E2]]=[1,2,3,4|E2]
    % RE = E2.                 % → S=[1,2,3,4|E]
S=[1,2,3,4|E]
```

Tree to list (remember solutions in Lecture 5)

% inorder/3 (+Tree, -DifferenceListStart, -DifferenceListEnd)

```
inorder(nil, Start, End):- Start = End.
inorder(t(Left,Key,Right), Start, End):-
    inorder(Left, LeftStart, LeftEnd),
    inorder(Right, RightStart, RightEnd),
    Start      = LeftStart,
    LeftEnd    = [Key|RightStart],
    End        = RightEnd.
```



Tree 2 list forward – code (Lecture #5)

% inorder1/3 (+Tree, +PartialResultList, -FinalResultList)

```
inorder1 (nil, L, L) .
```

```
inorder1 (n (Left, Key, Right), Lin, Lout) :-  
    inorder1 (Left, Lin, LLout),  
    append (LLout, [Key], LRin),  
    inorder1 (Right, LRin, Lout) .
```

```
inorder1 (Tree, Result) :-  
    inorder1 (Tree, [], Result) .
```

Computer Science

Tree to list

(same solution – check Lecture #5)

- Needs specialized call

```
inorder(ITree, OList) :-  
    inorder(ITree, OList, []).
```

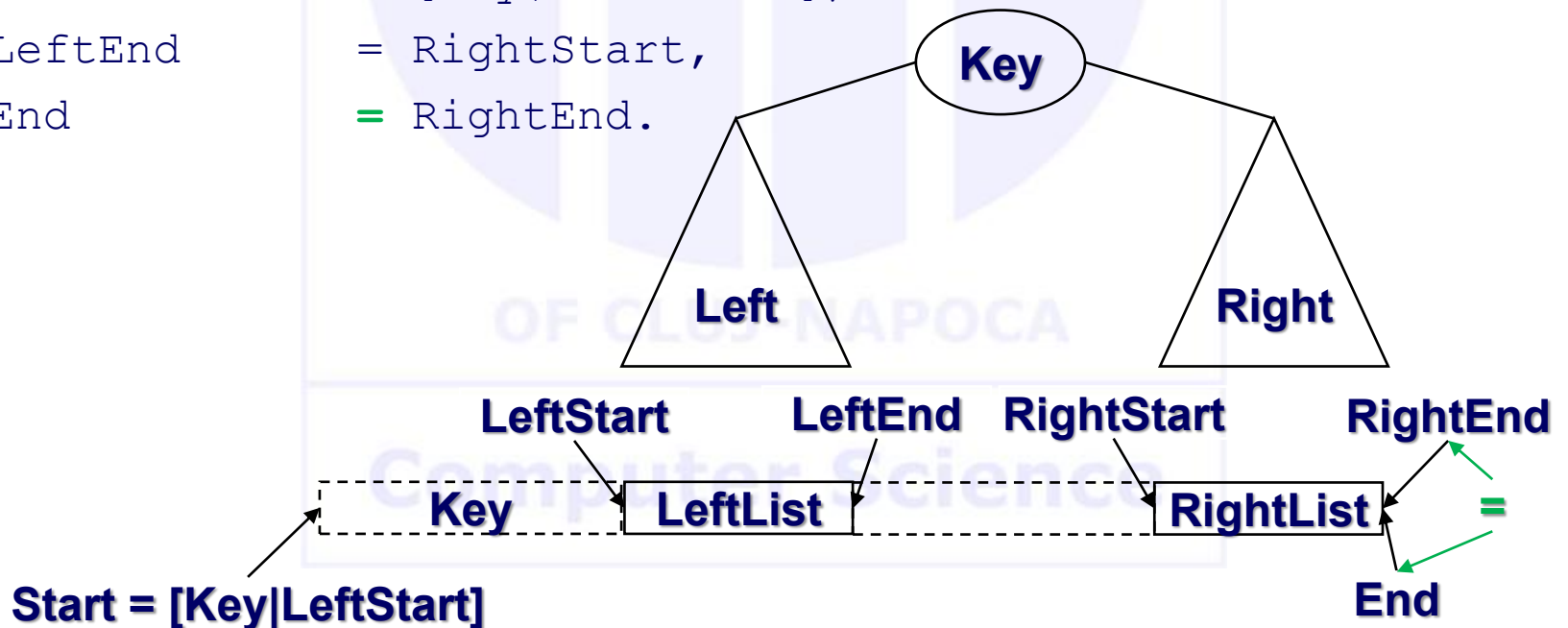
- Replace explicit unifications with implicit ones.

```
inorder(nil, L, L).  
inorder(t(Left, Key, Right), LeftStart, RightEnd) :-  
    inorder(Left, LeftStart, [Key | Intermediate]),  
    inorder(Right, Intermediate, RightEnd).
```

- Same as before, yet the utilization of the same name for variables (LeftStart) instead of making explicit unifications (Start= LeftStart and the other 2).

Pre-order

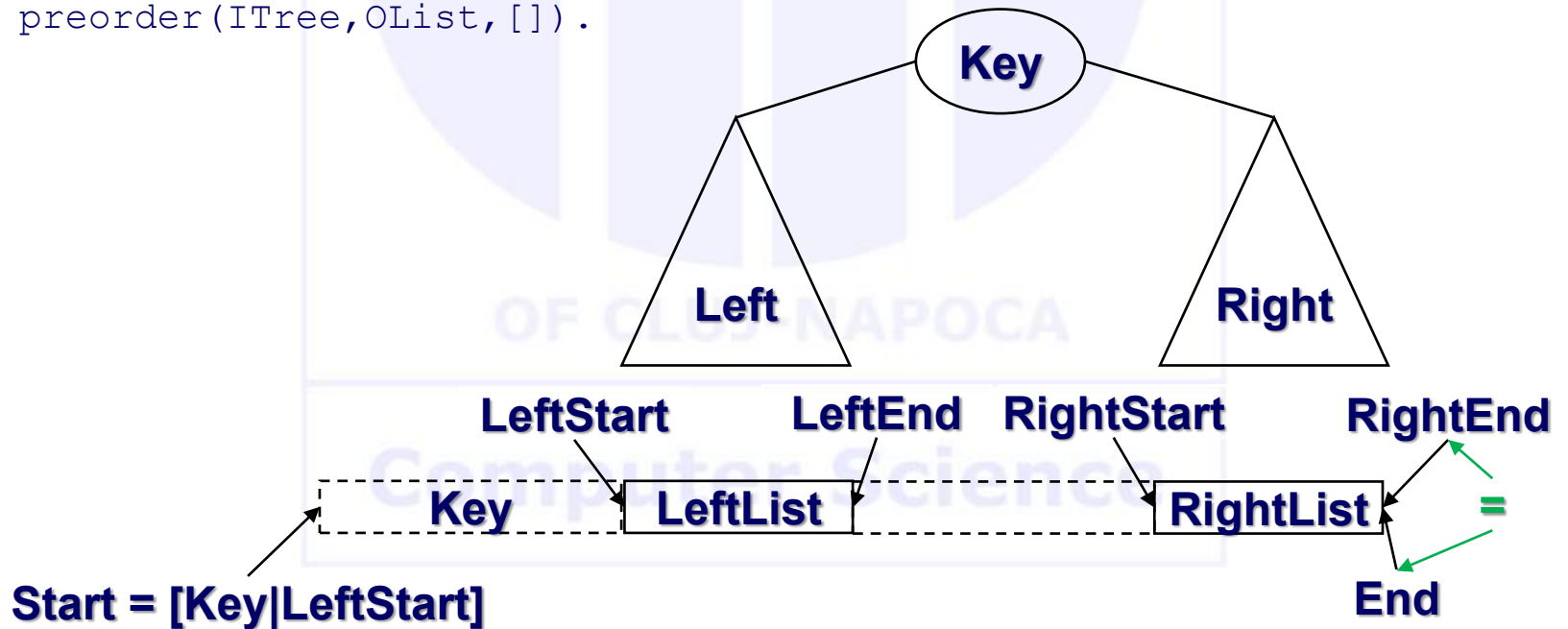
```
preorder(nil, Start, End):- Start = End.
preorder(t(Left, Key, Right), Start, End):-
    preorder(Left, LeftStart, LeftEnd),
    preorder(Right, RightStart, RightEnd),
    Start      = [Key|LeftStart],
    LeftEnd    = RightStart,
    End        = RightEnd.
```



Pre-order

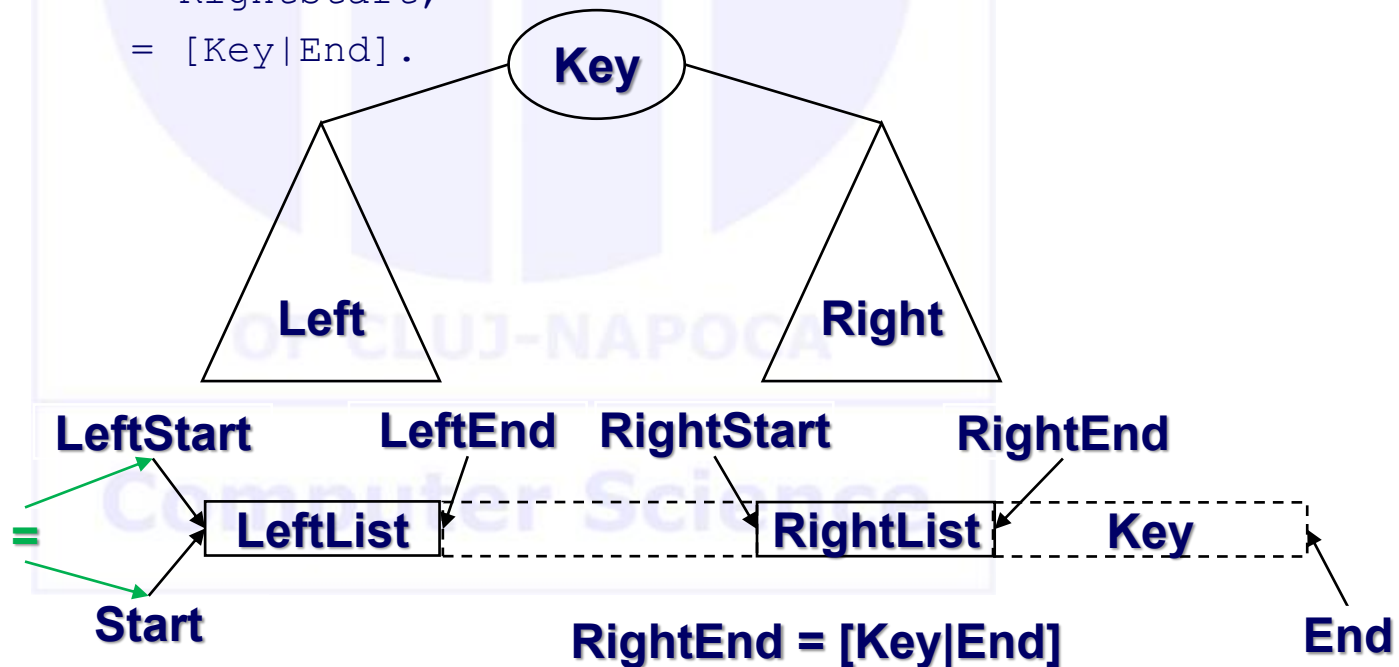
```
preorder(nil, L, L) .
preorder(t(Left, Key, Right), [Key | LeftStart], RightEnd) :-
    preorder(Left, LeftStart, Interm),
    preorder(Right, Interm, RightEnd) .

preorder(ITree, OList) :-
    preorder(ITree, OList, []).
```



Post-order

```
postorder(nil, Start, End):- Start = End.
postorder(t(Left,Key,Right), Start, End):-
    postorder(Left, LeftStart, LeftEnd),
    postorder(Right, RightStart, RightEnd),
    Start      = LeftStart,
    LeftEnd    = RightStart,
    RightEnd   = [Key|End].
```



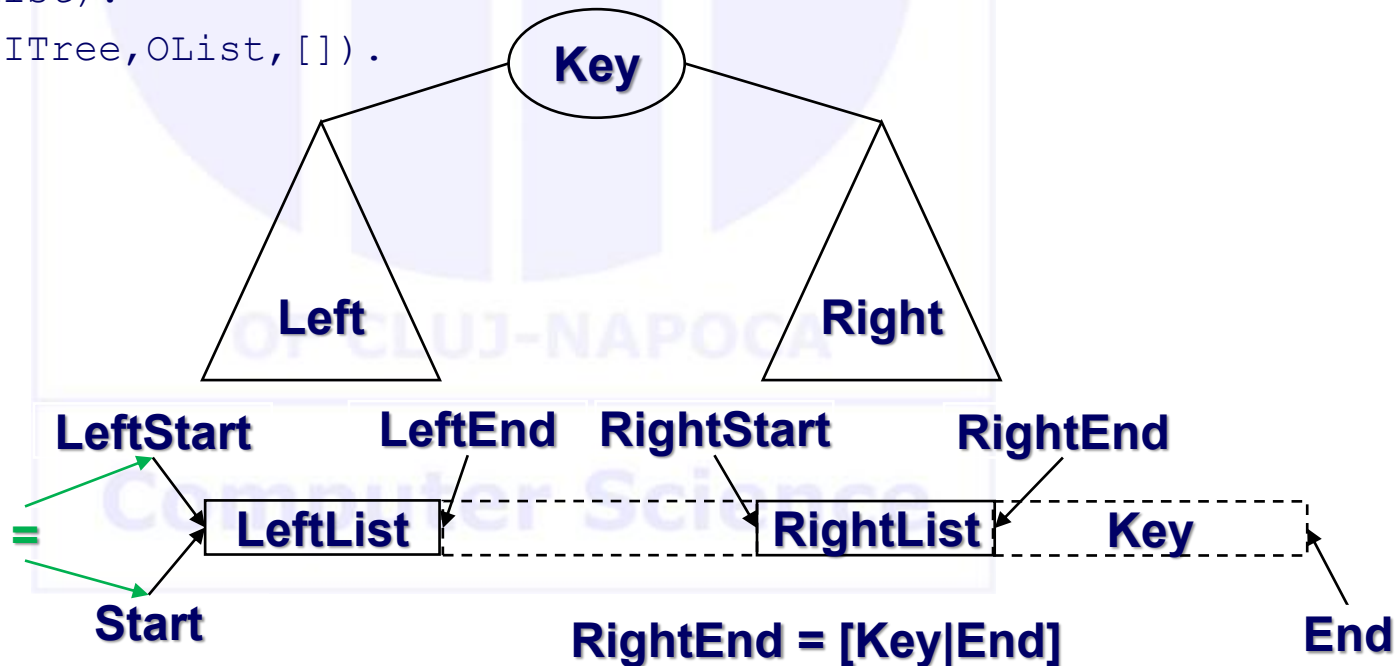
Post-order

```
postorder(nil, L, L).
```

```
postorder(t(Left, Key, Right), LeftStart, RightEnd) :-  
    postorder(Left, LeftStart, Intermediate),  
    postorder(Right, Intermediate, [Key | RightEnd]).
```

```
postorder(ITree, OList) :-
```

```
    postorder(ITree, OList, []).
```



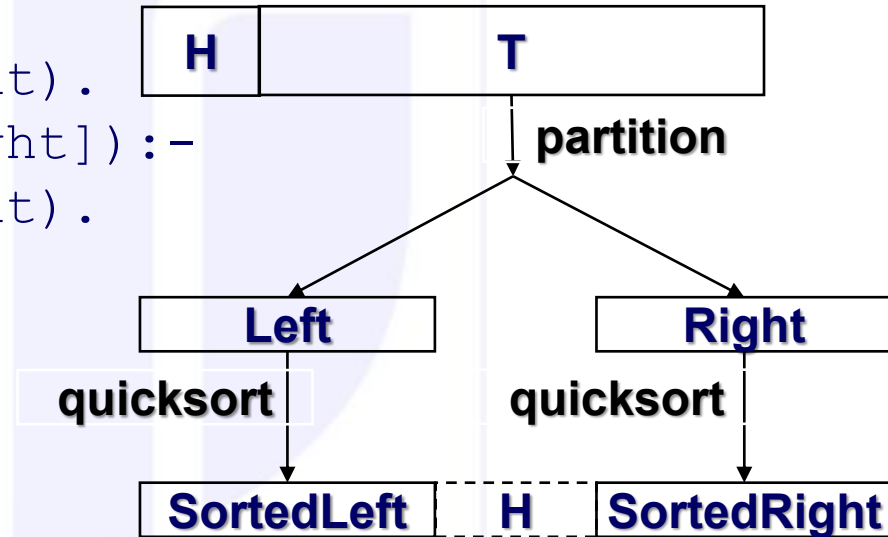
Difference lists

- Pair of arguments (Start,End) or Start/End, pointing to the **Start** and respectively AFTER the **End** element of the list
- Allow to speed up execution (avoid second traversal of list)
- Needs special initial call => make the warm up call by setting End on call to []
 - What if instead of [] we have some content?
- Stop condition: empty list as a difference
 - therefore, the difference empty list is between the same variable (L,L).

Quicksort

```
partition(X, [H|T], [H|Left], Right) :-
    H < X, !,
    partition(X, T, Left, Right).
partition(X, [H|T], Left, [H|Right]) :-
    partition(X, T, Left, Right).
partition(_, [], [], []).
```

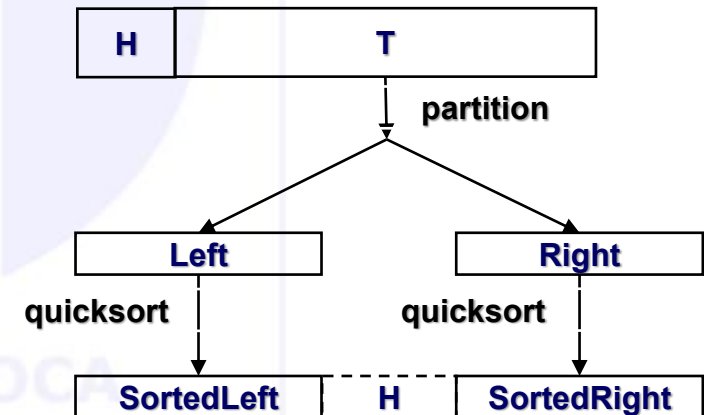
```
quicksort([H|T], Result) :-
    partition(H, T, Left, Right),
    quicksort(Left, SortedLeft),
    quicksort(Right, SortedRight),
    append(SortedLeft, [H|SortedRight], Result).
quicksort([], []).
```



Quicksort - analysis

- A divide et impera strategy
- Cost analysis per cases:
 - $O(n \lg n)$ in best and average case (why?)
 - $O(n^2)$ in worst (which?)
- Multiplicative constant larger than in imperative implementations!
 - Why?
 - How can this be avoided?

```
quicksort([H|T], Result) :-
    partition(H, T, Left, Right),
    quicksort(Left, SortedLeft),
    quicksort(Right, SortedRight),
    % how can we skip the append
    append(SortedLeft, [H|SortedRight], Result).
quicksort([], []).
```



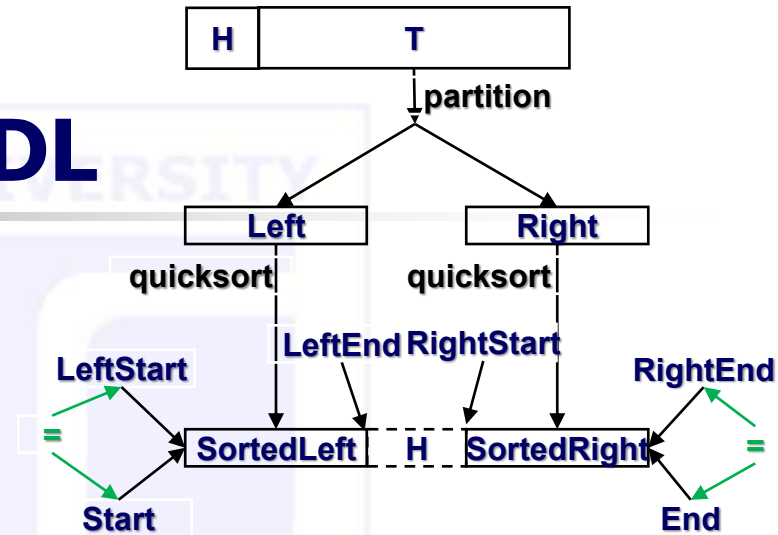
Quicksort – with DL

```
% quicksort_DL1/3
```

```
% (+List, -OutListStart, -OutListEnd)
```

```
quicksort_DL1([H|T], Start, End) :-
    partition(H, T, Left, Right),
    quicksort_DL1(Left, StartLeft, EndLeft),
    quicksort_DL1(Right, StartRight, EndRight),
    Start      = LeftStart,
    EndLeft    = [H|StartRight],
    RightEnd   = End.

quicksort_DL1([], L, L).
```



• Q:

- `partition` should not return difference lists? How should we do that?

14-Apr-2014 • Oral discussion.

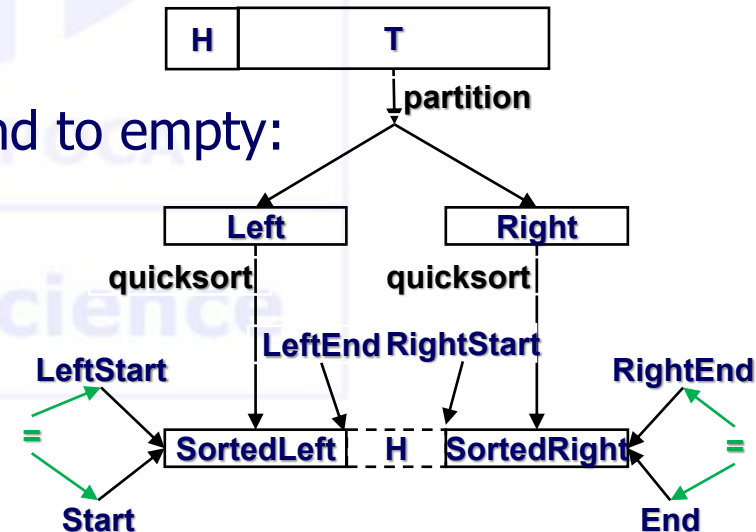
Quicksort – with DL (default unifications)

```
% quicksort_DL2/3 (+List, -OutListStart, -OutListEnd)
```

```
quicksort_DL2([H|T], Start, End) :-  
    partition(H, T, Left, Right),  
    quicksort_DL2(Left, Start, [H|Inter]),  
    quicksort_DL2(Right, Inter, End).  
quicksort_DL2([], List, List).
```

- Q: needs special initial call, to initialize End to empty:

```
quicksort2(List, Result) :-  
    quicksort_LD2(List, Result, []).
```



Queues with DL

(section 15.4 in The art of Prolog – Shapiro)

```
% enqueue/5 (+El2enQ, +QBeforeStart, +QBeforeEnd, -QAfterStart, -QAfterEnd)
```

```
enqueue (El, Start, [El | End], Start, End) .
```

Means:

```
enqueue (El, QS, QE, Start, End) :-
```

```
    QS=Start,           % after adding in the Q, it starts in the same place
```

```
    QE=[H | End],       % before adding, the list ends IN FRONT
```

```
    H=El.               % of the item to be added
```

How to query it?

```
?- enqueue (Item, QS, QE, Start, End) .
```

In the usual employment `QS` and `QE` are the Start/End element in the Queue before the call (known arguments), and the `Start/End` element in the Queue after the call (unknown arguments at call time).

Queues with DL

(section 15.4 in The art of Prolog – Shapiro)

```
% dequeue/5 (-El2deQ, +QBeforeStart, +QBeforeEnd, -QAfterStart, -QAfterEnd)
```

```
dequeue (El, [El | Start] , End, Start, End) .
```

Means:

```
dequeue (El, QS, QE, Start, End) :-  
    QS=[H | Int] ,    % extracted element H from the front of Q  
    El=H,             % assign it to our argument  
    Int=Start,        % Q without Start element is our StartQueueAfter  
    QE=End.           % list ends are the same as removal is from front
```

How to query it?

```
?- dequeue (Item, QS, QE, Start, End) .
```

- In the default meaning, `Item` gets instantiated at the end of the execution.
- What happens if we ask to dequeue some specific element (say 7), yet, it is NOT the Start in the queue? That is, we specify `Item`.

Types of lists

- Regular/Complete lists - ended in []
- Incomplete lists - ended in variable
- Difference lists
- Write specific predicates to make transformations among each of the 3 known types of lists (section 7.3 of the textbook):
 - **Complete to incomplete**
 - Complete to difference
 - Incomplete to complete
 - **Incomplete to difference**
 - Difference to complete
 - Difference to incomplete

Deep lists

- What is a deep list?

$L = [1, 1, [2, 2, 2, [3, 3, [4], 3], 2, 2, [3, [4, [5, 5, 5]]], 2]]$

- Levels? How do we know?

- Flatten deep lists. Meaning?

$L = [1, 1, 2, 2, 2, 3, 3, 4, 3, 2, 2, 3, 4, 5, 5, 5, 2]$

- How, without knowing structure / number of levels?

Flatten Deep lists

```
% deep2flat/2 (+DeepList, -FlatList).
```

```
deep2flat([], []) :- !.
```

```
deep2flat([H|T], [H| Tflat]) :-  
    atomic(H), !,  
    deep2flat(T, Tflat).
```

```
deep2flat([H|T], R) :-  
    deep2flat(H, Hflat),           % H is a list  
    deep2flat(T, Tflat),  
    append(Hflat, Tflat, R).
```

```
[q] ?- deep2flat([1,1,[2,2,2,[3,3,[4],3],2,2,[3,[4,[5,5,5]]],2]], R)  
R=  
    [1,1, 2,2,2, 3,3, 4, 3, 2,2, 3, 4, 5,5,5, 2]
```

Flatten Deep lists – with difference lists - explicit

```
% deep2flat_dl/3 (+DeepList, -FlatListStart, -FlatListEnd).
```

```
deep2flat_dl([], Start, End):- !, Start = End.
```

```
deep2flat_dl([H|T], Start, End):-  
    atomic(H), !,  
    deep2flat_dl(T, TStart, TEnd),  
    Start = [H|TStart],  
    End = TEnd.
```

```
deep2flat_dl([H|T], Start, End):-  
    deep2flat_dl(H, HStart, HEnd),  
    deep2flat_dl(T, TStart, TEnd),  
    Start = HStart,  
    HEnd = TStart,  
    End = TEnd.
```

Flatten Deep lists – with difference lists - default

```
% deep2flat_dl/3 (+DeepList, -FlatListStart, -FlatListEnd).
```

```
deep2flat_dl([], End, End) :- !.
```

```
deep2flat_dl([H|T], [H|Tflat], End) :-  
    atomic(H), !,  
    deep2flat_dl(T, Tflat, End).
```

```
deep2flat_dl([H|T], Start, End) :-  
    deep2flat_dl(H, Start, Int),  
    deep2flat_dl(T, Int, End).
```

```
flatten(Deep, Flat) :-  
    deep2flat_dl(Deep, Flat, []).
```

Computer Science

Flatten Deep lists – homework

- Try to flatten a deep list taking just elements with various properties:
 - Heads of lists ([1,2,3,4,3,4,5])
 - Atomic elements on End positions of lists ([4,3,5,2])
 - All atomic elements from lists at **odd/even** levels ([2,2,2,4,2,2,4,2])
 - Heads of lists at **odd/even** levels ([1,3,3,5])
 - Atomic elements on **odd/even** positions on any list
 - ...imagine 😊
 - (section 8.3 of the textbook)

`L=[1,1,[2,2,2,[3,3,[4],3],2,2,[3,[4,[5,5,5]]],2]]`

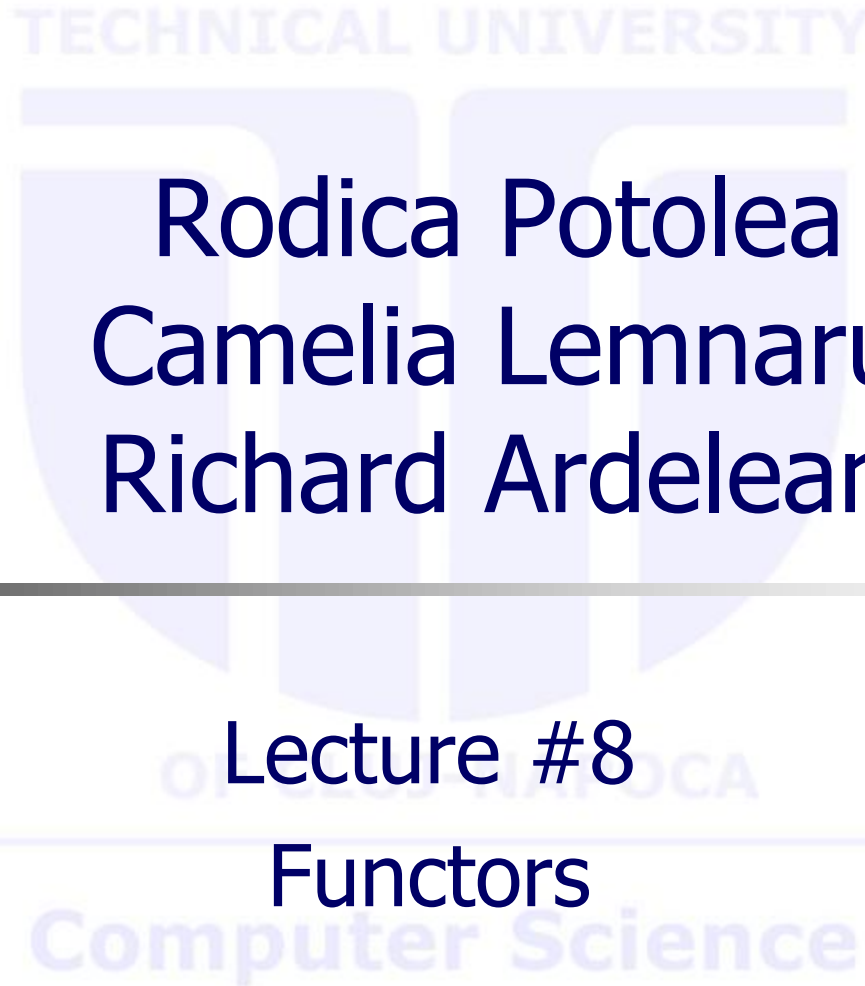
Computer Science

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #8

Functors



Agenda

- **Built in predicates**
 - `functor` and `arg` and `=..`
- **Utilization on complex tasks**
 - Substructure change

Creating/accessing structures

Built-in predicates

- `functor(T, F, N)`
 - means `T` is a structure named `F` with arity `N`
 - 2 functionalities:
 - **if** `nonvar(T)`
then `F` matches the functor of `T` and
`N` matches the arity of `T`
 - **if** `var(T)` **and** `nonvar(F)` **and** `nonvar(N)`
then `T` becomes a structure
with name `F` and `N` arguments

functor(T, F, N)

if nonvar(T) **then** F matches the functor of T
N matches the arity of T

```
?- functor(a+b,F,N).  
true, F=+, N=2.
```

```
?- functor(a,F,N).  
true, F=a, N=0.
```

```
?- functor([a,b,c],F,N).  
true, F=., N=2.    % Why 2? Which?
```

```
?- functor(f(a,b),F,N).  
true, F=f, N=2.
```

```
?- functor([a,b,c],F,3).  
false.            % Why?
```

`functor(T, F, N)`

if `var(T)` **and** `nonvar(F)` **and** `nonvar(N)`
then T becomes a structure with name F and N arguments

```
?- functor(F, f, 3) .  
true, F=f(_A, _B, _C) .
```

```
?- functor(F, +, 2) .  
true, F=_A+_B
```

Creating/accessing structures

Built-in predicates

- `arg(N, T, A)`
 - means the N^{th} argument of `T` is `A`
 - `N` and (partially) `T` must be instantiated

```
?- arg(2, f(a,b), X) .  
true, X=b.
```

```
?- arg(2, a+b+c, Z) .  
true, Z=b+c
```

```
?- arg(1, a+b+c, X) .  
true, X=a
```

```
?- arg(1, [a,b,c], X) .  
true, X=a
```

```
?- arg(3, a+b+c, Y) .  
false. % Why?
```

```
?- arg(2, [a,b,c], Y) .  
true, Y=[b,c]
```

Creating/accessing structures

Built-in predicates

$X = . . L$ % INFIX predicate (a.k.a. UNIV)

- means list L contains as first element the functor of X , followed by arguments of X
- 2 functionalities:
 - **if** $\text{var}(X)$ **then**
 L is used to construct the appropriate structure of X ;
the head of L must be an atom, and becomes the functor of X
 - **if** $\text{nonvar}(X)$ **then**
functor of X matches the first element (head) of L ,
while the arguments of X matches with the tail of
the list (left to right arguments of X , left to right in
tail of L)

$X = \dots L$

- **if** $\text{var}(X)$ **then** L is used to construct the appropriate structure of X ; the head of L must be an atom, and becomes the **functor** of X

```
?- X=..[a,b,c,d].  
true, X=a(b,c,d)
```

```
?- X=..[member,a,[b,c]].  
true, X=member(a,[b,c]).
```

$X = _ _ L$

- **if** `nonvar(X)` **then** functor of `X` matches the first element (head) of `L`, while the arguments of `X` matches with the tail of the list (left to right arguments of `X`, left to right in tail of `L`)

```
?- f(a,b,c)=..X.                % Can we ask with X on the left?
true, X=[f,a,b,c].
```

```
?- append([H|T],L,[H|R])=..X.
true, X=[append,[H|T],L,[H|R]].
```

```
?- (a+b)=..L.
true, L=[+,a,b].
```

```
?- (a+b+c)=..L.
true, L=[+,a,b+c]
```

```
?- [a,b,c,d]=..[H|T].          % Can we ask with [H|T] on the left?
true, H=., T=[a,[b,c,d]]
```


univ (= ..) implementation

```
univ(Eq, [F|Terms]) :-
    length(Terms, N),
    functor(Eq, F, N),           % Extract f and its arity
                                  % functor(f(a,b,c), f, 3)
    get_args(Eq, 0, N, Terms).
```

% extract each argument one by one based on a counter up to N (arity)

```
get_args(_, N, N, []) :- !.
get_args(Eq, C, N, [X|R]) :-
    C1 is C+1,
    arg(C1, Eq, X),              ?- f(a,b,c)=..L1, univ(f(a,b,c), L2).
                                  L1 = L2, L2 = [f, a, b, c]
    get_args(Eq, C1, N, R).      ?- F1=..[f,a,b,c], univ(F2, [f,a,b,c])
                                  F1 = F2 = f(a,b,c).
```

The predicates

```
?- functor(f(a,b,c), F, N).      ?- functor(F, f, 3).  
true, F=f, N=3.                  true, F=f(_1,_2,_3).
```

```
?- arg(2, f(a,b,c), X).  
true, X=b.
```

```
?- X=..[f,a,b,c].                ?- f(a,b,c)=..X.  
true, X=f(a,b,c).                true, X=[f,a,b,c].
```

Computer Science

Substituting expressions

- Given a (complex) expression, it is requested to replace a given subexpression with another one.

```
% subst /4 (+OldSubExpr, +NewSubExpr, +OldExpr, -NewExpr) .
```

Example: in a large arithmetic expression replace $(7-2*x)$ wherever it occurs with another one, say $(2*y+3/z)$

- 2 solutions
 - `subst1/4: =..`
 - `subst2/4: functor + arg`

```
% subst /4 (+OldSubExpr, +NewSubExpr, +OldExpr, -NewExpr) .
```

```
% subst_args /4
```

```
% (+OldSubExpr, +NewSubExpr, +ListOldArgs, -ListNewArgs) .
```

Substituting expressions

=..

```
subst1(Old, New, Old, New) :- !.           % in case the whole expression is represented
                                           % by only the subexpression to be replaced,
                                           % the old one is replaced by the new one

subst1(_, _, Old, Old) :-                  % in case the expression is an atomic Exprue
    atomic(Old), !.                        % it remains unchanged

subst1(Old, New, Expr, NewExpr) :-
    Expr=..[F|Args],                      % extract from the old expression its functor
                                           % and the list of arguments
    subst_args1(Old, New, Args, NewArgs), % take the old list of arguments and obtain the substitution
                                           % create the new expression with the same functor and the
    NewExpr=..[F|NewArgs].                % new list of arguments, updated by substitutions

subst_args1(_, _, [], []).
subst_args1(Old, New, [Arg|Args], [NewArg|NewArgs]) :-
    subst1(Old, New, Arg, NewArg),         % as Arg is an expression, go back
                                           % to the other predicate
    subst_args1(Old, New, Args, NewArgs).  % continue with the rest of
                                           % arguments, tail of list
```

Substituting expressions =..

Just code, comments removed

```
subst1(Old,New,Old,New):-!.
subst1(_,_,Old,Old):-
    atomic(Old),!.
subst1(Old,New,Expr,NewExpr):-
    Expr=..[F|Args],
    subst_args1(Old,New,Args,NewArgs),
    NewExpr=..[F|NewArgs].

subst_args1(_,_,[],[]).
subst_args1(Old,New,[Arg|Args],[NewArg|NewArgs]):-
    subst1(Old,New,Arg,NewArg),
    subst_args1(Old,New,Args,NewArgs).
```

Qs:

1. Order of processing the structure?
2. Where the execution ends?

[q] ?- subst1(a, x, f(a, b, g(a)), R).
R = f(x,b,g(x))

Substituting expressions functor+arg

```
subst2 (Old, New, Old, New) :-!.           % same as before
subst2 (_, _, Old, Old) :-                 % same as before
    atomic(Old), !.
subst2 (Old, New, Expr, NewExpr) :-
    functor(Expr, F, N),                  % extract from the old expression Expr its functor F
                                           % and the number of arguments N
    functor(NewExpr, F, N),               % create a new expression NewExpr from the
                                           % known functor F and N arguments, free variables at the time
    subst_args2(N, Old, New, Expr, NewExpr).

% for each of the arguments, 1, 2, ..., N, process it one at a time
subst_args2(0, _, _, _, _) :-!.           % arg 0 needs no change
subst_args2(N, Old, New, Expr, NewExpr) :-
    arg(N, Expr, OldArg),                  % extract the Nth arg of the old expression
    arg(N, NewExpr, NewArg),               % set the Nth arg of the new expression;
                                           % a var at the time
    subst2(Old, New, OldArg, NewArg),      % as OldArg is an expression, go back to the other
                                           % predicate and make the same processing
    N1 is N-1,                             % continue by processing the previous argument
    subst_args2(N1, Old, New, Expr, NewExpr). % continue with the rest of
                                           % arguments, tail of list
```

Substituting expressions **functor+arg**

Just code, comments removed

```
subst2(Old, New, Old, New) :- !.  
subst2(_, _, Old, Old) :-  
    atomic(Old), !.  
subst2(Old, New, Expr, NewExpr) :-  
    functor(Expr, F, N),  
    functor(NewExpr, F, N),  
    subst_args2(N, Old, New, Expr, NewExpr).  
  
subst_args2(0, _, _, _, _) :- !.  
subst_args2(N, Old, New, Expr, NewExpr) :-  
    arg(N, Expr, OldArg),  
    arg(N, NewExpr, NewArg),  
    subst2(Old, New, OldArg, NewArg),  
    N1 is N-1,  
    subst_args2(N1, Old, New, Expr, NewExpr).
```

Qs:

1. Order of processing the structure?
2. Where the execution ends?

[q] ?- subst2(a, x, f(a, b, g(a)), R).
R = f(x,b,g(x))

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #9

Side effects

Agenda

- **Built in predicates**
 - **Side Effects:** `assert` **and** `retract`
- Utilization on complex tasks
 - Towers of Hanoi

Side effects

- Allow for persistent structure
- Allow for wider scope visibility (modelling global variables; created in some predicate and visible elsewhere, without passing arguments)
- Allow for metaprogramming
- By using them, we exceed the pure Logic Programming (operations NOT undone on backtracking)

Side effects

- Built in predicates
- **assert/retract** for adding/extracting ONE clause of the predicate with the name provided as argument
- The argument (for both assert and retract) is a variable representing a dynamic predicate
 - `assert(X) / retract(X)`
- The variable *X* must be ALREADY instantiated for the predicate to be updated

Side effects – assert

- **assert** comes into 2 flavors:
 - **asserta** – adds in the beginning (on top) of the existing clauses for the predicate provided as argument
 - **assertz** / **assert** – adds at the end (at bottom) of the existing clauses for the predicate provided as argument
- It NEVER fails
- on backtracking, the item (clause) is NOT removed. However, **assert** does not re-execute on backtracking (NO new clause is added).

Side effects – retract

- removes the first item (clause of the predicate provided as argument) with whom it matches successfully
- if no match (the matching fails), retract fails. So, as opposed to assert, it can fail
- on backtracking, the item (clause removed) is NOT added back
 - On the contrary, another item is removed in case another successful match is possible, **or** retract fails

assert

- Assume we have some (3 in the ex.) clauses for a predicate $p(\dots)$ (arguments do not matter here).
- Here on the right, the clauses have numbers added to represent their index

$p(\dots) : -\dots$

$p(\dots) : -\dots$

$p(\dots) : -\dots$

$p_1(\dots) : -\dots$

$p_2(\dots) : -\dots$

$p_3(\dots) : -\dots$

asserta / assertz

- After the execution of `asserta (p (...))`, the predicate becomes:

$p_{\text{new}} (...) : - \dots$

$p_1 (...) : - \dots$

$p_2 (...) : - \dots$

$p_3 (...) : - \dots$

- After the execution of `assertz (p (...))`, the predicate becomes:

$p_1 (...) : - \dots$

$p_2 (...) : - \dots$

$p_3 (...) : - \dots$

$p_{\text{new}} (...) : - \dots$

Assert example

```
:-dynamic p/1.
```

No result is actually shown, it just results in a true.

```
p(1).
```

```
?- listing(p/1).
```

```
p(2).
```

```
p(1).
```

```
p(3).
```

```
p(2).
```

```
p(4).
```

```
p(3).
```

```
p(5).
```

```
p(4).
```

```
p(5).
```

```
q:- assertz(p(6)), fail.
```

```
?- q, listing(p/1).
```

```
q.
```

```
p(1).
```

```
p(2).
```

```
p(3).
```

```
p(4).
```

```
p(5).
```

```
p(6).
```

```
?- q.
```

error (No permission to modify static procedure 'p/1')

```
true.
```


retract

- Assume this scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- The execution of `retract(p(...))` would:
 - Start scanning the list of clauses of the predicate p
 - Stop at the first clause where arguments (not mentioned here) match
 - Remove the given clause and report success
 - If no successful matching, the retract fails

retract – success on the first clause

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract (p (...))` succeeds on matching with **first** clause of p , it leads to:

$p_2 (...) : -...$

$p_3 (...) : -...$

retract – success on the second clause

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract (p (...))` succeeds on matching with **second** clause of p , it leads to:

$p_1 (...) : -...$

$p_3 (...) : -...$

retract – success on the third clause

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract (p (...))` succeeds on matching with **third** clause of p , it leads to:

$p_1 (...) : -...$

$p_2 (...) : -...$

retract – fails

- Assume the same scenario:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

- If execution of `retract(p(...))` does NOT succeed trying to match p to ANY of the 3 clauses, the result would be:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

retract – on backtracking

- After the execution of retract, on backtracking, NO undo is performed (the removal stays). That clause is NEVER restored, unless an explicit request (assert) is made
- Moreover, on the attempt to REsatisfy the goal (due to backtracking), Prolog continues FROM THE PLACE it previously removed and tries to remove ANOTHER clause, and:
 - it does so if the match succeeds,
 - or else fails and the execution is resumed (backtracked) to the predicate on the left of retract

Computer Science

retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

$p_4 (...) : -...$

$p_5 (...) : -...$

- Assume the execution of `retract (p (...))` succeeds on matching with the **second** clause of predicate $p_2 (...)$, after execution the predicate contains:

$p_1 (...) : -...$

$p_3 (...) : -...$

$p_4 (...) : -...$

$p_5 (...) : -...$

retract – on backtracking - contd

- If now we backtrack again, and retract matches the 3rd clause (i.e. $p_3 (...)$ which is the second actually), the predicate looks:

$p_1 (...)$: -...

$p_4 (...)$: -...

$p_5 (...)$: -...

- If we backtrack again, and retract matches the 5th clause (i.e. $p_5 (...)$ which is the third actually) the predicate looks:

$p_1 (...)$: -...

$p_4 (...)$: -...

- A new backtrack now would fail, as we reached the end of the list of clauses.

Retract example

```
:-dynamic p/1.
```

No result is actually shown, it just results in a true.

```
p(1) .
```

```
p(2) .
```

```
p(3) .
```

```
p(4) .
```

```
p(5) .
```

```
?- listing(p/1) .
```

```
p(1).
```

```
p(2).
```

```
p(3).
```

```
p(4).
```

```
p(5).
```

```
q:- retract(p(_)), fail.
```

```
q.
```

```
?- q, listing(p/1) .
```

```
true.
```

```
?- q.
```

error (No permission to modify static procedure 'p/1')

```
true.
```

retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

$p_4 (...) : -...$

$p_5 (...) : -...$

- Assume the execution of `retract(p(_))` cannot unify with ANY of the 5 clauses, the execution fails and `p` remains unchanged:

$p_1 (...) : -...$

$p_2 (...) : -...$

$p_3 (...) : -...$

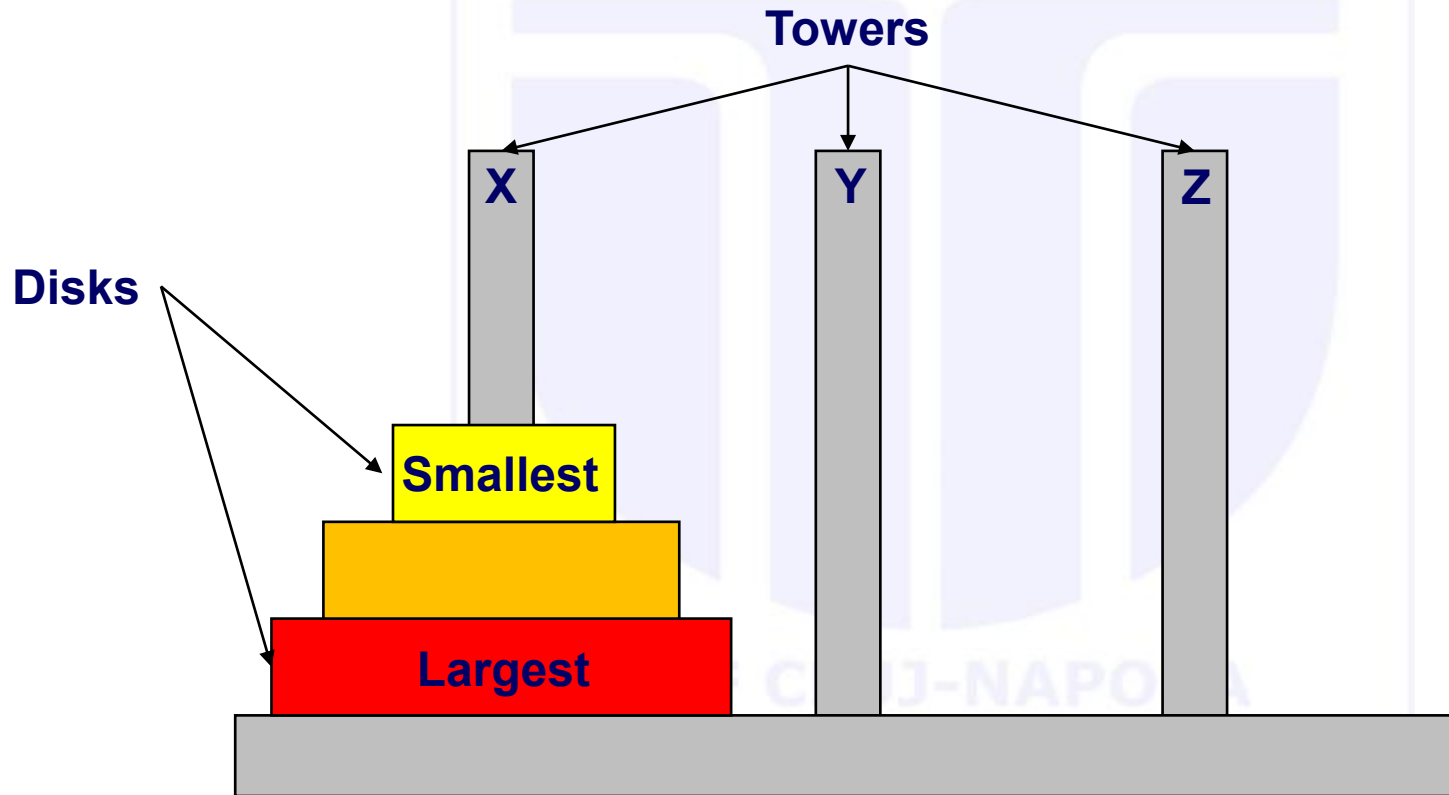
$p_4 (...) : -...$

$p_5 (...) : -...$

assert and retract

- Combining assert and retract
 - `asserta + retract => LIFO`
 - `assertz + retract => FIFO`

Towers of Hanoi – the problem

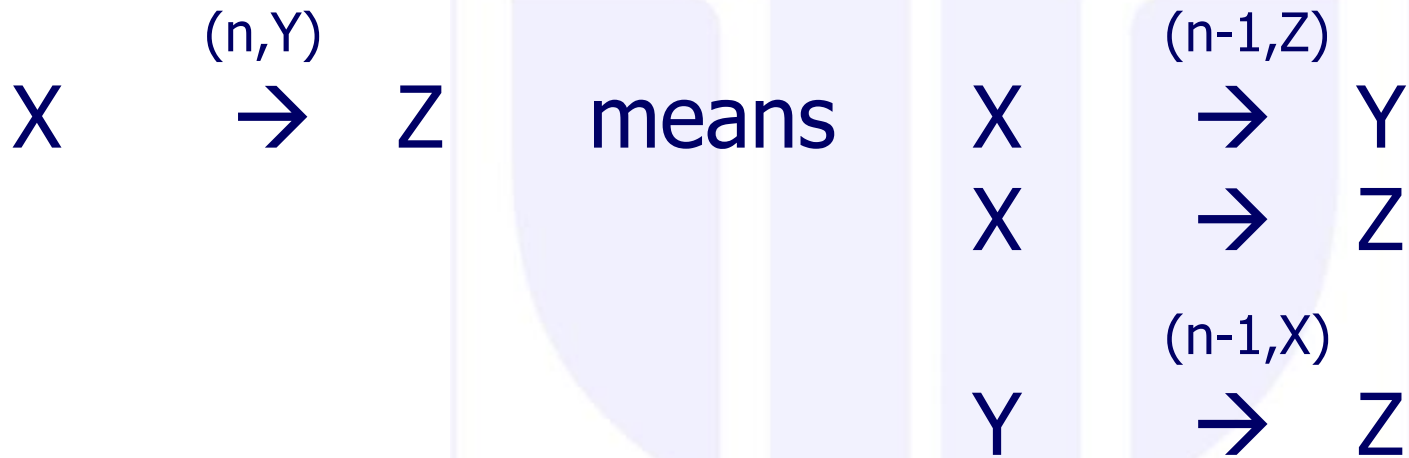


Towers of Hanoi - formulation

- Given n disks and 3 towers
- Move the disks from tower X to Z (same order – smallest to largest)
- At any time, the constraints are:
 - One disk at a time
 - Just the top disk is accessible
 - On any tower, any disk has the diameter smaller than the diameter of the disk on top of which it stays

Towers of Hanoi - rephrasing

Start with n disks on X and end with n disks on Z



- Means:

- Move n discs from X to Z using Y as intermediate by means of:
 - Move $(n-1)$ discs from X to Y using Z as intermediate
 - Move one single disc from X to Z
 - Move $(n-1)$ discs from Y to Z using X as intermediate

Towers of Hanoi - rephrasing

Start with n disks on X and end with n disks on Z

$$\begin{array}{ccc} X & \xrightarrow{(n,Y)} & Z \end{array} \quad \text{means} \quad \begin{array}{ccc} X & \xrightarrow{(n-1,Z)} & Y \\ X & \xrightarrow{\quad} & Z \\ Y & \xrightarrow{(n-1,X)} & Z \end{array}$$

BUT this is almost Prolog code:

```
move_discs(N,X,Y,Z):-
    N1 is N-1,
    move_discs(N1,X,Z,Y),
    move_disc(X,Z),
    move_discs(N1,Y,X,Z).
```

Towers of Hanoi - realization

- Disc representation (as stack of facts in the knowledge base):

`x(1) .`

`⋮`

`x(n-1) .`

`x(n) .` % n is NOT a variable, it is a value.

- How is it realized? % init_tower/2 (+Input, +List)

`init_tower(_,0):-!.`

`init_tower(T,N):-`

`D=..[T,N],` % T name of tower, as x in example; N size

`asserta(D),` % put on top

`N1 is N-1,`

`init_tower(T,N1) .`

Towers of Hanoi – moving one disk

- Move one disc from Input Pole to Output Pole:

```
move_disc(IP,OP):-
```

```
    DI=..[IP,Size],      % IP name of input pole, Size diam of disk
    retract(DI),!,       % extract from top
    DO=..[OP,Size],      % IP name of output pole, Size diam of disk
    asserta(DO).          % put on top
```

- If input has:

```
x(3).
x(4).      y(6).
x(5).      y(7).
```

- And call `move_disc(x,y)` . goes to:

```
    y(3).
x(4).      y(6).
x(5).      y(7).
```

Towers of Hanoi – complete code

```

move_discs(0,_,_,_):-!.
move_discs(N,X,Y,Z):-
    N1 is N-1,
    move_discs(N1,X,Z,Y),
    move_disc(X, Z),
    move_discs(N1,Y,X,Z).

move_disc(IP,OP):-
    DI=..[IP,Size],
    retract(DI),!,
    DO=..[OP,Size],
    asserta(DO).

init_tower(_,0):-!.
init_tower(T,N):-
    D=..[T,N],
    asserta(D),
    N1 is N-1,
    init_tower(T,N1).

:- dynamic(input/1).
:- dynamic(int/1).
:- dynamic(out/1).

% INITIAL call
hanoi(N):-
    init_tower(in,N),
    move_discs(N,in,int,out).

[q] ?- retractall(out(_)),
hanoi(3), listing(in/1),
listing(out/1).

```

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #10

Graphs

Computer Science

Agenda

- **Graphs**
 - Representations
 - Search for a path
 - One way
 - V1
 - V2
 - V3 – nonmonotonic reasoning!
 - Best way
 - All ways
 - Restricted way

Graphs - Representations

- **Traditional representations:**
 - Adjacency matrix – not efficient here
 - Adjacency lists – 2 ways of doing this:
 - **Neighbor list:**
 - Knowledge base with facts of
 - (node, list of neighbors)
 - **Adjacency list**
 - Knowledge base with facts of edge pairs:
 - (node, node)

Graphs - Representations

- Neighbor list – Knowledge base with facts of nodes, list of neighbors

```
% neighb/2 (+Vertex, +NeighborList).  
neighb(a, [b, c]).
```

- Directed graphs? What changes?

- Isolated nodes?

```
neighb(z, []).
```

Graphs - Representations

- Adjacency list - Knowledge base with edge pairs

```
% edge/2 (+Vertex1, +Vertex2).  
edge(a,b).  
edge(a,c).
```

- UNdirected graphs? Add a predicate to ask for searching both ways.

```
% is_edge/2 (+Vertex1, +Vertex2).  
is_edge(X,Y):-  
    edge(X,Y); % check one way  
    edge(Y,X). % and the other way
```

- Isolated nodes?

```
edge(z,nil). % not a built-in way; just assume an edge to some dummy vertex
```

Search for a path in a graph

- Finding the way out from the labyrinth
- Labyrinth =
 - a set of rooms
 - With doors linking them
 - Unique entrance
 - One objective = way out = reach a given room
 - You **know** you reach it just **AFTER** you entered the room! (that is, you cannot define the problem in terms of "go to room x " as you don't know what x would be beforehand. You have this ability just after you reach there).

Computer Science

Search for a path in a graph

Labyrinth representation

- Is a graph
 - Rooms = vertices
 - Doors = edges
- Is it directed? Why/why not?
 - Graph undirected => add necessary predicate
- Enter the labyrinth from room *a*.

```
is_door(a,b).  
is_door(b,c).  
is_door(b,e).  
is_door(c,d).  
is_door(d,e).  
is_door(e,f).  
is_door(e,g).
```

```
is_objective(g).
```

```
is_pass(X,Y):-  
    is_door(X,Y);  
    is_door(Y,X).
```

Search for a path in a graph

Labyrinth way out

- Search the way out = check every possible root. When deadlock, backtrack. Prolog advantage: backtrack is there for free.

```
% path /3 (+Source, +Target, -Path).
```

```
path(X, Y, Path) :-  
    try(X, Y, [X], Path),           % try a path from X to Y with the PartialPath  
                                     % containing just the starting vertex at first  
    is_objective(Y), !.             % why not start with this?
```

Call it with:

```
?- path(a, X, Path).                % is X safe here? Not Y?
```

Search for a path in a graph

Labyrinth way out – main predicate (v1)

```
% try1 /4 (+Source, +Target, +PartialPath, -FinalPath)
try1(Y,Y,FPath,FPath).    % at every step, stop and check if over
try1(X,Y,PPath,FPath):-   % if not over (how do we know is not over here?)
    is_pass(X,Z),          % find next step to Z
    not(member(Z,PPath)),  % verify if unchecked door
    try1(Z,Y,[Z|PPath],FPath). % make the step
```

- Order of clauses? Why?
- Should it be `is_pass(X,Z)`? Why not `is_door(X,Y)`?
- Pattern composition `[Z|PPath]` in the recursive call?
 - Is it correct? Shouldn't it be in the head of the rule? Why?
- Make the analysis of the calling tree.
 - When does try stop? When does try eventually close?
- What answer would we get to the initial query? Discussion on `Path`.

```
?- path(a,X,Path).
```

Search for a path in a graph

Labyrinth way out – main predicate (v2)

```
% try2/4 (+Source, +Target, +PartialPath, -FinalPath)
try2(Y,Y,_,[Y]).
try2(X,Y,PPath,[X|FPath]):-
    is_pass(X,Z),
    not(member(Z,PPath)),
    try2(Z,Y,[Z|PPath],FPath).
```

- Initial call?
- Why do we need both `PPath` and `FPath`? Do we really need both?
- Just use the 4th arg?
 - With `not(member(Z,PPath))`?
- Is it possible? What do `PPath` and `Path` contain? Make the difference
 - Understand and NEVER make confusions. Learn on the meaning of pattern composition on:
 - The head of the rule
 - Recursive call

Search for a path in a graph

Labyrinth way out – main predicate (v3)

```
% try /3 (+Source, +Target, -Path).  
try3(Y,Y,[Y]).  
try3(X,Y,[X|Path]):-  
    is_pass(X,Z),  
    accept(Z),                % can Z be part of the PartialPath  
    try3(Z,Y,Path).  
  
% accept /1 (+Vertex).  
accept(X):-  
    seen(X),!,  
    fail.  
accept(X):-  
    assert(seen(X)).  
accept(X):-  
    retract(seen(X)),!,  
    fail.
```

Search for a path in a graph

Labyrinth way out – main predicate (v3)

- Is the predicate which states if and WHEN a vertex makes part of the solution
- Reasoning is nonmonotonic (part of default logic)
 - During the reasoning,
 - If an assumption does NOT already take part of the reasoning
 - And does NOT contradict ANY other assumption
 - Is added to the knowledge base
 - If at a latter point
 - If the reasoning cannot be closed (completed), chance is made to attempt some reasoning WITHOUT that piece of knowledge
 - Therefore, quantity of knowledge is not monotonic

```
accept(X) :-  
    seen(X) , ! ,      % is the contradiction! The ONLY contradiction is that the vertex is  
    fail.              % ALREADY in the solution. If there, don't loop; fail to backtrack!  
  
accept(X) :-  
    assert(seen(X)) .   % no contradiction, add it in the solution  
  
accept(X) :-  
    retract(seen(X)) , ! , % cannot conclude with x in solution, remove and  
    fail.                % backtrack to try WITHOUT it!
```

Labyrinth way out

v3 / v2 comparative analysis

```
% Code v3
accept(X):-
    seen(X),!,
    fail.

accept(X):-
    assert(seen(X)).

accept(X):-
    retract(seen(X)),!,
    fail.

% v3 comments
% although x was in the solution
% at some point since there is no final way
% decide to remove it from path

% comments v2
% if member(Z,Thread) succeeds, then
% not(member(Z,Thread)) fails and execution in BOTH versions
% backtrack to a different neighbor of the current vertex

% not(member(Z,Thread)) succeeds, hence
% Z could be added to the current attempted solution

% is there any point in v2 where we do this?
% if so, where?
% if not, are solutions similar?
```

Graphs – Find the Best way

```
best_path(X,Y,_):-
    assert(best([],1000)), % not a wise init; better sum of all weights
    a_path(X,Y,[X],1).
best_path(_,_,Thread):-
    retract(best(Thread,_)).

a_path(Y,Y,Path,PathLen):-
    is_objective(Y), !,
    retract(best(_,_)), !,
    asserta(best(Path,PathLen)),
    fail. % cut above. So, where does backtracking fail? Mistake?
a_path(X,Y,Path,PathLen):-
    best(_,BestLen),
    PathLen1 is PathLen +1,
    PathLen1 < BestLen,
    is_pass(X,Z), % nondeterministic call. Why nondeterministic?
    not(member(Z,Path)),
    a_path(Z,Y,[Z|Path],PathLen1).
```


Graphs – Find all paths

- Assume neighbor representation:

```
% a_path /3 (+Source, +Target, -Path).
```

```
a_path(Y,Y,[Y]).
```

```
a_path(X,Y,[X|Path]):-
```

```
    neighb1(X,L),
```

```
    all_paths(L,Y,Path).
```

```
% all_paths /3 (+SourcesList, +Target, -Path).
```

```
all_paths([X|_],Y,Path):-
```

```
    a_path(X,Y,Path).
```

```
all_paths([_|Rest],Y,Path):-
```

```
    all_paths(Rest,Y,Path).
```

```
% Call it with:
```

```
% ?- all_paths([a], X, Path).
```

- Q: How are loops avoided?
- How/when does this work?

Graphs – Find restricted path

- Assume again the edge representation.

```
% restricted_path /4 (+Source, +Target, +RestrictionsList, -Path)
restricted_path(X,Y,Restrictions,Path):-
    path_obj(X,Y,Path),
    restrict(Restrictions,Path). % how else could we do?

path_obj(X,Y,Path):-
    nonvar(X), % meaning? Why needed?
    try2(X,Y,[X],Path), % any try from v1, v2, v3
    is_objective(Y).

restrict([HR|TR],[HR|T]):- !, % what is this cut cutting?
    restrict(TR,T).
restrict([HR|TR],[_|T]):-
    restrict([HR|TR],T).
restrict([],_).
```

Back to graphs

The issue with this implementation

```
% path1 /3 (+Source, +Target, -Path)
path1(X, Y, Path):- path1(X, Y, [X], Path).

path1(Y, Y, _, []).
path1(X, Y, PPath, [Z|FPath]):-
    edge(X, Z),
    not(member(Z, PPath)),
    path1(Z, Y, [Z|PPath], FPath).
```

- What is the issue with this implementation from a software development perspective?
- The graph is represented as a set of facts `edge/2`, if we want to run this on another graph, we have to change the code

```
?- path1(a, g, Path).
```

Back to graphs

Use = ..

```
% path1 /4 (+Graph, +Source, +Target, -Path)
path1(Graph, X, Y, Path):-
    path1(Graph, X, Y, [X], Path).

path1(_, Y, Y, _, []).
path1(G, X, Y, PPath, [Z|FPath]):-
    Edge=..[G, X, Z], % G= edge from query,
                      % Edge = G(X, Z) = edge(X, Z)
    call(Edge),
    not(member(Z, PPath)),
    path1(G, Z, Y, [Z|PPath], FPath).

?- path1(edge, a, g, Path).
```

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #11

Graph traversal

Agenda

- **Built-in predicates** (and relationship to graphs)
 - *findall* and more
- **Constructs and relationship with graphs**
 - For all is true
 - Application – same graph
- **Graph Traversal**
 - DFS
 - BFS

Built-in predicates

their utility in graph searching

- `findall` – selects all items (and only those items) with a certain property from a knowledge-base
- You must know the predicate to use it appropriately
 - Know HOW it is implemented
 - to utilize the strategy in graph problems

findall

- (example from Clocksin and Mellish)
- Suppose we have a knowledge base of some drinks lovers:
`% likes/2 (+Person, +Drink).`

```
likes(bill, wine).  
likes(dick, beer).  
likes(harry, beer).  
likes(john, beer).  
likes(peter, wine).  
likes(tom, beer).
```

```
[q] ?- findall(X, likes(X,beer), L).  
true, L=[dick,harry,john,tom].
```


findall – implementation

```
% findall/3 (+CollectedVar, +CollectorPred, -CollectorVar).
```

```
findall(X,P,_):-
    P,
    asserta(found(X)),
    fail.
findall(_,_,L):-
    collect_found([],L).

collect_found(P, L):-
    retract(found(X)), !,
    collect_found([X|P],L).
collect_found(L,L).
```

% uses an auxiliary knowledge base (**akb**) - found
 % = call(P), P is a predicate;
 % its exe instantiates some argument X.
 % puts P's instantiated argument on top of **akb**
 % request for backtrack to P.
 % starts collecting from the **akb**,
 % initializes the partial result to empty list.
 % extracts from top of the **akb**
 % succeeds if a genuine data; fails when no more
 % adds it in front and go recurse

```
likes(bill, wine).
likes(dick, beer).
likes(harry, beer).
likes(john, beer).
likes(peter, wine).
likes(tom, beer).
```

findall – implementation

```
% findall/3 (+CollectedVar, +CollectorPred, -CollectorVar).
```

```
findall(X,P,_):-  
    P,  
    asserta(found(X)),  
    fail.
```

```
findall(_,_,L):-  
    collect([],L).
```

```
collect_found(P,L):-  
    retract(found(X)), !,  
    collect_found([X|P],L).  
collect_found(L,L).
```

```
% What type of recursion is used in collect?
```

findall – execution

```
findall(X,P,_):-  
    P,  
    asserta(found(X)),  
    fail.  
findall(_,_ ,L):-  
    collect_found([],L).  
  
collect_found(P,L):-  
    retract(found(X)), !,  
    collect_found([X|P],L).  
collect_found(L,L).
```

% initial kb

```
likes(bill, wine).  
likes(dick, beer).  
likes(harry, beer).  
likes(john, beer).  
likes(peter, wine).  
likes(tom, beer).
```

% akb

```
found(tom).  
found(john).  
found(harry).  
found(dick).
```

%created bottom-up↑ collected top-down↓

```
[q] ?- findall(X, likes(X,beer), L).  
true, L=[dick,harry,john,tom].
```

findall

- General call:

?- `findall(X,P,L)`

- where X is a variable
- P a predicate and X occurs in P
- L would collect all X 's so that P 's execution succeeds on X
- Creates the list L of X 's that satisfy P , in the order of occurrence in the original knowledge base

findall – related

- Associate predicates:

`setof(X, P, S)`

- Creates the set of terms (standard order, without duplicates; represented also as list) of `X`'s that satisfy `P`. If none, fails.
- **Differences to `findall`:**
 - Order: standard vs as found in the knowledge base
 - Duplicates: no vs found in the knowledge base
 - No term: fails vs empty list

`bagof(X, P, S)`

- Same as `setof` BUT list NOT ordered + may have duplicates
- So same as `findall` but fails if no term matches

```
findall(X, P, L) :- bagof(X, P, L), !.  
findall(_, _, []).
```

For all is true technique

- Construct to check if `for_all Predicate1`
`is_true Predicate2`
- Aims to identify/verify whether in all cases when P1 is true, P2 is true as well. Thus, checks $P1 \Rightarrow P2$.
- Template looks like:

```
for_all_is_true(X,Y):-
    X,                                % calls X, a predicate,
                                     %   whose execution may instantiate hidden args
    not(Y) , !,                       % calls Y in the same context. If succeeds, its negation fails,
                                     %   and backtracks to X who is executed again, with a
                                     %   new instance of args; generates a new context
                                     %   the first context of X where Y fails, succeeds, and !
                                     %   is irreversible crossed and the predicate fails
    fail.                             % overall.

                                     % if we reached here, in all contexts when X holds true
for_all_is_true(_,_) % succeeds as well, therefore,  $X \Rightarrow Y$ 
```

For all is true in graphs context

- Given graphs G1 and G2 with neighbor and edge representations respectively, do they represent the same graph?

G1

`neighbor(Node, List_of_Neighbors)`

G2

`edge(Node1, Node2)`

- G1 and G2 are the same means $G1 \Leftrightarrow G2$ which is $(G1 \Rightarrow G2) + (G2 \Rightarrow G1)$

(G1 $\Rightarrow$ G2)

`for_all edges_in_G1
is_true edge_in_G2`

(G2 $\Rightarrow$ G1)

`for_all edges_in_G2
s_true edge_in_G1`

I

- Define the edges in the 2 representations:

An edge in G1: (defines the nondeterministic check for all edges)

```
is_edge_1(X, Y) :-
    neighbor(X, L),           % to find all edges in G1, call nondeterminist(X,Y,free)
    member(Y, L).             % finds first pair first (and next on backtrack), instantiates both X and L.
                                % nondeterm call on member; instantiate Y, one at a time, eventually all.
```

An edge in G2:

```
is_edge_2(X, Y) :-
    edge(X, Y);
    edge(Y, X).
```

Same graph?

- $(G1 \Rightarrow G2)$ $(G2 \Rightarrow G1)$
`for_all edges_in_G1` `for_all edges_in_G2`
`is_true edge_in_G2` `is_true edge_in_G1`
- Denote the `for_all_is_true` as `eq1` and `eq2` depending on the implication $(G1 \Rightarrow G2)$ respectively $(G2 \Rightarrow G1)$ we verify

`eq1:-`

```
is_edge_1(X,Y),
not(is_edge_2(X,Y)),!,
fail.
```

% for each edge in the first representation

% there is one edge in the other graph

% otherwise, we reach here, hence, fail.

`eq1.`

% if we get here, $G1 \Rightarrow G2$ shown

`eq2:-`

```
is_edge_2(X,Y),
not(is_edge_1(X,Y)),!,
fail.
```

`edge(a,b).`

`edge(b,a).`

`neighbor(a,[b]).`

`neighbor(b,[a]).`

`eq2.`

`same_graph:- eq1, eq2.`

`[q] ?- same_graph.`

Change representation of a graph

- Given graph **G1** a weighted, edge representation and we need to get **G2** with neighbor representation

G1

edge(a,b,15).

edge2neighb:-

```
is_edge(X,_,_),
not(neighbor(X,L)),
findall(Z,convert(X,Z),L),
assertz(neighbor(X,L)),
fail.
```

edge2neighb.

convert(X,Z):-

```
is_edge(X,Y,W),
Z=p(Y,W).
```

- Note:**

findall(Z,succ(X,Z),L) same as findall(p(Y,W),is_edge(X,Y,W),L).

- Given G2, generate G1. Homework.

G2

neighbor(a,[p(b,15),...]).

Depth first search

```
collect([X|R]) :-  
    retract(node(X)), !,  
    collect(R).  
collect([]).
```

- Assume undirected, unweighted graph, edge representation
- 2 stage process:
 1. 1st clause: make the trail placing in a queue the vertices in order of visitation (being in the additional knowledge base `node` - hence a dynamic predicate - is as if we have a grey vertex).
 2. 2nd clause: when done, collect them.

```
dfs(X, L) :-
```

```
    assertz(node(X)),           % place start/current node as visited  
    is_edge(X, Y),             % take first/next neighbor of current node X  
    not(node(Y)),              % if already visited, backtrack to take another;  
    dfs(Y, L).                 % if not, continue from Y
```

```
dfs(_, L) :-
```

```
    collect(L).                 % similar to collect in findall, but on node akb.
```

- It is covering just the connected component of the starting vertex (is similar to `dfs_visit` in Cormen).
 - Need a wrapper to re-run it from all connected components (all remaining white nodes – color NOT implemented here).

```
collect([X|R]):-
    retract(node(X)),!,
    collect(R).
collect([]).
```

Breadth first search

- Assume undirected, unweighted graph, edge representation
- Queue made by the akb named `node` (hence dynamic predicate)

```
bfs(X,_):-
    assertz(node(X)),           % add at the end of the Q the current node
    node(Y),                   % reads from front of Q, gets Y instantiated (initially X=ONLY one in Q)
    is_edge(Y,Z),              % take first/next neighbor of current Y (gets Z instantiated);
                                % if none, backtrack and take next from Q
    not(node(Z)),              % if Z already visited, backtrack to take another;
    assertz(node(Z)),          % if not, put Z in Q and
    fail.                      % backtrack anyway to another neighbor of Y

bfs(_,L):-
    collect(L).                % similar to collect in findall, but on node in akb.
```

- There is an issue with this implementation for some languages/dialects (due to an optimization strategy). Oral explanation.

BFS – queues explicit

- BFS with 3 arguments representing:
 - Queue (Q) = list of candidates = list of items we reached, yet not finalized to process = grey nodes. Implemented as incomplete lists. Why? Try to find out. Oral discussion!
 - Expanded list (E_{xp}) = list of items reached and processed = black nodes. Implemented as regular (complete) lists.
 - Result (R) = nodes in order of bfs processing. Copy of Expanded list when processing ends.

```
% bfs /3 (+Queue, +ExpansionList, -Result).
```

```
[q] ?- bfs([a|_], [], R).
```

```
bfs(Q,R,R) :- var(Q), !.
```

% when Q is empty, end exe, the Exp becomes R

```
bfs([X|Q],Exp,R) :-
```

% else, take first from Q

```
    expand(X,Q,Exp),
```

% and expand (put all white neighbors in Q) and

```
    bfs(Q,[X|Exp],R).
```

% continue by moving it in expanded

% = make it black

BFS – queues explicit

- The expansion step (2-stage process) – decides which neighbors to add in Q
- dynamic predicate `desc`; potential neighbors stored in auxiliary knowledge base (akb)

```

expand(X,_,Exp):-
    is_edge(X,Y),                % nondeterministically take z, first neighbor of x
                                % (eventually all of them)
    not(member(Y,Exp)),          % should NOT be already processed (not a black node)
    assertz(node(Y)),            % potentially add it in Q
    fail.                        % backtrack to evaluate another neighbor of x
expand(_,Q,_):-
    collect(Q).                  % we get here, end

collect(Q):-
    retract(node(X)),!,          % take x and if not under processing (not a grey one)
                                % as long as akb not empty
    insert_IL(X,Q),              % add in Q
    collect(Q).                  % continue. How is possible with the SAME argument?
collect(Q).                      % end when akb empty

```

BFS – complete code

```
bfs(Q,R,R):- var(Q),!.
```

```
bfs([X|Q],Exp,R):-  
    expand(X,Q,Exp),  
    bfs(Q,[X|Exp],R).
```

```
expand(X,_,Exp):-  
    is_edge(X,Y),  
    not(member(Y, Exp)),  
    assertz(node(Y)),  
    fail.
```

```
expand(_,Q,_):-  
    collect(Q).
```

```
collect(Q):-
```

```
    retract(node(X)),!,  
    insert_IL(X,Q),  
    collect(Q).
```

```
collect(Q).
```

```
insert_IL(X,[X|_]):-!.
```

```
insert_IL(X,[_|L]):-  
    insert_IL(X,L).
```

```
[q] ?- bfs([a|_],[],Rez).
```

BFS – execution

[q] ?- bfs([a|_], [], Rez). % for the first graph – search for a path

Step1

Exp: []
Q: [a|_]

```
bfs(Q,R,R):- var(Q),!.
```

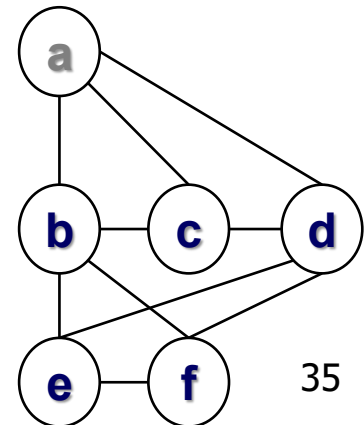
```
bfs([X|Q],Exp,R):- % [a|_]
    expand(X,Q,Exp), % expand(a, ...
    bfs(Q,[X|Exp],R).
```

```
expand(X,_,Exp):-
    is_edge(X,Y),
    not(member(Y, Exp)),
    assertz(node(Y)),
    fail.
```

```
expand(_,Q,_):-
```

```
30-Apr-26 collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

[q] ?- bf_search([a|_], [], Rez).

% for the first graph – search for a path

	Step1	Step2
Exp:	[]	[a]
Q:	[a _]	[b,c,d _]

```
bfsearch(Q,R,R):- var(Q),!.
```

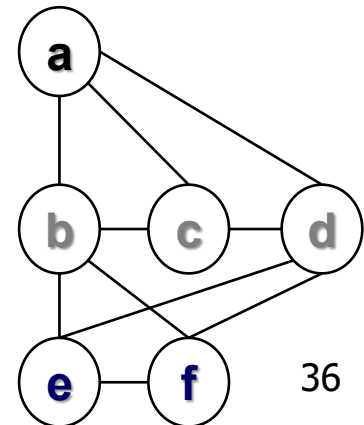
```
bfsearch([X|Q],Exp,R):- % [b| ... ]
    expand(X,Q,Exp), % expand(b, ...
    bfsearch(Q,[X|Exp],R).
```

```
expand(X,_,Exp):-
    is_edge(X,Y),
    not(member(Y, Exp)),
    assertz(node(Y)),
    fail.
```

```
expand(_,Q,_):-
```

```
30-Apr-26 collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

```
[q] ?- bf_search([a|_], [], Rez).
```

% for the first graph – search for a path

	Step1	Step2	Step3
Exp:	[]	[a]	[b,a]
Q:	[a _]	[b,c,d _]	[c,d,f,e _]

% Why is f first?

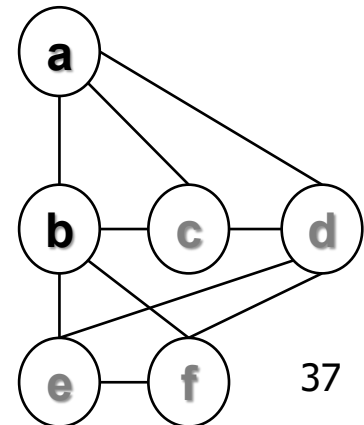
```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
    expand(X,Q,Exp),
    bfs(Q,[X|Exp],R).
```

```
expand(X,_,Exp):-
    is_edge(X,Y),
    not(member(Y, Exp)),
    assertz(node(Y)),
    fail.
```

```
expand(_,Q,_):-
```

```
30-Apr-26 collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

[q] ?- bf_search([a|_], [], Rez).

% for the first graph – search for a path

	Step1	Step2	Step3	Step4
Exp:	[]	[a]	[b,a]	[c,b,a]
Q:	[a _]	[b,c,d _]	[c,d,f,e _]	[d,f,e _]

```
bfs(Q,R,R):- var(Q),!.
```

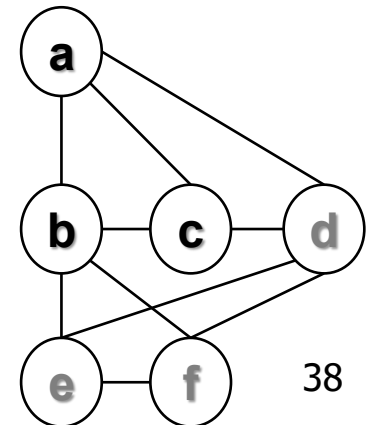
```
bfs([X|Q],Exp,R):-
    expand(X,Q,Exp),
    bfs(Q,[X|Exp],R).
```

```
expand(X,_,Exp):-
    is_edge(X,Y),
    not(member(Y, Exp)),
    assertz(node(Y)),
    fail.
```

```
expand(_,Q,_):-
```

```
30-Apr-26 collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

[q] ?- bf_search([a|_], [], Rez). % for the first graph – search for a path

	Step1	Step2	Step3	Step4	Step5
Exp:	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]
Q:	[a _]	[b,c,d _]	[c,d,f,e _]	[d,f,e _]	[f,e _]

```

bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
    expand(X,Q,Exp),
    bfs(Q,[X|Exp],R).

```

```

expand(X,_,Exp):-
    is_edge(X,Y),
    not(member(Y, Exp)),
    assertz(node(Y)),
    fail.

```

```

expand(_,Q,_):-

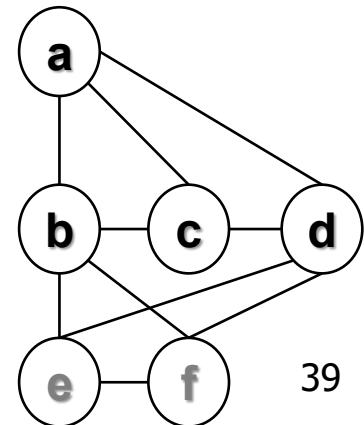
```

30-Apr-26 collect(Q).

```

edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).

```



BFS – execution

[q] ?- bf_search([a|_], [], Rez). % for the first graph – search for a path

	Step1	Step2	Step3	Step4	Step5	Step6	Step7
Exp :	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]	[f,d,c,b,a]	[e,f,d,c,b,a] DONE!
Q:	[a _]	[b,c,d _]	[c,d,f,e _]	[d,f,e _]	[f,e _]	[e _]	_

```

bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
    expand(X,Q,Exp),
    bfs(Q,[X|Exp],R).

```

```

expand(X,_,Exp):-
    is_edge(X,Y),
    not(member(Y, Exp)),
    assertz(node(Y)),
    fail.

```

```

expand(_,Q,_):-

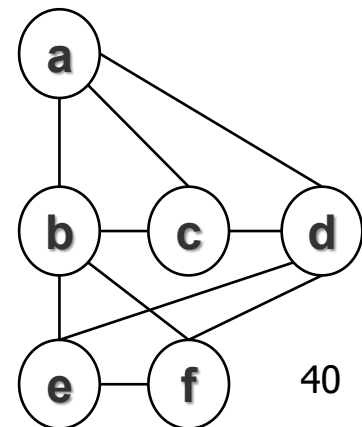
```

30-Apr-26 collect(Q).

```

edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).

```



Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #12

Graph isomorphism

Agenda

- **Graphs isomorphism**
 - with metaprogramming
 - various implementations (compare & contrast analysis)

Graphs isomorphism problem statement

- Let us:
 - $G1=(V1,E1)$
 - $G2=(V2,E2)$
 - $G1$ iso $G2$ iff
 - i) $|V1|=|V2|$
 - ii) $\exists \varphi : V1 \rightarrow V2$ a bijection so that
 - iii) $\forall x, y \in V1 [(x, y) \in E1 \Leftrightarrow (\varphi(x), \varphi(y)) \in E2]$
- In simpler words:
 - Graphs are essentially "the same" in structure, differing only by the names or arrangement of their vertices.
 - φ matches up vertices one-to-one while preserving adjacency
- Ex: Are the 2 graphs isomorphs?

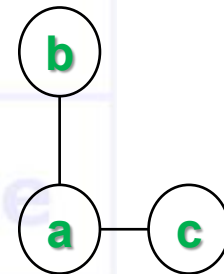
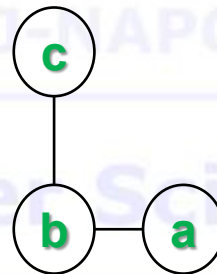
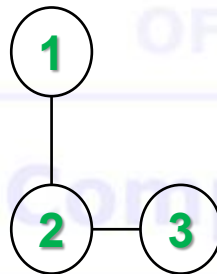
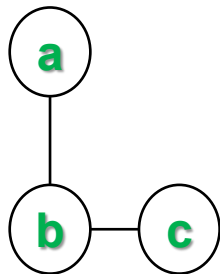
```
graph1([
  n(a,[b,c,d]),
  n(b,[a,c]),
  n(c,[a,b,d]),
  n(d,[a,c])
]).
```

```
graph2([
  n(1,[2,4]),
  n(2,[1,3,4]),
  n(3,[2,4]),
  n(4,[1,2,3])
]).
```

Graphs isomorphism

Simple example

- Consider these two graphs:
 - Graph A** has vertices labelled $\{a,b,c\}$ with edges $(a,b),(b,c)$.
 - Graph B** has vertices labelled $\{1,2,3\}$ with edges $(1,2),(2,3)$.



Graphs isomorphism approach (v1)

```
?- graph1(G1), graph2(G2), iso_graph1(G1,G2).

iso_graph1(L1,L2):- eq_perm(L1,L2,eq_neighb).

eq_perm([H1|T1],L2,EQ):- % is the perm predicate with an equivalence
    delete(H2,L2,T2), % nondeterministic behavior. Implemented how?
    P=..[EQ,H1,H2], % which is created here (metapredicate)
    call(P), % and called here to execute
    eq_perm(T1,T2,EQ).

eq_perm([],[],_). % succeeds iff sets have the same cardinality
% an immediate improvement would check cardinalities beforehand

eq_neighb(n(N1,L1),n(N2,L2)):- % 2 neighb pairs are equivalent
    eq_node(N1,N2), % if the nodes are equivalent
    eq_perm(L1,L2,eq_node). % and the neighb lists are equivalent

delete(H,[H|T],T). % what if we add a cut here?
delete(X,[H|T],[H|R]):-
    delete(X,T,R). % what if we add the [] clause?
```

Nodes equivalence

(implements φ function)

V1 – default logic with side effects

- p a dynamic predicate, used for the nonmonotonic reasoning
- implements φ function (to form p as akb)

```

eq_node(N1,N2):-           % if nodes N1 and N2 already form a pair in the akb
    p(N1,N2).              % the evaluation continues
eq_node(N1,_):-            % if N1 forms a pair with some OTHER node
    p(N1,_),!,             % we get an inconsistency, so should NOT allow
    fail.                  % (N1,N2) form a pair, so, fail to backtrack
eq_node(_,N2):-            % symmetric on N2
    p(_,N2),!,             % not an injective function
    fail.                  % How/where bijection is insured?
eq_node(N1,N2):-           % if you reach here, there is no inconsistency
    asserta(p(N1,N2)).      % hence pair N1, N2 is legitimate.
eq_node(N1,N2):-           % if at a later moment,
    retract(p(N1,N2)),!,    % although pair N1, N2 is legitimate,
                             % the reasoning cannot conclude,
                             % remove it from the akb, and
                             % fail to backtrack and
                             % resume execution WITHOUT the pair in the akb
    fail.

```

Graphs isomorphism approach (v1)

```
?- graph1(G1), graph2(G2), iso_graph1(G1,G2).
```

```
iso_graph1(L1,L2):- eq_perm(L1,L2,eq_neighb).
```

```
eq_perm([H1|T1],L2,EQ):-  
    delete(H2,L2,T2),  
    P=..[EQ,H1,H2],  
    call(P),  
    eq_perm(T1,T2,EQ).
```

```
eq_perm([],[],_).
```

```
eq_neighb(n(N1,L1),n(N2,L2)):-  
    eq_node(N1,N2),  
    eq_perm(L1,L2,eq_node).
```

```
delete(H,[H|T],T).  
delete(X,[H|T],[H|R]):- delete(X,T,R).
```

```
eq_node(N1,N2):- p(N1,N2).  
eq_node(N1,_):- p(N1,_), !, fail.  
eq_node(_,N2):- p(_,N2), !, fail.  
eq_node(N1,N2):- asserta(p(N1,N2)).  
eq_node(N1,N2):- retract(p(N1,N2)), !, fail.
```

Nodes equivalence

V2 – with arguments (lists)

```
?- graph1(G1), graph2(G2), iso_graph2(G1,G2).
% instead of the akb p in v1, here we store the pairs in the arg list
% arg 4 = partial list, empty initially (arg to model the akb)
% arg 5 = final list; a copy of the partial list at the end of the execution
% (arg with the status of the akb at the end; the bijection)
iso_graph2(L1,L2):- eq_perm(L1,L2,eq_neighb,[],Lout).

eq_perm([H1|T1],L2,EQ,LI,LO):- % same as in v1
    delete(H2,L2,T2),
    P=..[EQ,H1,H2,LI,Lint], % same but with args
    call(P),
    eq_perm(T1,T2,EQ,Lint,LO).
eq_perm([],[],_,L,L).

eq_neighb(n(N1,L1),n(N2,L2),LI,LO):-
    eq_node(N1,N2,LI,Lint),
    eq_perm(L1,L2,eq_node,Lint,LO).
```

Nodes equivalence

(implements φ function)

V2 – with arguments (lists)

- p is a pair of nodes in the list argument
- models φ function

```
eq_node (N1, N2, LI, LI) :-  
    member (p (N1, N2), LI), !.  
eq_node (N1, N2, LI, [p (N1, N2) | LI]) :-  
    not (member (p (N1, _), LI)),  
    not (member (p (_, N2), LI)).
```

- These 2 clauses do the same as those 5 in v1

Nodes equivalence

(φ function v2 VS v1)

```
eq_node (N1, N2, LI, LI) :-  
    member (p (N1, N2), LI), !.  
  
eq_node (N1, N2, LI, [p (N1, N2) | LI]) :-  
  
    not (member (p (N1, _), LI)),  
  
    not (member (p (_, N2), LI)).
```

```
eq_node (N1, N2) :-  
    p (N1, N2) .  
eq_node (N1, _) :-  
    p (N1, _), !,  
    fail.  
eq_node (_, N2) :-  
    p (_, N2), !,  
    fail.  
eq_node (N1, N2) :-  
    asserta (p (N1, N2)) .  
eq_node (N1, N2) :-  
    retract (p (N1, N2)), !,  
    fail.
```

Nodes equivalence

(implements φ function)

V3 – with arguments (incomplete lists)

- Same as v2 but lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the “not found” case
- Just the equivalence changes
 - (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node(N1,N2,[p(N1,N2)|_]) :- !.
```

```
eq_node(N1,_,[p(N1,_)|_]) :- !,  
    fail.
```

```
eq_node(_,N2,[p(_,N2)|_]) :- !,  
    fail.
```

```
eq_node(N1,N2,[_|T]) :-  
    eq_node(N1,N2,T).
```

- Compare v3 with v1. Why is clause 1 (from v1) missing (in v3)?
- Compare v3 with v2.

Nodes equivalence

(φ function v3 VS v1)

```
eq_node (N1, N2, [p(N1, N2) | _]) :- !.  
eq_node (N1, _, [p(N1, _) | _]) :-  
    !,  
    fail.  
eq_node (_, N2, [p(_, N2) | _]) :-  
    !,  
    fail.  
eq_node (N1, N2, [_ | T]) :-  
    eq_node (N1, N2, T).
```

```
eq_node (N1, N2) :-  
    p(N1, N2).  
eq_node (N1, _) :-  
    p(N1, _), !,  
    fail.  
eq_node (_, N2) :-  
    p(_, N2), !,  
    fail.  
eq_node (N1, N2) :-  
    asserta(p(N1, N2)).  
eq_node (N1, N2) :-  
    retract(p(N1, N2)), !,  
    fail.
```


Nodes equivalence

(φ function v3 VS v2)

```
eq_node(N1,N2,[p(N1,N2)|_]) :- !.  
eq_node(N1,_, [p(N1,_) | _]) :-  
    !,  
    fail.  
eq_node(_,N2,[p(_,N2) | _]) :-  
    !,  
    fail.  
eq_node(N1,N2,[_|T]) :-  
    eq_node(N1,N2,T).
```

```
eq_node(N1,N2,LI,LI) :-  
    member(p(N1,N2),LI), !.  
eq_node(N1,N2,LI,[p(N1,N2)|LI]) :-  
    not(member(p(N1,_) ,LI)),  
  
    not(member(p(_,N2) ,LI)).
```

Nodes equivalence

(implements φ function)

V4 – with arguments (incomplete lists)

- Same as v2 just that lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the “not found” case
- Just the equivalence changes
 - (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node4 (N1,N2,LI) :-  
    member_il (p (N1,N2) , LI) , !.  
eq_node4 (N1,N2, [p (N1,N2) | LI]) :-  
    not (member_il (p (N1,_), LI)) ,  
    not (member_il (p (_,N2), LI)) .
```

```
member_il (_, L) :- var (L) , ! , fail.  
member_il (X, [X|_]) :- !.  
member_il (X, [_|T]) :- member_il (X, T).
```

- Compare v4 with v2.

Nodes equivalence

(φ function v4 VS v2)

```
eq_node (N1, N2, LI) :-  
    member_il (p (N1, N2), LI), ! .
```

```
eq_node (N1, N2, [p (N1, N2) | LI]) :-  
  
    not (member_il (p (N1, _), LI)) ,  
  
    not (member_il (p (_, N2), LI)) .
```

```
eq_node (N1, N2, LI, LI) :-  
    member (p (N1, N2), LI), ! .
```

```
eq_node (N1, N2, LI, [p (N1, N2) | LI]) :-  
  
    not (member (p (N1, _), LI)) ,  
  
    not (member (p (_, N2), LI)) .
```

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #13
Hamiltonian cycle

Computer Science

Agenda

- **Hamiltonian cycle**
 - What it is
 - Class of the problem
 - How (not to) solve it
 - How to address the issue
 - NP complete/hard problems can be addressed

Hamiltonian cycle

- A Hamiltonian cycle is a graph cycle (a closed loop) that visits every vertex in a graph exactly once, returning to the starting vertex
- Consider the undirected weighted graph:

edge (a, b, 7) .

edge (a, c, 1) .

edge (a, d, 8) .

edge (b, c, 2) .

edge (b, f, 5) .

edge (b, e, 10) .

edge (c, d, 3) .

edge (d, f, 4) .

edge (d, e, 8) .

edge (f, e, 3) .

is_edge (X, Y, W) :-

edge (X, Y, W) ;

edge (Y, X, W) .

Hamiltonian cycle (NP complete/NP hard problem)

```
[q] ?- hamilton(6,a,Cycle,Cost).
```

```
hamilton(N,X,Cycle,Cost):-
```

```
    N1 is N-1,
```

```
    try(N1,X,X,[X],0,Cycle,Cost).
```

```
try(N, X, Y, PPath, PCost, Cycle, Cost):-
```

```
    N > 0,
```

```
    N1 is N-1,
```

```
    is_edge(Y, Z, W),
```

```
    not(member(Z, PPath)),
```

```
    PCost1 is PCost + W,
```

```
    try(N1, X, Z, [Z|PPath], PCost1, Cycle, Cost).
```

```
try(0, X, Y, PPath, PCost, [X|PPath], Cost):-
```

```
    is_edge(Y, X, W),
```

```
    Cost is PCost + W.
```

Hamiltonian cycle (NP complete / NP hard problem)

- Should not be addressed straight away unless the space is VERY small (how small? Size ~ 20 ?)
- What should be done?

Hamiltonian cycle with branch and bound

- Branch and bound
- Estimate the cost of the path in 2 steps, as follows:
 - $\phi_i = \psi_i + X_i$
 - ϕ_i = cost of the path from Start to Stop via the *intermediate* node
 - ψ_i = cost of the path from Start to the *intermediate* node
 - X_i = cost of the path from the *intermediate* node to Stop
 - They are estimated as: $E[\phi_i] = E[\psi_i] + E[X_i]$
 - $E[\phi_i]$ = estimate cost of the path from Start to Stop via the *intermediate* node
 - $E[\psi_i]$ = estimate cost of the path from Start to the *intermediate* node
 - $E[X_i]$ = estimate cost of the path from the *intermediate* node to Stop
- $E[\psi_i] = \psi_i$
- $E[X_i]$ = the better the estimate, more we narrow the search space (bound more effective). Even a rough estimate can help. Even just 0 proves good.
- The approach (similar to `bf_search`) is also using 2 argument lists:
 - Candidate list – potential path to follow (it is a regular list; not incomplete)
 - Expanded list – part of paths already covered (from *start* to *current* node; none complete to objective)

Hamiltonian cycle with branch and bound

- A structure will be used to estimate costs:
`% n(Node, PathLength, EstimatedCost)`
- The Cand lists looks like: `[n(X,Len,Fi)|Cy]`
 X = node explored
 F_i = estimate value of ϕ_i $E[\phi_i] = E[\psi_i] + E[X_i] = \psi_i + 0$
 Len = number of nodes so far, from Start to current (X)
 Cy = parent 'node' = actually the whole path from parent to Start. It is a candidate list!
- Helpers:
`[n(X,Len,Cost) | _] .` % Accesses the first element in the Candidate list
`n(_,Len,_) .` % **Accesses** the **length** of an element in the Candidate list
`n(X,_,_)` % **Accessing** the **node**

Computer Science

Hamiltonian cycle with branch and bound

```

hamilton(N,X,Cycle):-
    search(N,X,[[n(X,0,0)]],[],Cycle).
% searches for N nodes to end in X the way; put in Cands a path containing just one element: [n(X,0,0)]
% start node X, estimated weight of ham E[Xi]=0, number of nodes 0, NO parent. Expanded list empty.

search(N,_,_,[Cycle|_],Cycle):-
    Cycle=[n(_,N,_)|_],!.
search(N,X,[C|Cands],Exp,Cycle):-
    expand(N,X,C,Cands,NewCands),
    search(N,X,NewCands,[C|Exp],Cycle).

% stop with path the way in front of Expanded
% if its length equals the number of nodes in graph

% if not, expand best node so far= the element in front
% and continue after expansion with the new Cands

% 3rd arg is the best candidate list which is expanded,
% take first node (last added) and find neighbours
expand(N,X,[n(Y,Len,Fi)|Cy],_,_):-
    is_edge(Y,Z,W),           % nondeterministically take Z, first neighbor of Y (eventually all of them) and weight W
    (not(member_path(Z,Cy)); (Len is N-1,Z=X)), % should NOT be already processed
    % just if it is the source
    FiW is Fi+W,Len1 is Len+1, % estimate the new parameters
    assertz(node(n(Z,Len1,FiW))), % put it in the akb
    fail. % backtrack to a new neighbor of Y
expand(_,_,C,Cands,NewCands):-
    collect(C,Cands,NewCands). % start collecting from akb and place in NewCand

```

Hamiltonian cycle with branch and bound

```

collect(C, Cands, NewCands) :- % L is the parent, a whole path. Is Y in Candidate from clause 2 in search
    retract(node(Node)), !, % take top from akb -> Node=n(Node, Len, Cost)
    ins_ord_list([Node|C], Cands, IntCands), % add in Cands with the whole path
    collect(C, IntCands, NewCands). % continue with the Intermediate Candidates
collect(_, Cands, Cands).

ins_ord_list(X, [H|T], [H|R]) :-
    X = [n(_,_,Xfi)|_], % Extract estimated cost of X
    H = [n(_,_,Yfi)|_], % Extract estimated cost of Y
    Xfi >= Yfi, !, % Compares the E[ψi] functions for nodes X and Y
    ins_ord_list(X, T, R).
ins_ord_list(X, T, [X|T]) :- !. % adds an element

member_path(X, [n(X,_,_)|_]). % checks if a node is in a list
member_path(X, [_|T]) :- member_path(X, T).

```

Hamiltonian cycle with branch and bound

Complete code

```
% hamilton /3 (+NumberOfNodes, +Source, -Cycle)
hamilton(N,X,Cycle):-
    search(N,X,Cycle,[[n(X,0,0)]],[[]]).
```

```
search(N,X,_,[Cycle|_],Cycle):-
    Cycle=[n(_,N,_)|_],!.
search(N,X,[C|Cands],Exp,Cycle):-
    expand(N,X,C,Cands,NewCands),
    search(N,X,NewCands,[C|Exp],Cycle).
```

```
expand(N,X,[n(Y,Len,Fi)|Cy],_,_):-
    is_edge(Y,Z,W),
    (
        not(member_thread(Z,Cy));
        (Len is N-1,Z=X)
    ),
    FiW is Fi+W,
    Len1 is Len+1,
    assertz(node(n(Z,Len1,FiW))),
    fail.
expand(_,_,C,Cands,NewCands):-
    collect(C,Cands,NewCands).
```

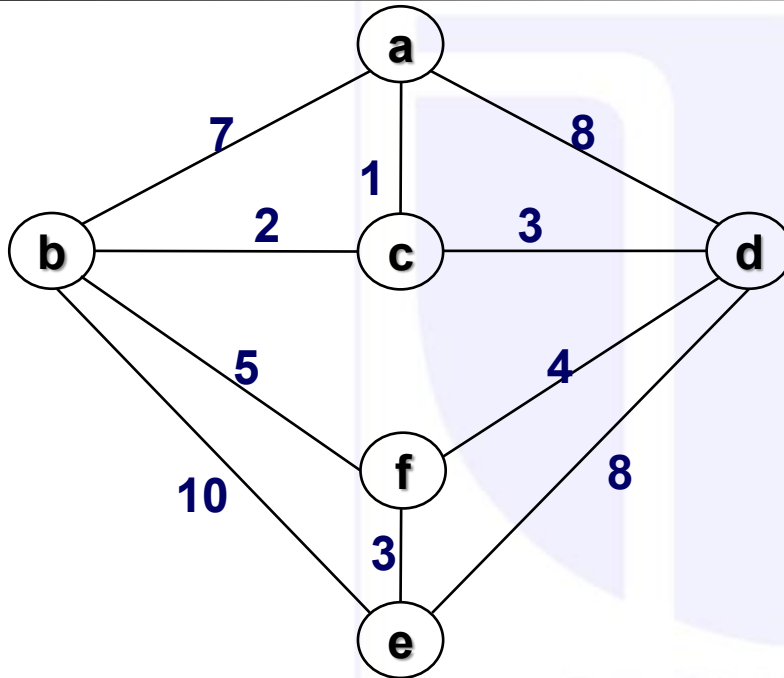
```
collect(C,Cands,NewCands):-
    retract(node(N)),!,
    ins_ord_list([N|C],Cands,IntCands),
    collect(C,IntCands,NewCands).
collect(_,Cands,Cands).
```

```
ins_ord_list(X,[H|T],[H|R]):-
    X = [n(_,_,Xfi)|_],
    H = [n(_,_,Yfi)|_],
    Xfi>=Yfi,!,
    ins_ord_list(X,T,R).
ins_ord_list(X,T,[X|T]):-!.
```

```
member_path(X,[n(X,_,_)|_]).
member_path(X,[_|T]):- member_path(X,T).
```

```
?- hamilton(6, a, Path)
?- search(6,a,[[n(a,0,0)]],[[]],Way).
```

Example



Cand – ordered list of candidate partial paths, waiting expansion

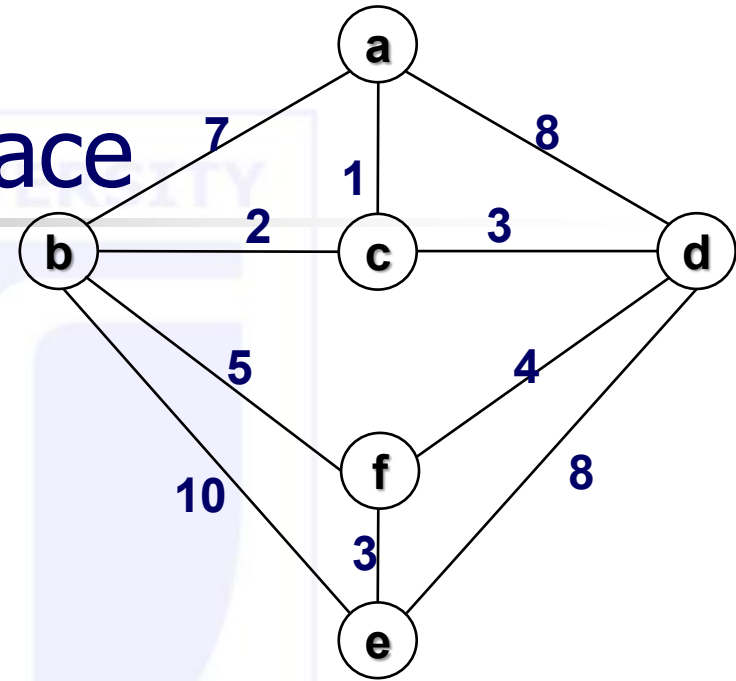
Exp – accumulator list of expanded partial paths (also ordered)

?- search(6,a,[[n(a,0,0)]],[],Cycle).

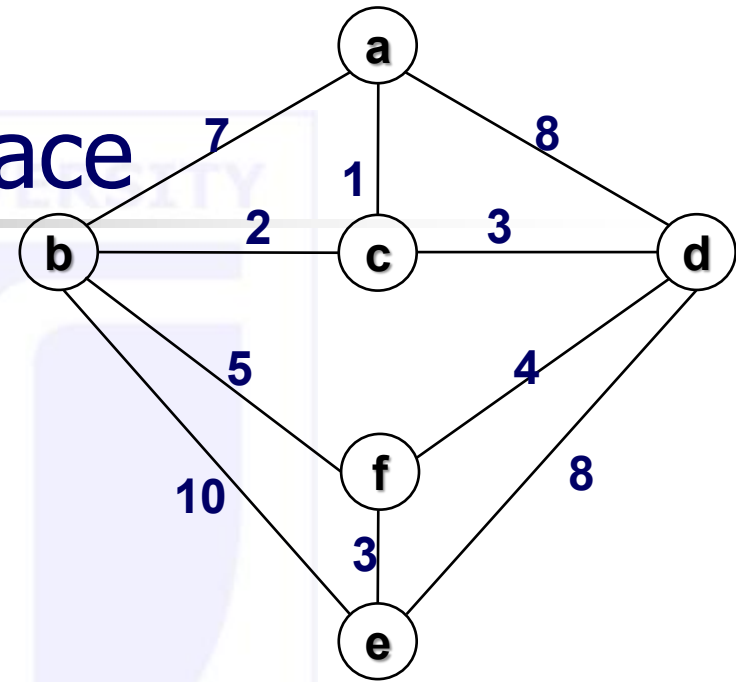
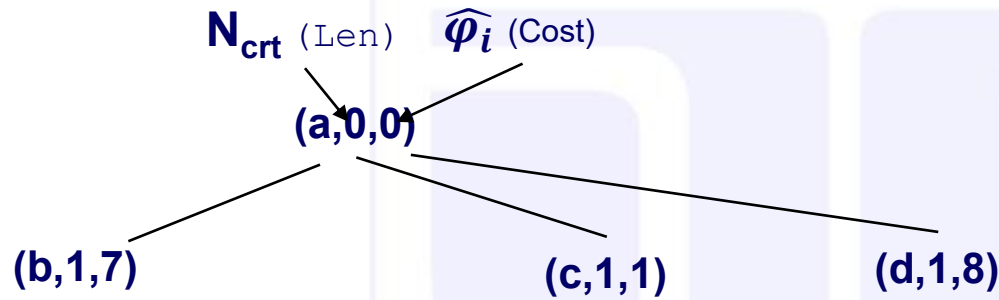
result list; at the end (crt length = N) it is first element of **Exp**

Example – search space

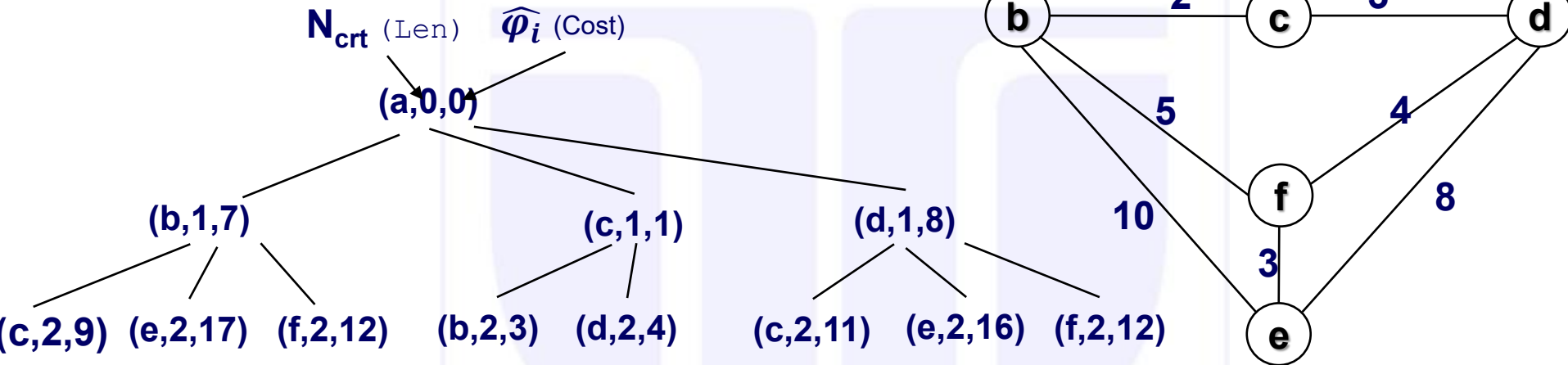
N_{crt} (Len) $\widehat{\varphi}_i$ (Cost)
 (a,0,0)



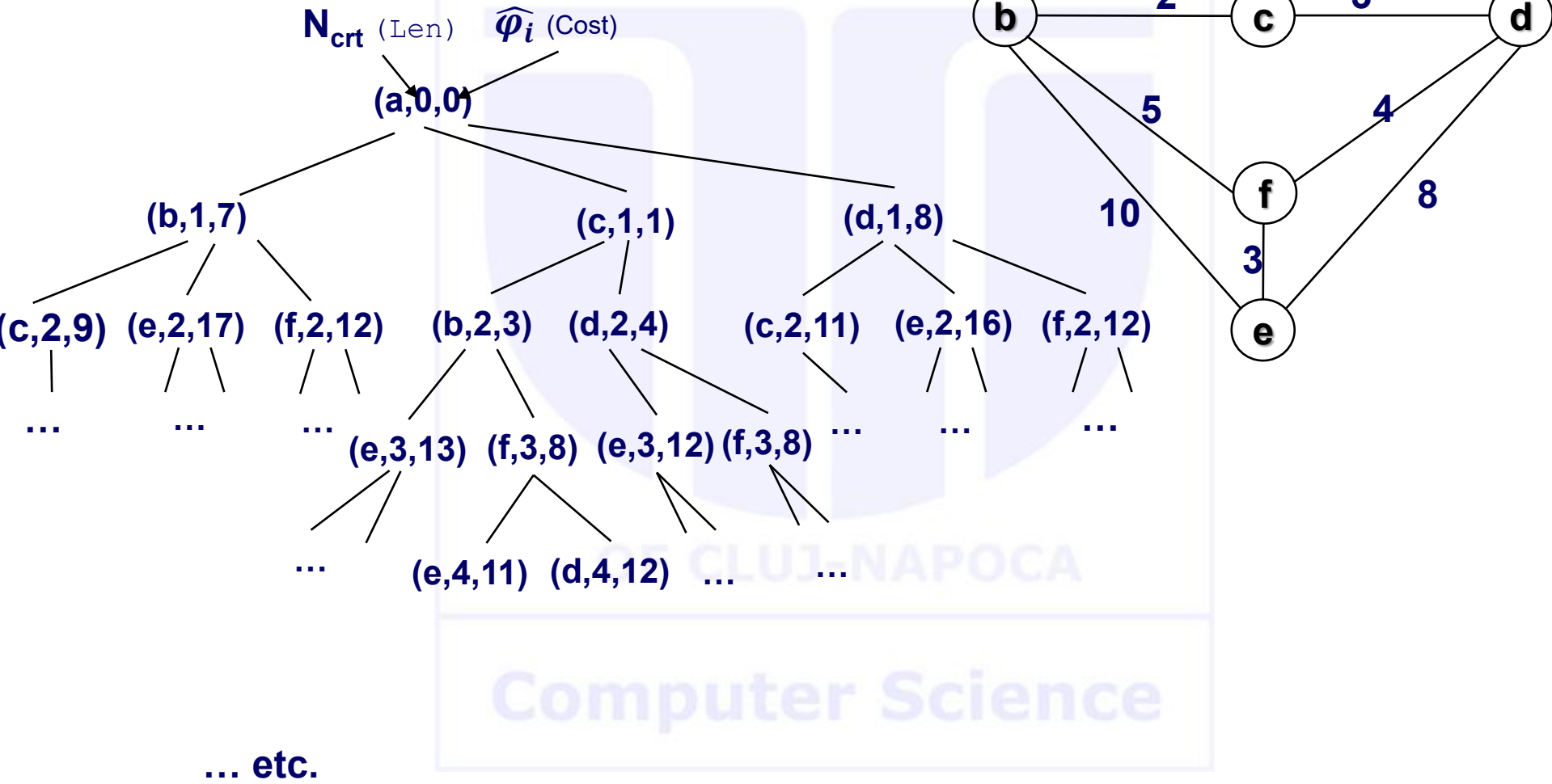
Example – search space



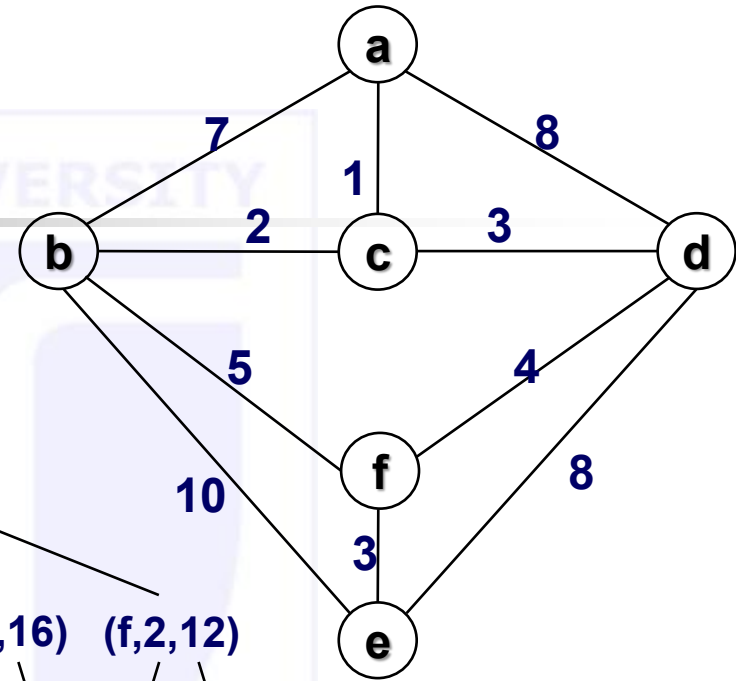
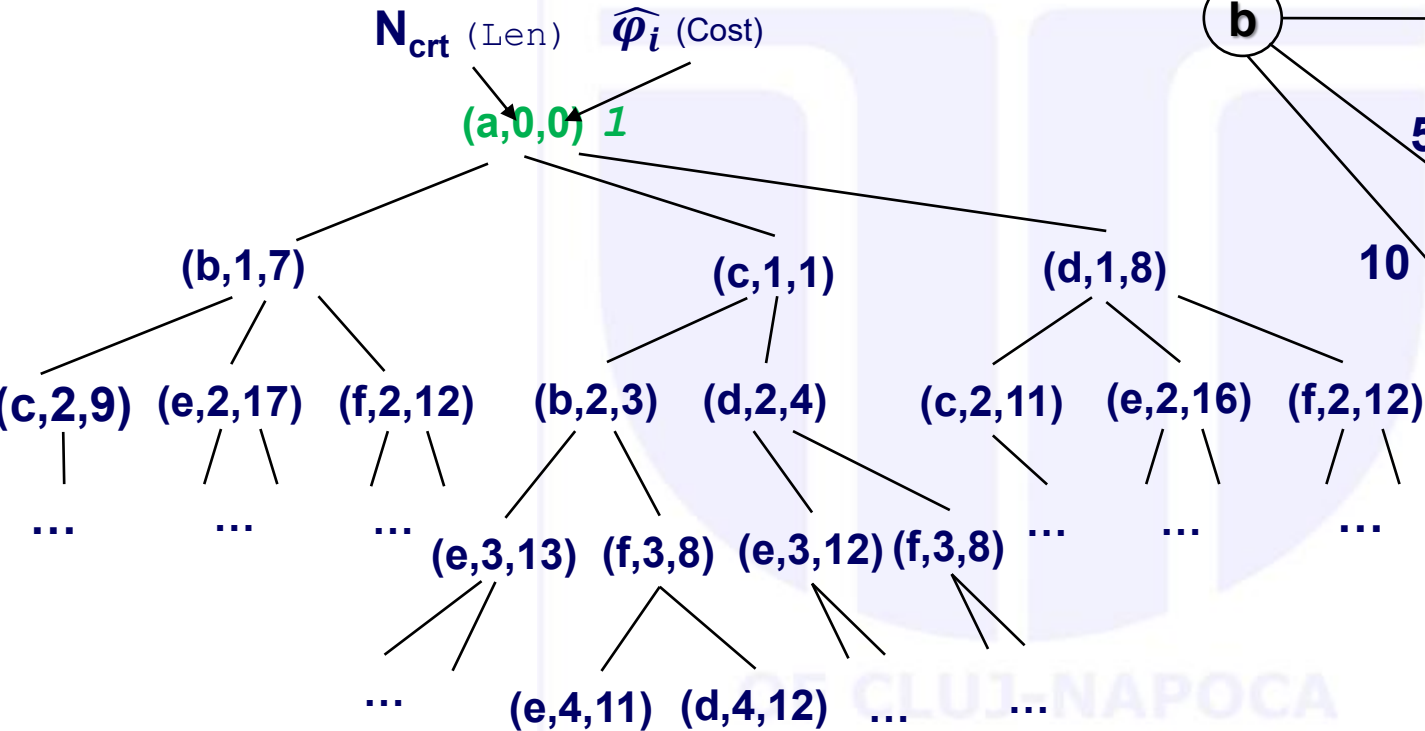
Example – search space



Example – search space



Example – traversal

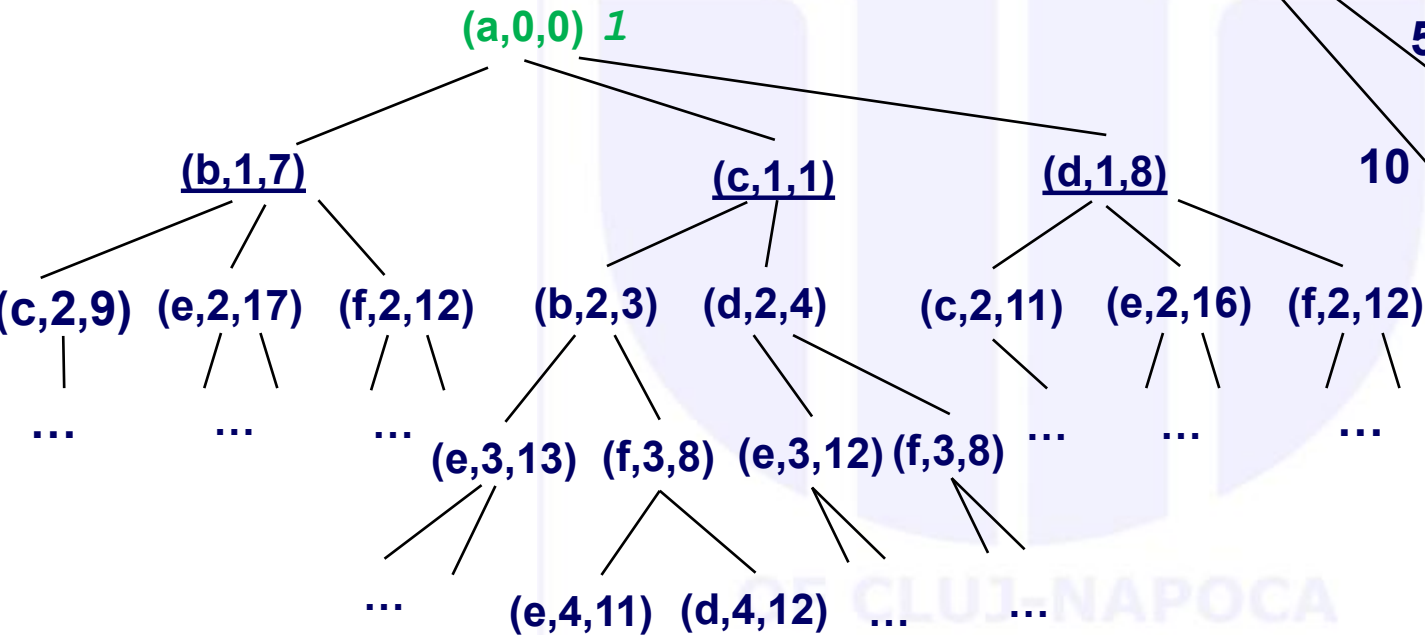
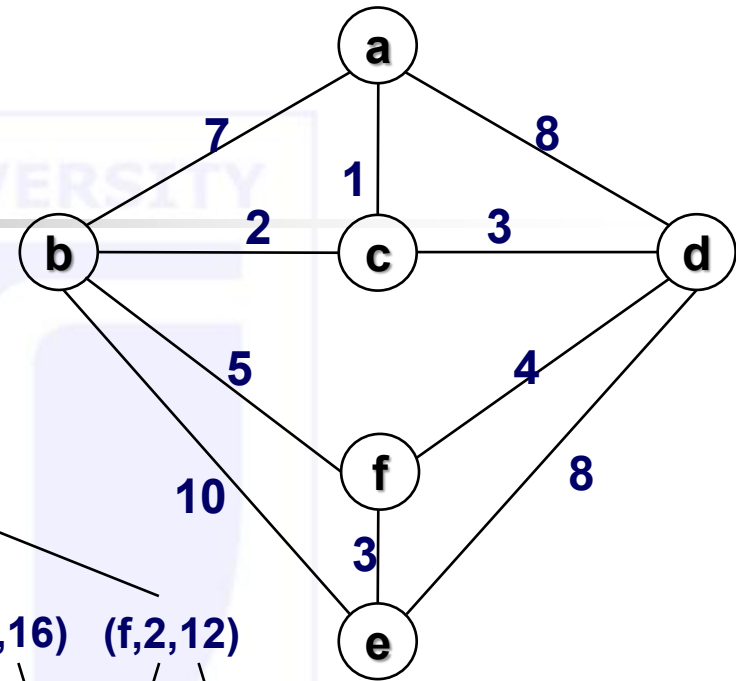


... expand first element in Cand

Cand = $[[n(a, 0, 0)]]$

Exp = $[[$

Example – traversal

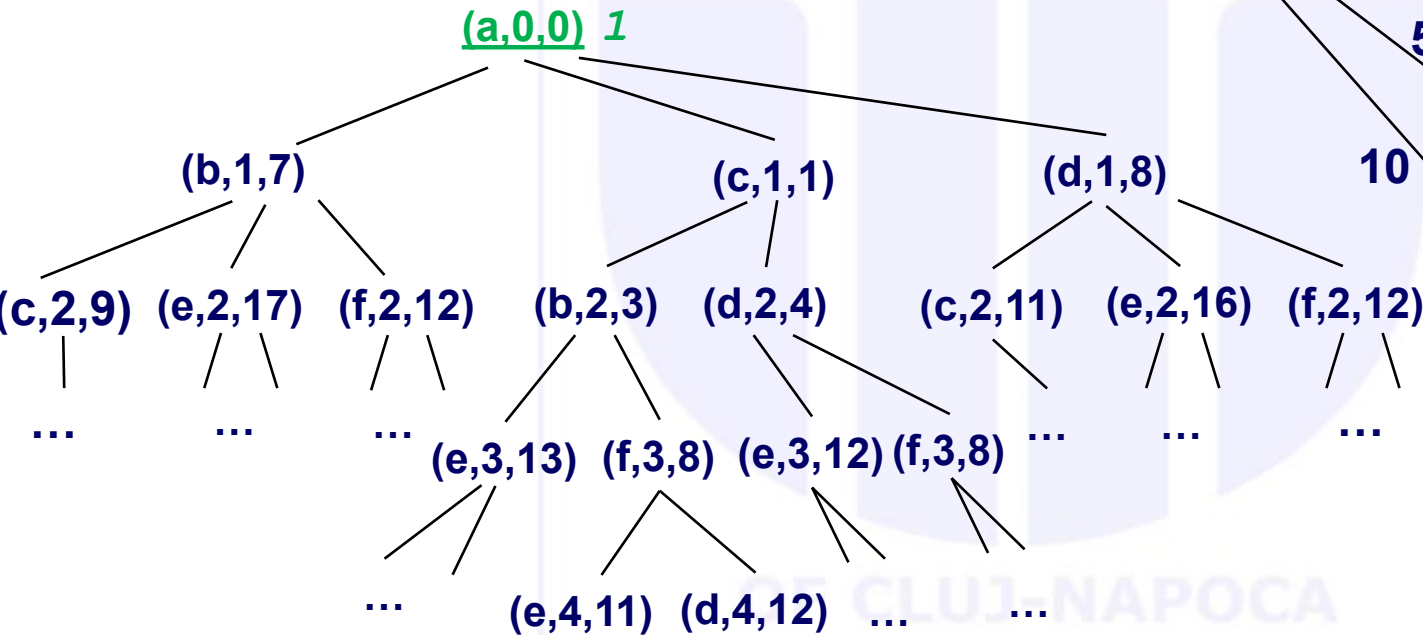
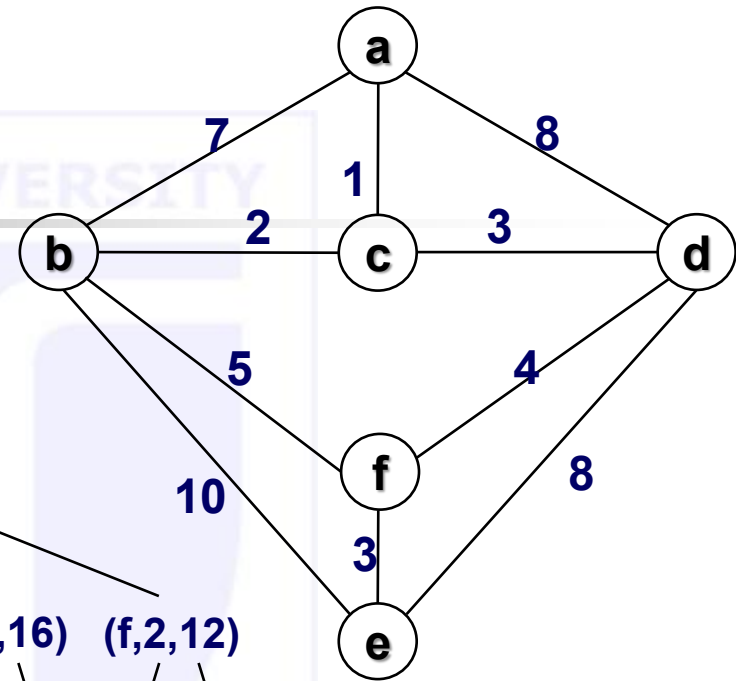


... after expanding $[n(a, 0, 0)]$

Cand = [
 $[n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$

]
 Exp = []

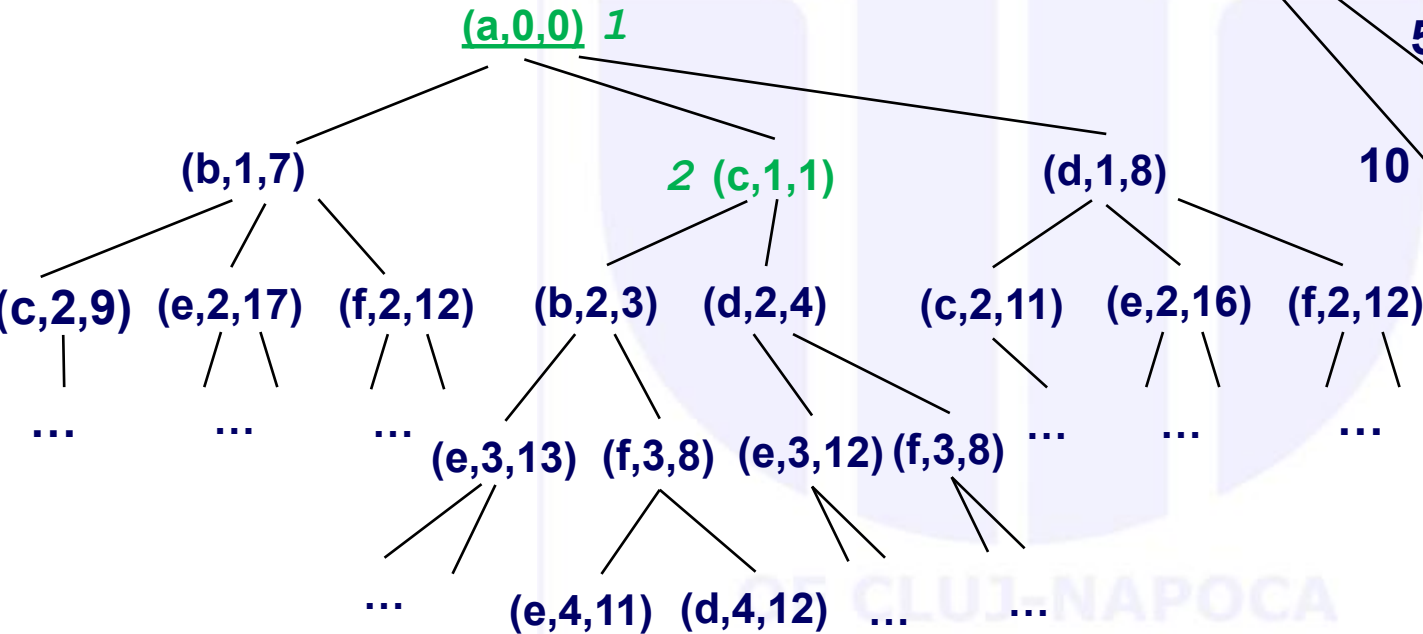
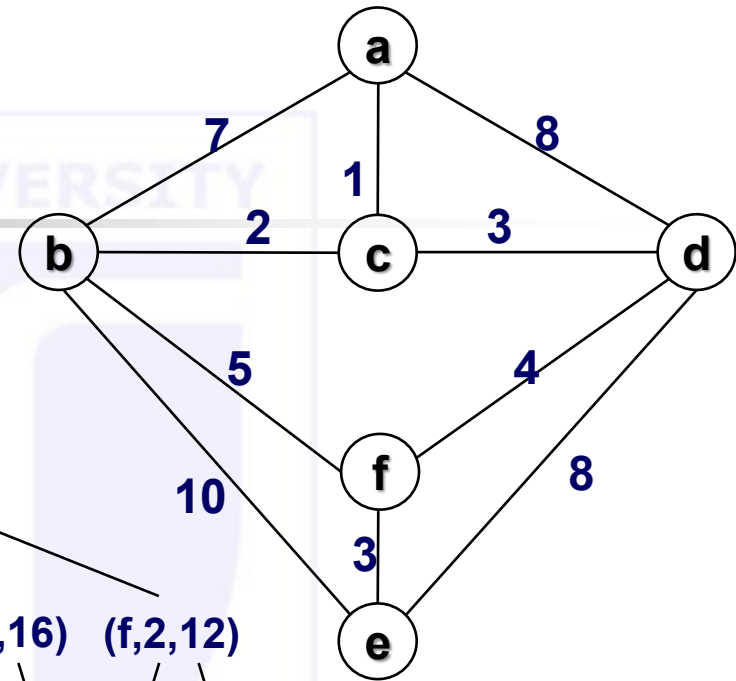
Example – traversal



... recursive call search

```
Cand = [
    [n(c,1,1), n(a,0,0)],
    [n(b,1,7), n(a,0,0)],
    [n(d,1,8), n(a,0,0)]
]
Exp = [[n(a,0,0)]]
```

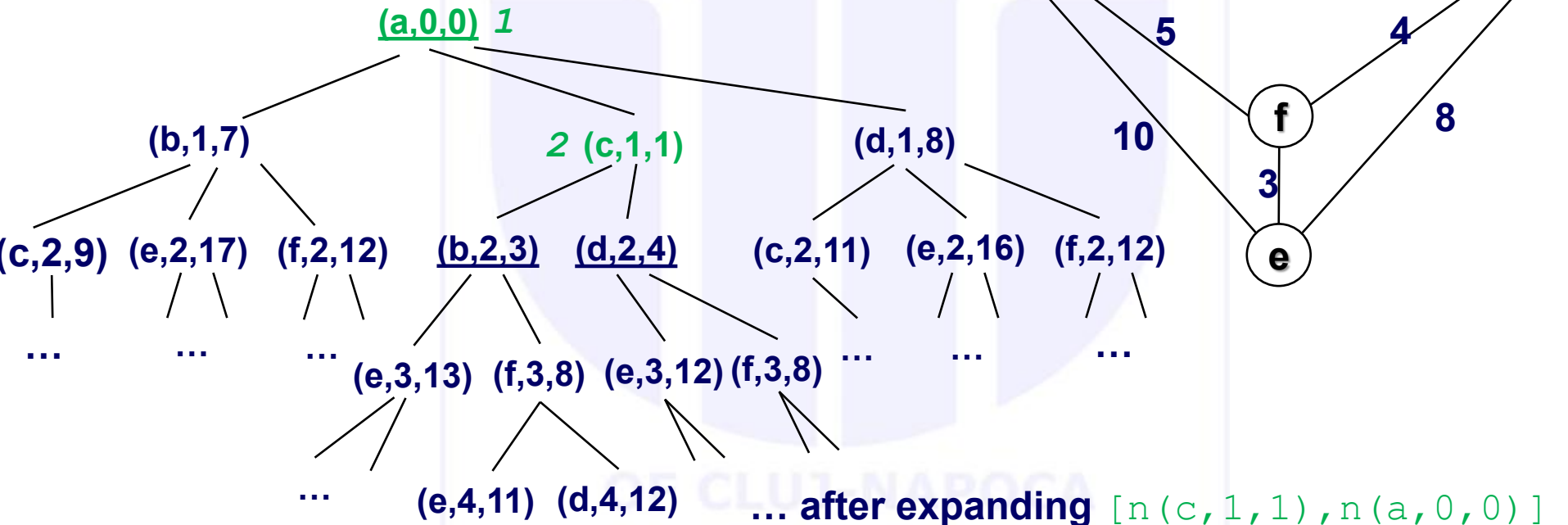
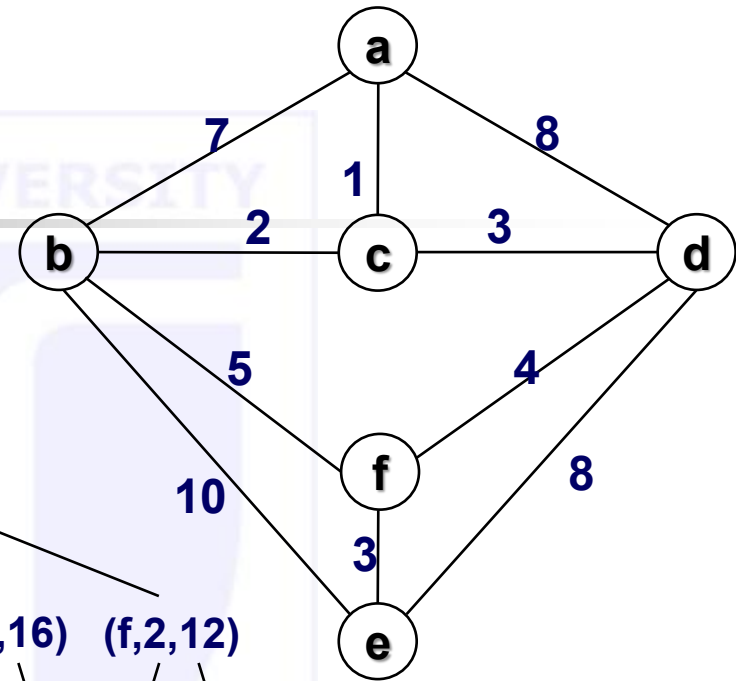
Example – traversal



... this will be expanded next

Cand = [
 $[n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$
]
 Exp = $[[n(a, 0, 0)]]$

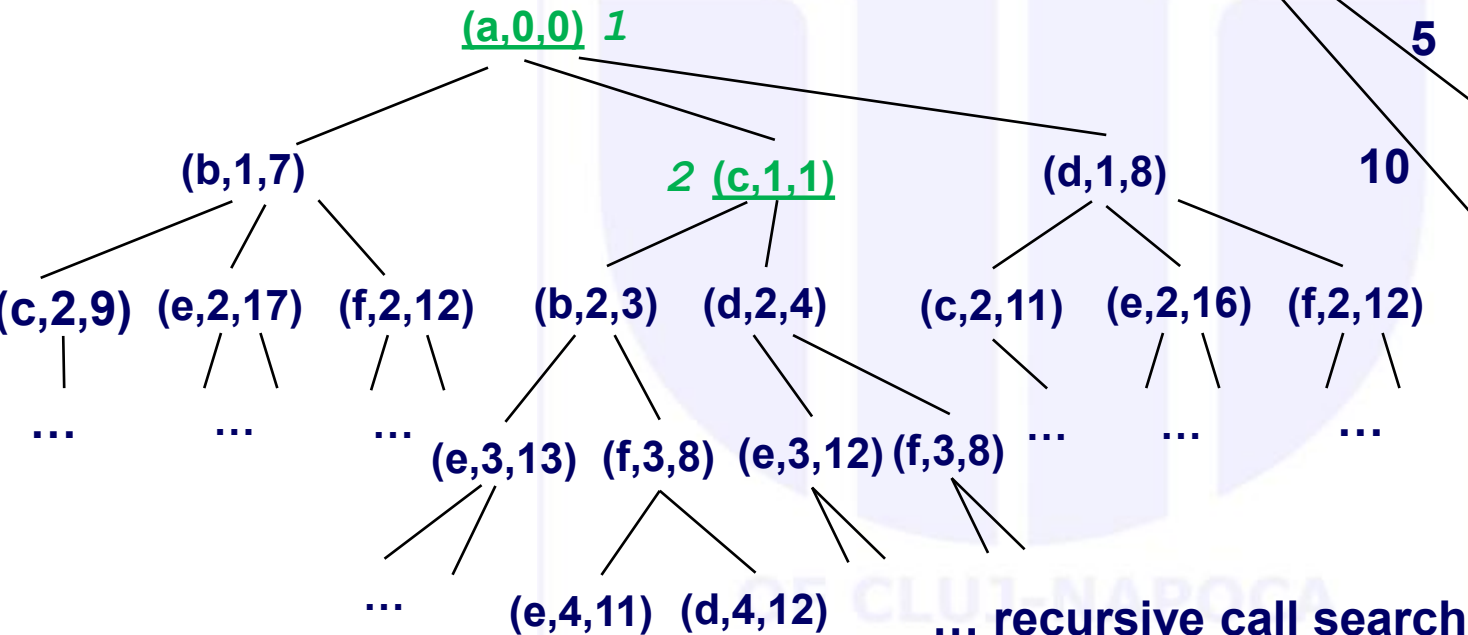
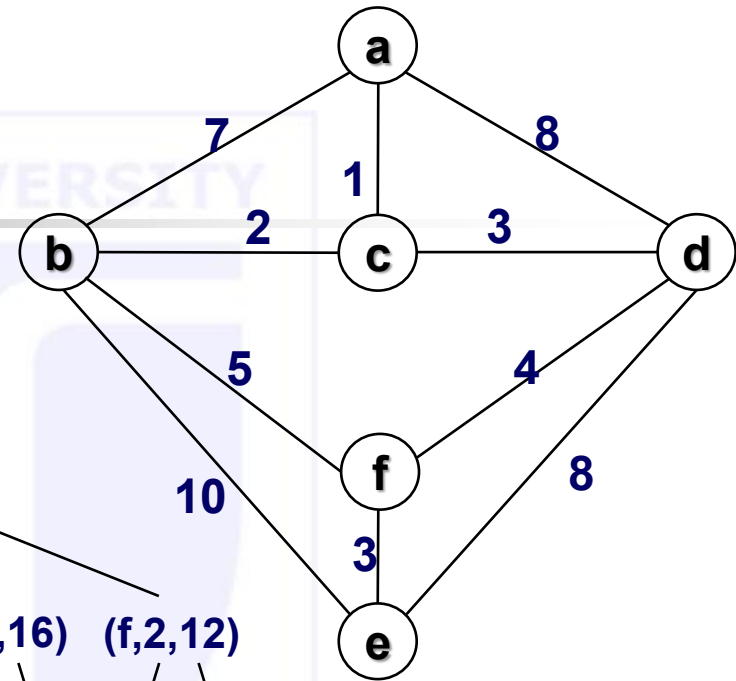
Example – traversal



... after expanding $[n(c, 1, 1), n(a, 0, 0)]$
 Cand = [
 $[n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(d, 2, 4), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$
]

Exp = $[[n(a, 0, 0)]]$

Example – traversal

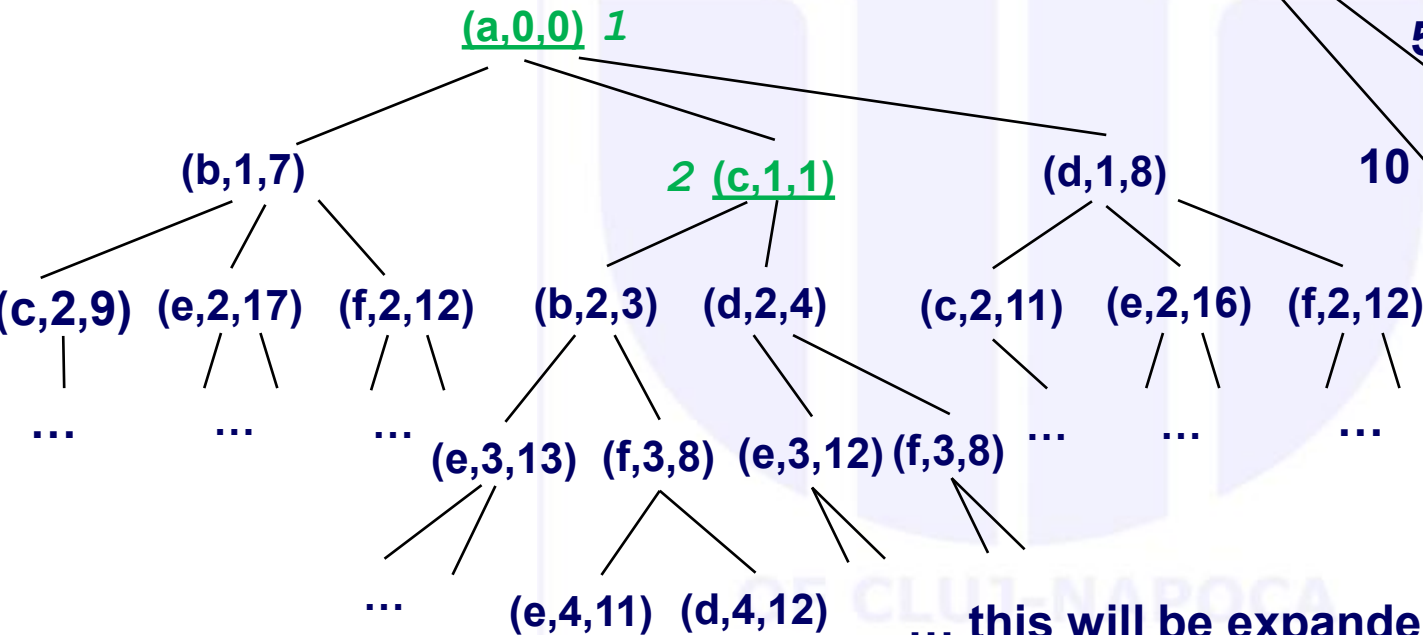
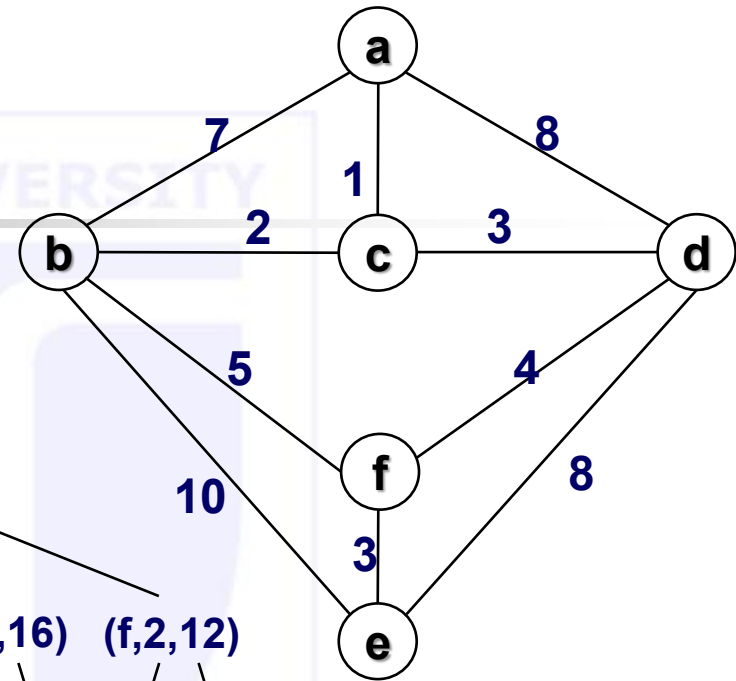


... recursive call search

Cand = [
 $[n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(d, 2, 4), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$
 $]$

Exp = $[[n(c, 1, 1), n(a, 0, 0)], [n(a, 0, 0)]]$

Example – traversal



... this will be expanded next

Cand

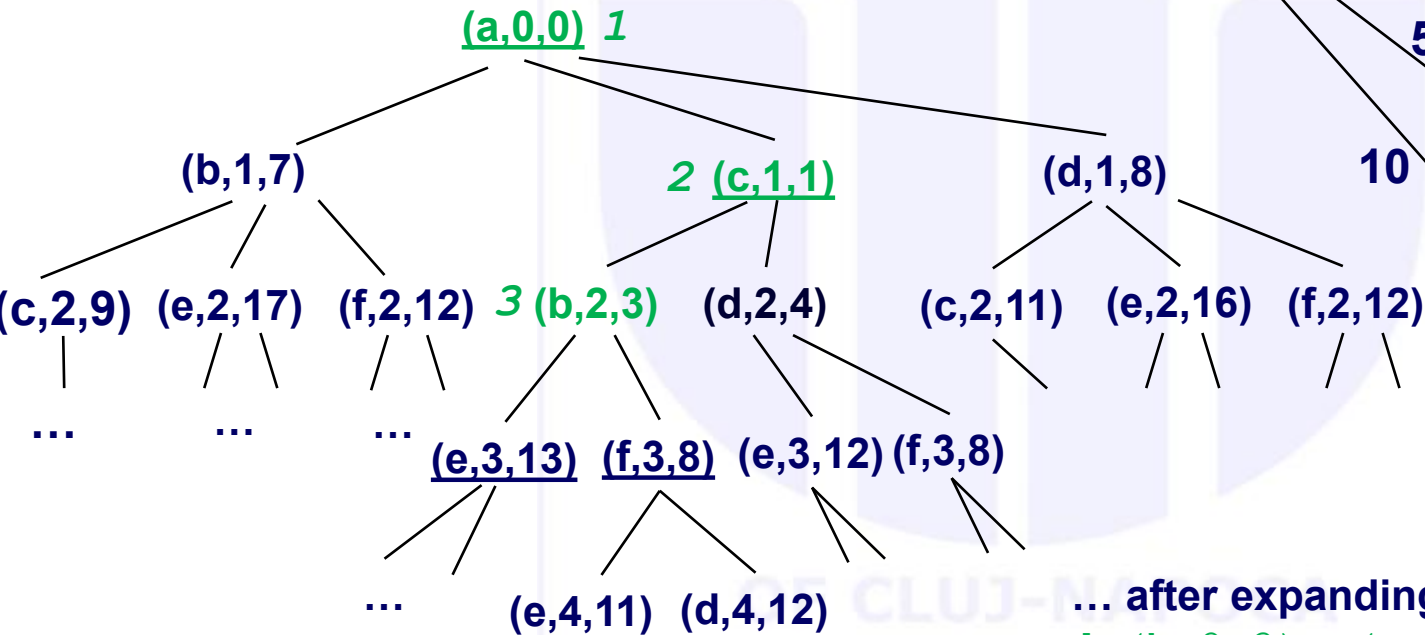
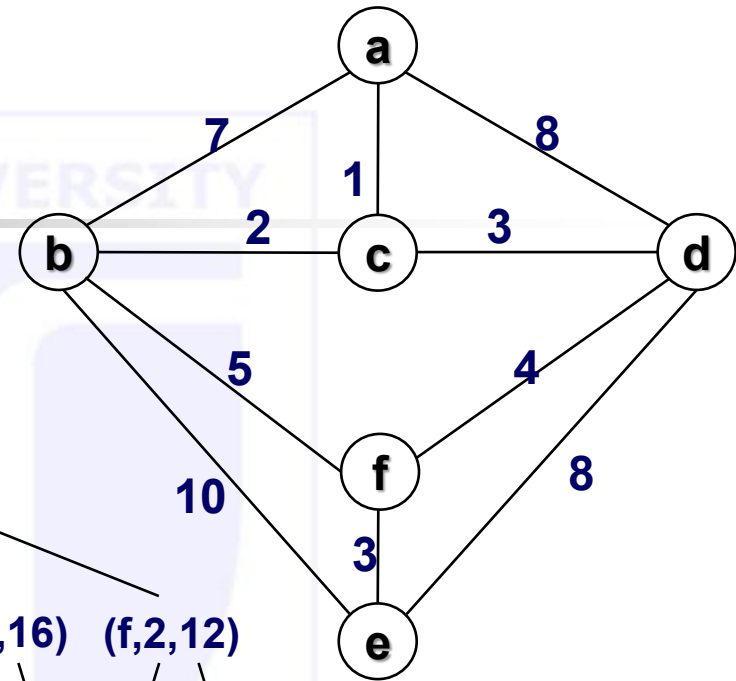
= [
 [n(b,2,3), n(c,1,1), n(a,0,0)],
 [n(d,2,4), n(c,1,1), n(a,0,0)],
 [n(b,1,7), n(a,0,0)],
 [n(d,1,8), n(a,0,0)]

]

Exp

= [[n(c,1,1), n(a,0,0)], [n(a,0,0)²²]]

Example – traversal



... after expanding

$[n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$

Cand

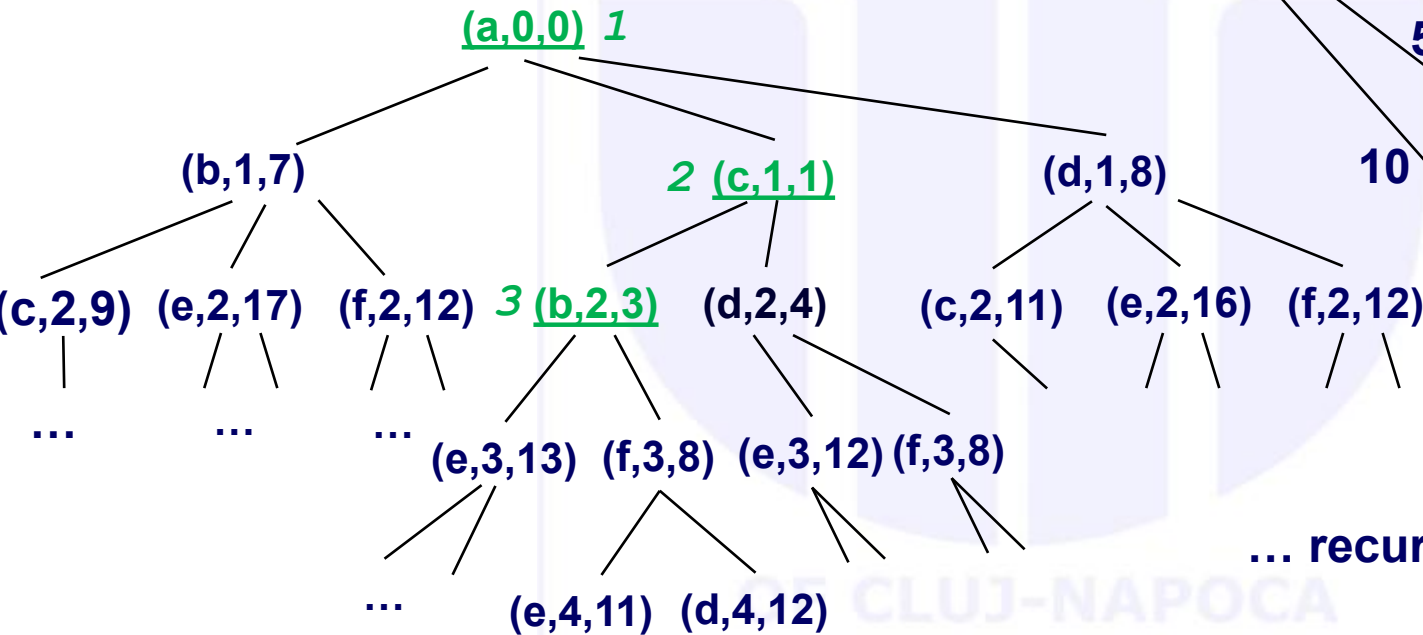
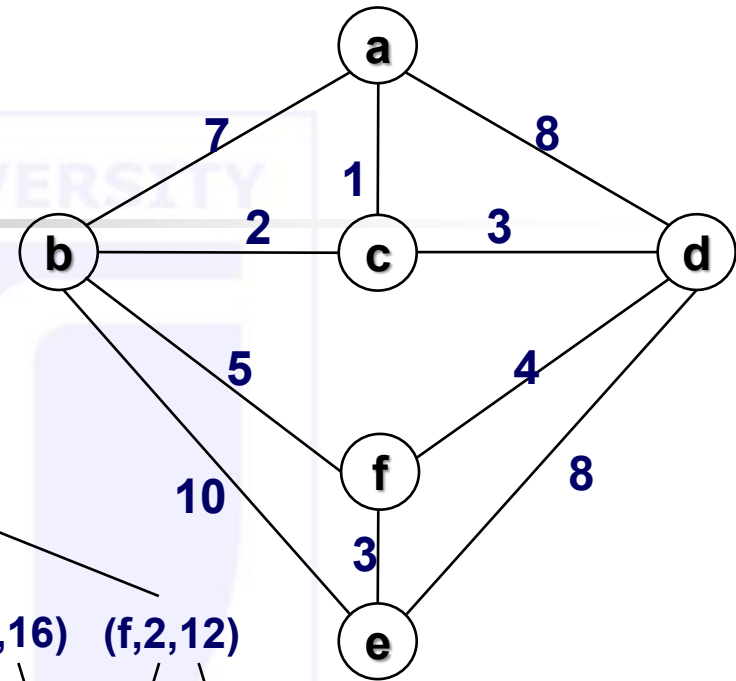
= [
 $[n(d, 2, 4), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(b, 1, 7), n(a, 0, 0)]$,
 $[n(d, 1, 8), n(a, 0, 0)]$,
 $[n(f, 3, 8), n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$,
 $[n(e, 3, 13), n(b, 2, 3), n(c, 1, 1), n(a, 0, 0)]$

]

Exp

= $[[n(c, 1, 1), n(a, 0, 0)], [n(a, 0, 0)]]$

Example – traversal

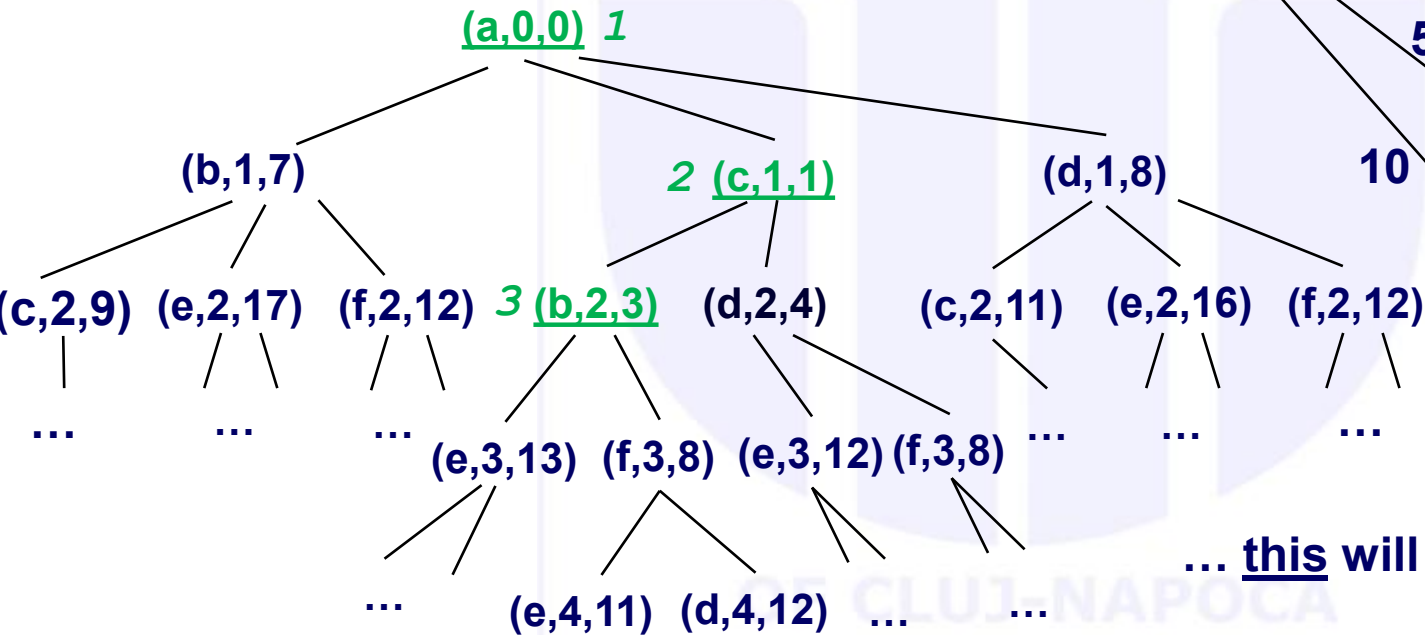
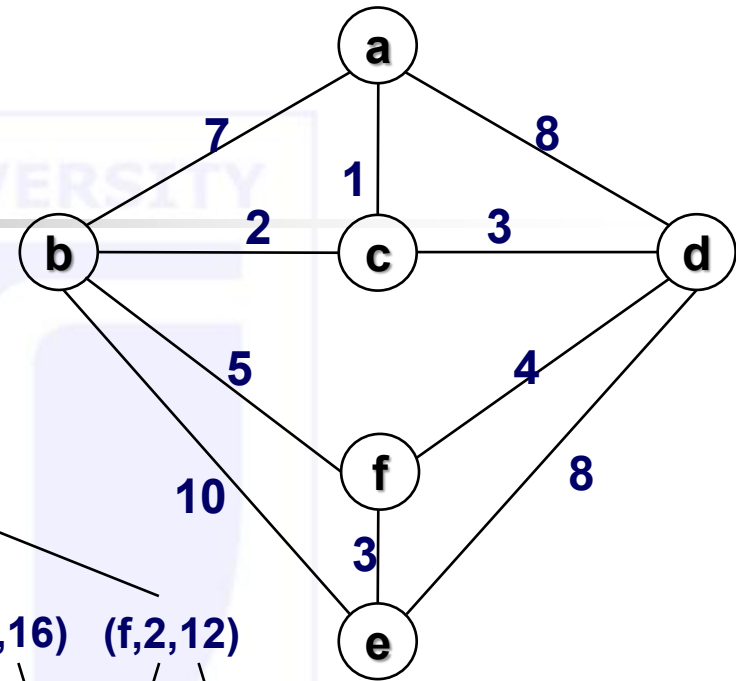


... recursive call search

```
Cand = [
    [n(d,2,4), n(c,1,1), n(a,0,0)],
    [n(b,1,7), n(a,0,0)],
    [n(d,1,8), n(a,0,0)],
    [n(f,3,8), n(b,2,3), n(c,1,1), n(a,0,0)],
    [n(e,3,13), n(b,2,3), n(c,1,1), n(a,0,0)]
]

Exp = [[n(b,2,3), n(c,1,1), n(a,0,0)],
        [n(c,1,1), n(a,0,0)],
        [n(a,0,0)]]
```

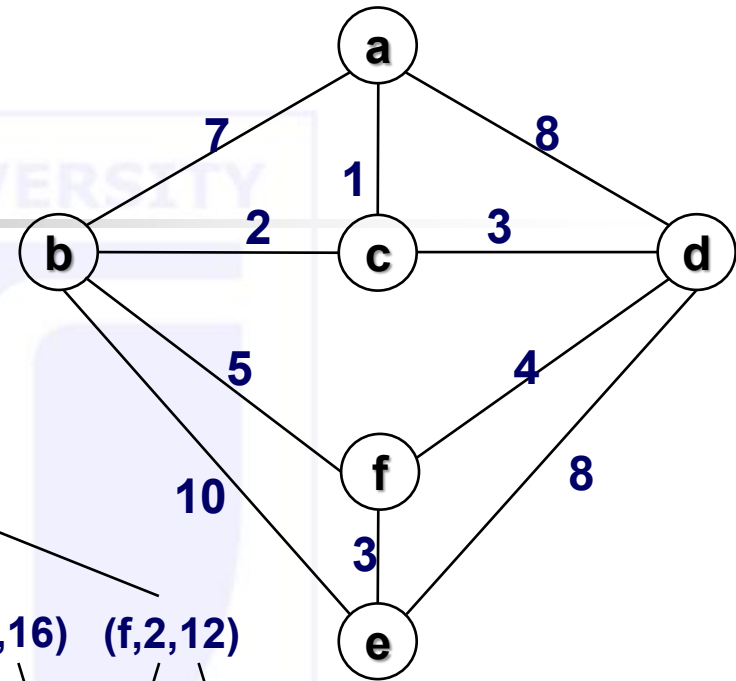
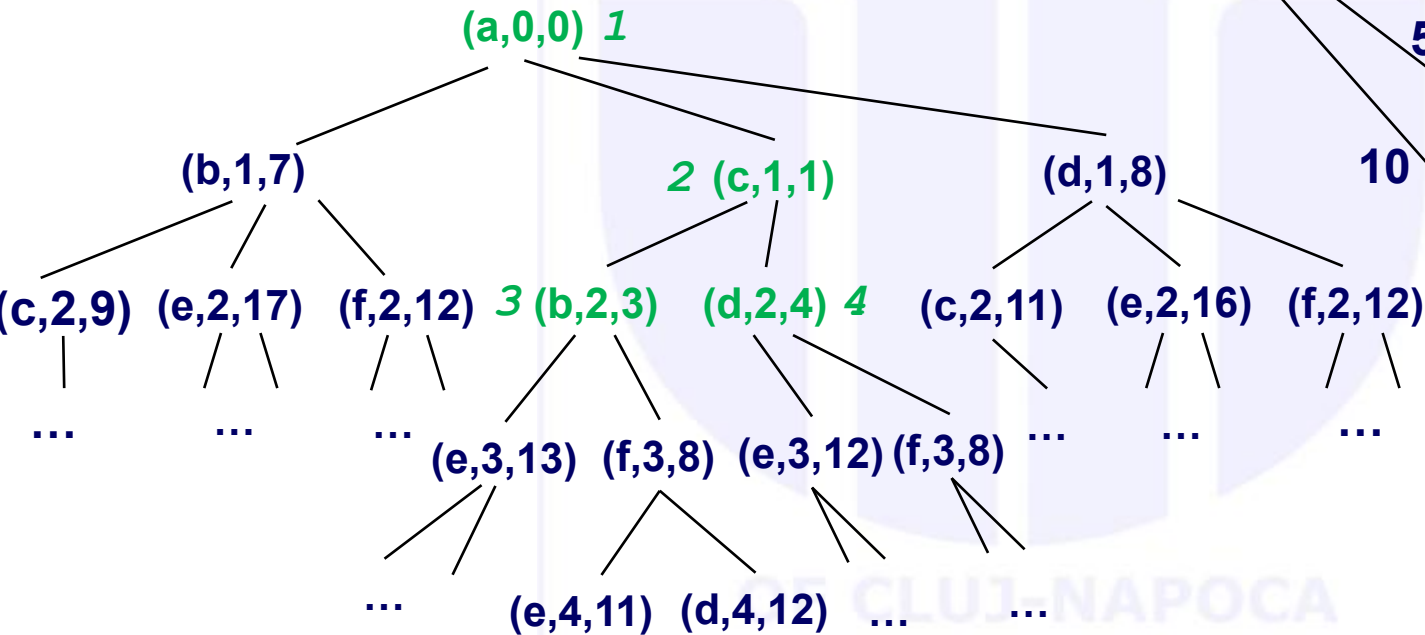
Example – traversal



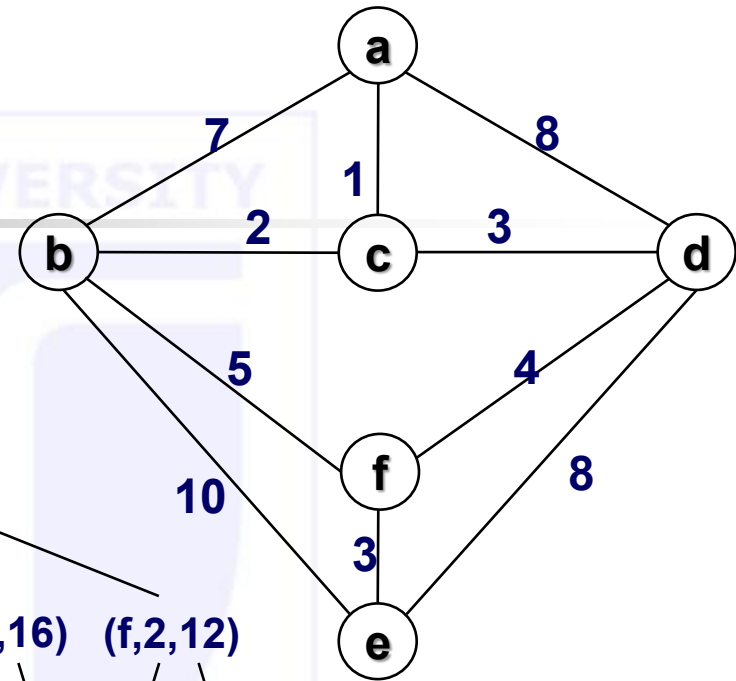
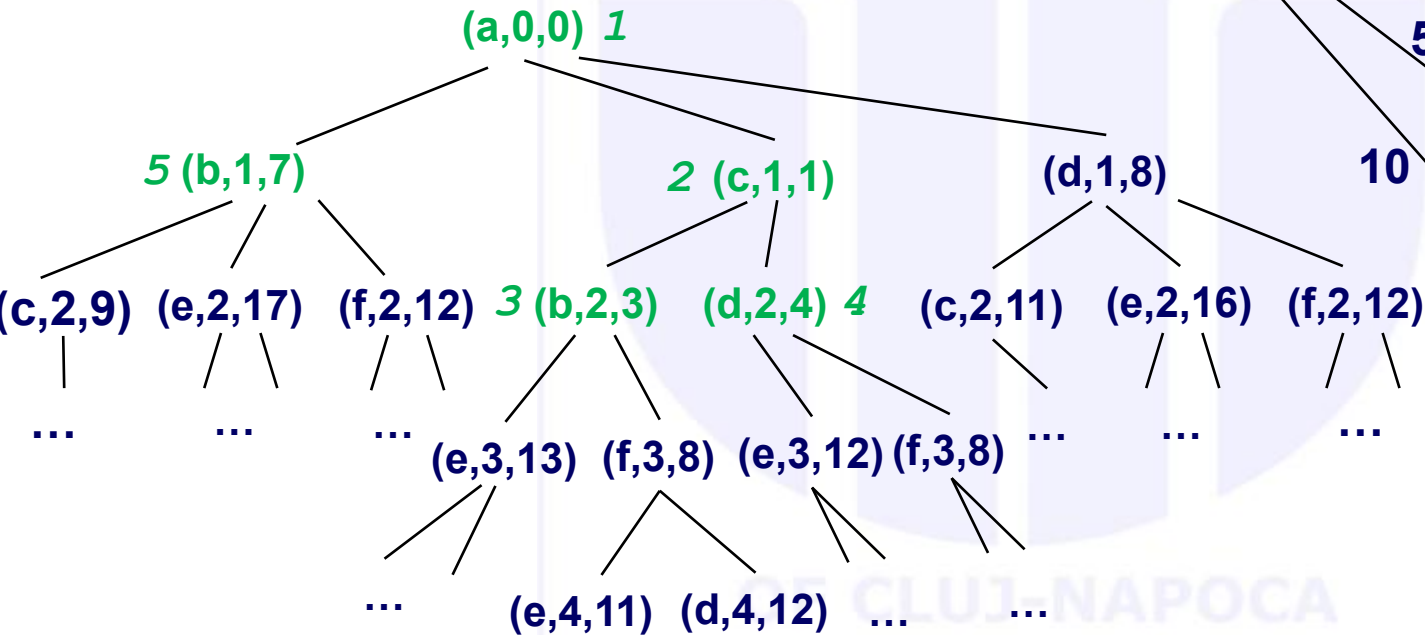
... this will be expanded next

Cand = [
 $\frac{[n(d,2,4), n(c,1,1), n(a,0,0)]}{[n(b,1,7), n(a,0,0)]},$
 $[n(d,1,8), n(a,0,0)],$
 $[n(f,3,8), n(b,2,3), n(c,1,1), n(a,0,0)],$
 $[n(e,3,13), n(b,2,3), n(c,1,1), n(a,0,0)]$
 $]$
 Exp = $[[n(b,2,3), n(c,1,1), n(a,0,0)],$
 $[n(c,1,1), n(a,0,0)],$ $[n(a,0,0)]]$

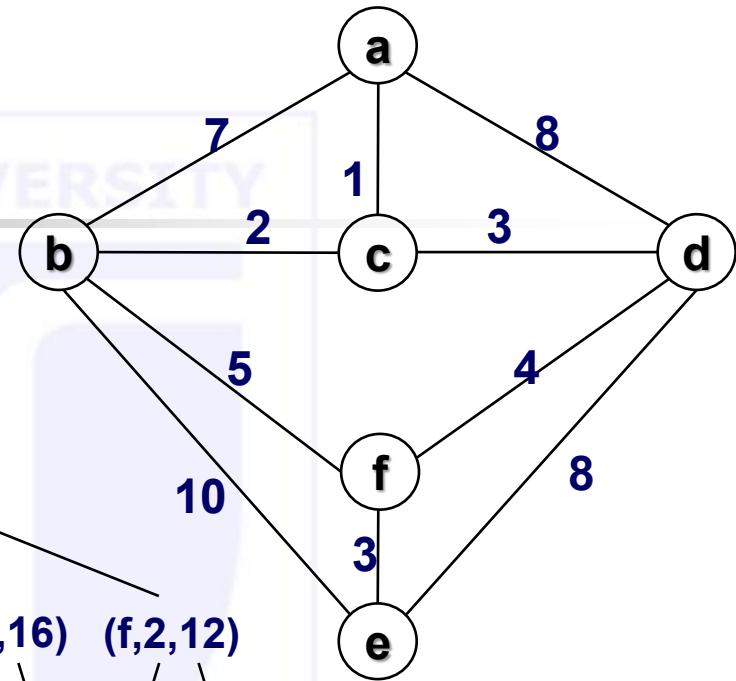
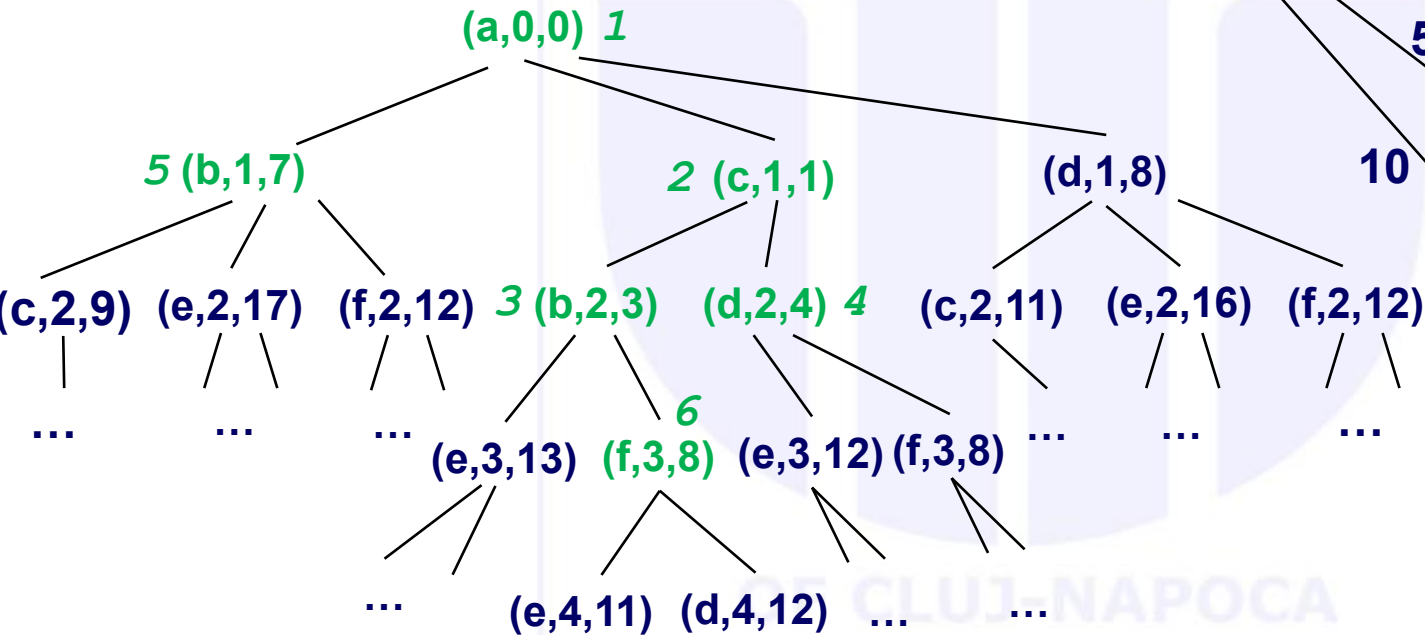
Example – traversal



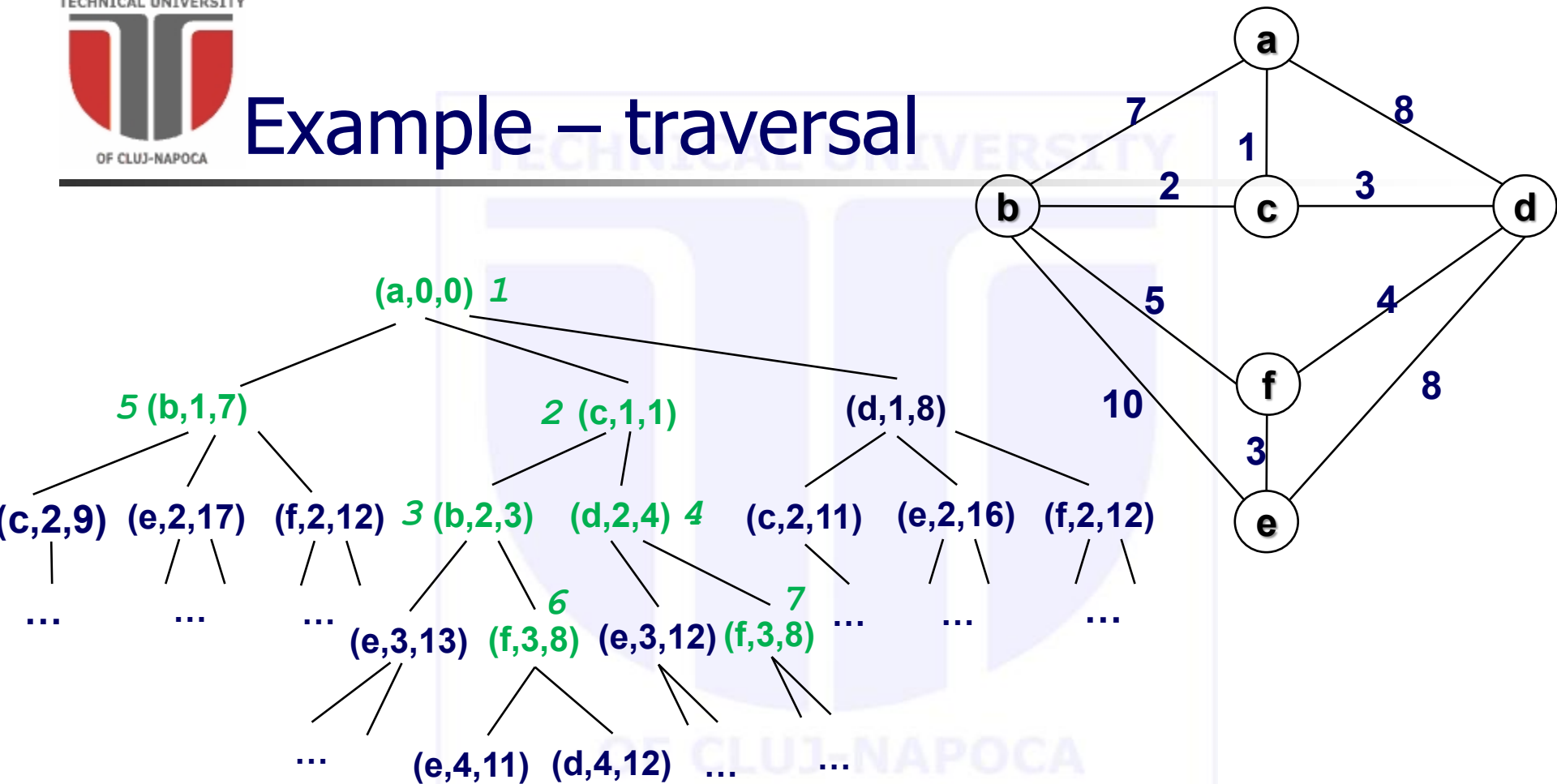
Example – traversal



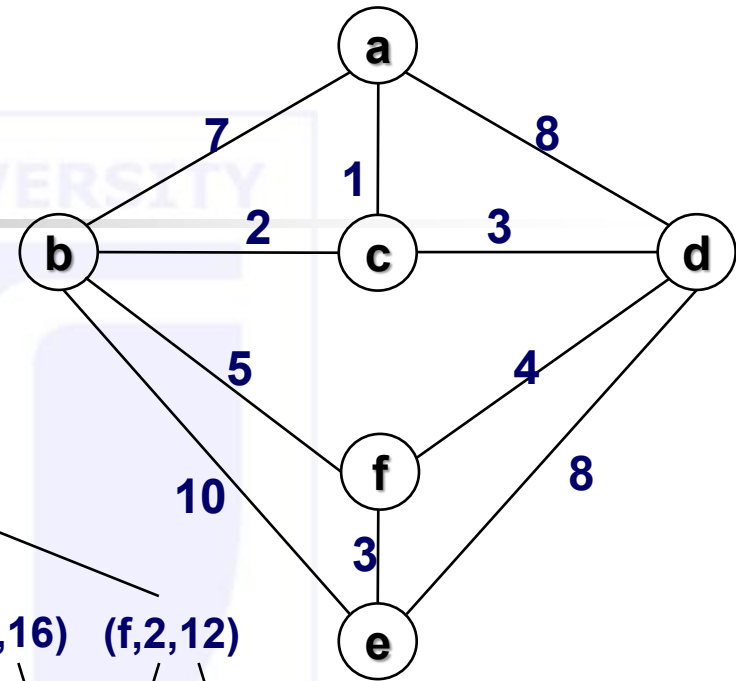
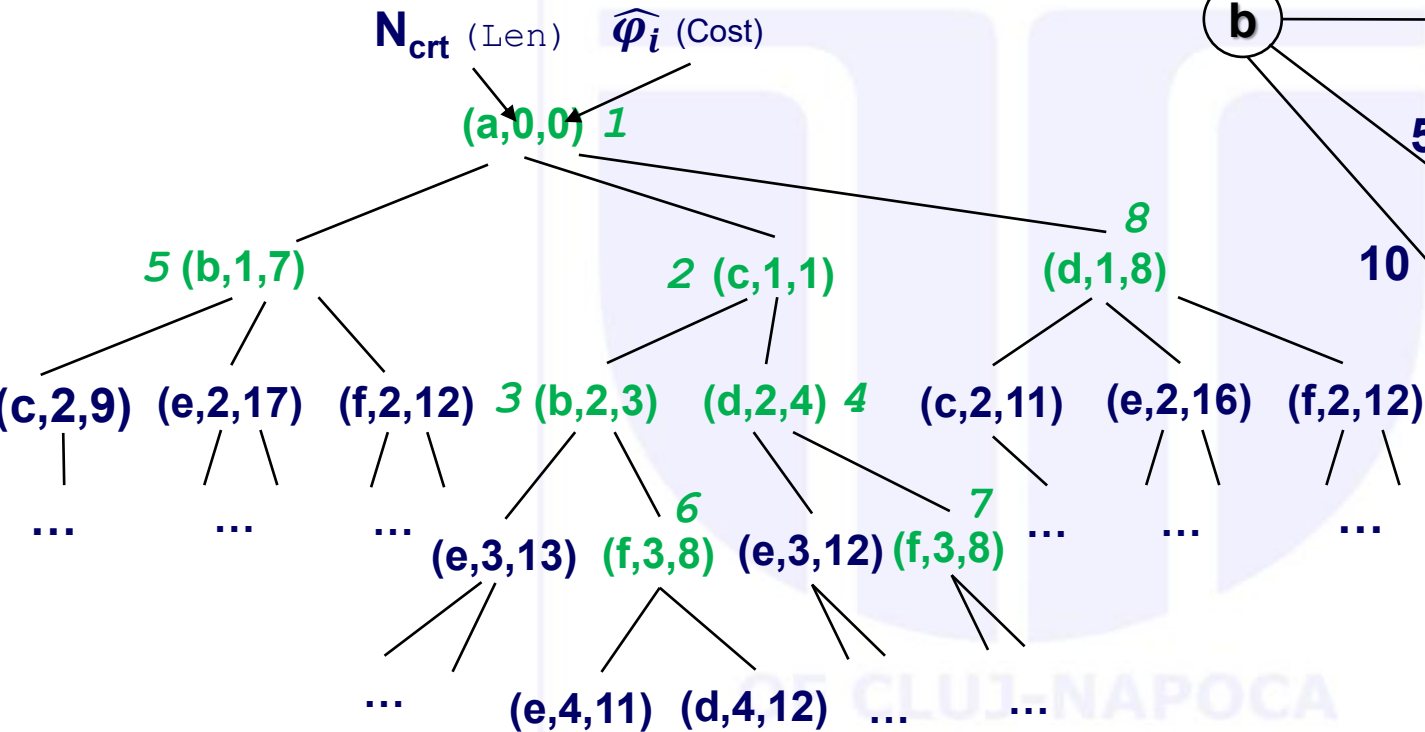
Example – traversal



Example – traversal



Example – traversal



Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #14

Surprise

Computer Science

Agenda

- **Surprise**



Surprise (Your questions)

- Quiz analysis – example of exercises
- Summary / Cheat Sheet

Quiz example

Given the compound term $n(c(A, B), v, m(1, [2, 3]))$, which of the following terms it CANNOT unify with, and why?

Select one:

- ☐ a. $N(c(a, B), v, m(1, [2, 3]))$, because the name of the structure has to be constant
- ☐ b. $n(A, B, C)$, because not all three arguments can be variable
- ☐ c. $N(c(a, B), v, m(1, [H|T]))$, because the third arguments do not unify
- ☐ d. $n(c(a, B), v, m(1, [2, B]))$, because the unifications involving B are in contradiction

Quiz example – contd.

Choose the right implementation of the `member` predicate to ensure a nondeterministic behavior:

Select one or more:

- ☐ a. `member(H, [H|_]) .`
`member(H, [_|T]) :-`
`member(H, T) .`
- ☐ b. `member(H, [H|_]) :- ! .`
`member(H, [_|T]) :-`
`member(H, T) .`
- ☐ c. `member(H, [H|_]) .`
`member(H, [_|T]) :- ! ,`
`member(H, T) .`
- ☐ d. `member(_, []) .`
`member(H, [H|_]) .`
`member(H, [_|T]) :-`
`member(H, T) .`

Quiz example – contd.

The predicate below is assumed to remove **just the first** occurrence of the first argument, from the list given in the second argument, to generate the list in the third argument.

To behave accordingly (as in the associated example), the code should be updated in the following way:

```
delete(X, [X|T], T) .  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .  
delete(_, [], []) .  
?- delete(b, [a,b,c,b,d], R) .  
R= [a,c,b,d]? ;  
no
```

Select one:

- ☐ a. Update the clause 1 as `delete(X,[X|T],T):-!`.
- ☐ b. Remove the clause `delete(X,[X|T],T)`.
- ☐ c. Add the clause: `delete(X,[X],[])`.
- ☐ d. Remove the clause `delete(_,[],[])`.
- ☐ e. Swap the order of clauses 1 and 3.
- ☐ f. Update the clause 2 as `delete(X,[H|T],[H|R]):-!,delete(X,T,R)`.
- ☐ g. Swap the order of clauses 1 and 2.
- ☐ h. Update the clause 2 as `delete(X,[H|T], R):-!,delete(X,T,R)`.

Quiz example – contd.

The code below is assumed to collect in **IN**order the keys from a BST. Choose the best fit statement among the options:

```
tree_2_list_in(nil,_,_).  
tree_2_list_in(t(Left,Key,Right),First,Last):-  
    tree_2_list_in(Right,Int,Last),  
    tree_2_list_in(Left,First,[Key|Int]).
```

Select one:

- ☐ a. The code is correct as it is if and only if the initial call sets the last argument to the empty list.
- ☐ b. To be correct, the order of the recursive calls must be swapped.
- ☐ c. The code is correct as it is.
- ☐ d. To be correct, in the first clause arguments 2 and 3 should share the same variable.

Quiz example – contd.

Given the edge representation of an undirected graph, the predicate `difference_graph` below generates the difference graph Gdiff of G1 and G2 (assume a predicate `is_edge` is present for each 3 graphs - G1 G2 and Gdiff):

```
difference_graph:-
    is_edge_1(X,Y),
    not(is_edge_2(X,Y)),
    not(is_edge_diff(X,Y)),
    assertz(edge_diff(X,Y)),
    fail.
difference_graph.

is_edge(X,Y):-
    edge(X,Y);
    edge(Y,X).
```

Choose the most specific correct answer from below:

Select one:

- ☐ a. The sequence is correct as both G1 and G2 are entirely scanned once.
- ☐ b. The sequence is incorrect, as it is missing a ! after `not(is_edge_2(X,Y))`.
- ☐ c. The sequence is correct as G1 is entirely scanned and G2 is scanned for each edge in G1.
- ☐ d. The sequence is incorrect, as it is missing a ! after `not(is_edge_diff(X,Y))`.

To find the value for "init":

- you can think of the neutral element for that operation (ex. 0 for sum)
- or what is the result for the base case (ex. sum of all elements in [] is 0).

Summary / Cheat Sheet

% Template for complete list operations with backward recursion

```
pred([], "init").
pred([H|T], R):- pred(T, R1), R "=" R1 "op" H.
```

% Template for complete list operations with forward recursion

```
pred([], R, R).
pred([H|T], Acc, R):- Acc1 "=" Acc "op" H, pred(T, Acc1, R).
```

```
pred(L, R):- pred(L, "init", R).
```

% Template for complete list conditional operations

```
pred([], "init").
pred([H|T], R):- cond(...), !, pred(T, R1), R "=" R1 "op" H.
pred([H|T], R):- pred(T, R1), R "=" R1 "op".
```

% Template for deep list operations

```
pred([], "init").
pred([H|T], R):- atomic(H), !, pred(T, R1), R "=" R1 "op" H.
pred([H|T], R):- pred(H, R1), pred(T, R2), R "=" R1 "op" R2.
```

% Template for incomplete list operations

```
pred(L, "init"):-var(L),!.
pred([H|T], R):- pred(T, R1), R "=" R1 "op" H.
```

% Template for difference list (1st and 2nd arguments) operations

```
pred(S, E, "init"):-var(S),!,var(E),!,S==E,!.
pred([H|T], E, R):- pred(T, E, R1), R "=" R1 "op" H.
```

% Template for complete tree operations

```
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" K "op" NR.
pred(nil, "init").
```

% Template for

% complete tree conditional operations

```
pred(t(K,L,R), Result):-
    cond(...), !,
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" K "op" NR.
```

```
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" NR.
pred(nil, "init").
```

% Template for incomplete tree operations

```
pred(T, "init"):- var(T), !.
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result "=" NL "op" K "op" NR.
```

Summary / Cheat Sheet

% Templates for dynamic operations (2 stage processes)

```
:-dynamic p/1.
p(1).
...
```

% Template for failure driven loop

```
pred(_):- failure_driven_loop.
pred(L):- collect(L).
```

failure_driven_loop:-

```
    p(X), % / retract(p(X)),
    process(X),
    fail.
```

% Template for retract recursion loop

```
pred(_):- retract_recursion.
pred(L):- collect(L).
```

retract_recursion:-

```
    retract(p(X)), % / p(X), not(seen(X)), !, assert(seen(X)),
    process(X),
    retract_recursion.
```

```
process(X):- assert(q(X)).
```

```
collect([X|P]):-
    retract(visited_node(X)), !,
    collect(P).
collect([]).
```

% Template for graph path operations

```
path(X, Y, [X|Path]):- path(X, Y, [X], Path).
```

```
path(Y, Y, _, []).
```

```
path(X, Y, PPath, [Z|Path]):-
```

```
    is_edge(X, Z),
    not(member(Z, PPath)),
    path(Z, Y, [Z|PPath], Path).
```

```
edge(1,2).
```

```
% ...
```

```
is_edge(X,Y):- edge(X,Y);edge(Y,X).
```